

UNIVERSIDAD DE GUAYAQUIL

INGENERÍA DE SOFTWARE

MANUAL TÉCNICO

SISMOAPP

(nombre provisional)

PROYECTO DE VINCULACIÓN
EVALUACIÓN DE LA VULNERABILIDAD SÍSMICA
DE LAS EDIFICACIONES UBICADAS EN LAS PARROQUIAS
OLMEDO Y BOLIVAR DE LA CIUDAD DE GUAYAQUIL

GUAYAQUIL - ECUADOR

AGOSTO 2025

Tabla de contenido

MANUAL TÉCNICO.....	1
1. INTRODUCCIÓN.....	6
2. REQUERIMIENTOS TÉCNICOS (borrador)	8
3. ARQUITECTURA DEL SOFTWARE.....	10
3.1. Arquitectura de Alto Nivel.....	10
3.2. Componentes y Tecnologías	10
Front-end:	10
Back-end:.....	11
Infraestructura de Datos:	11
3.3. Flujo de Procesos	12
Detalle del Flujo de Peticiones:	12
4. HERRAMIENTAS UTILIZADAS PARA EL DESARROLLO	13
Android studio 2024.3.2.....	13
¿Por qué usarlo?.....	13
Beneficios	13
Flutter	13
¿Por qué usarlo?.....	14
¿Cómo funciona?	14
Supabase.....	14
¿Por qué usarlo?.....	14
¿Cómo funciona?	14
JAVA.....	14
¿Por qué usarlo?.....	15
¿Cómo funciona?	15
GITHUB.....	15
¿Por qué usarlo?.....	15
¿Cómo funciona?	16
NODE.JS	16
¿Por qué usarlo?.....	16
¿Cómo funciona?	17
EXPRESS.JS	17
Visual Studio Code.....	18

5.	INSTALACIÓN	19
	CONFIGURACIÓN DEL APLICATIVO	19
	Descarga de Android Studio	19
	Descarga Visual Studio Code	22
	Instalación e implemetacion de Flutter	23
	FASE 1: Descargar la carpeta Flutter	23
	FASE 2: Implementación de Flutter en el entorno de trabajo	28
	Fase 3: Verificación e instalación.....	33
	Descarga del servidor para conexión de la base de datos	40
6.	COMPILADOR DEL PROYECTO	44
	Opcion 1: Emulador de Android Studio	45
	Opcion 2: Sacar APK del Programa	46
	Opción 3: Terminal de Flutter para dispositivo físico	48
	Opción 4: Extensión ADB desde el entorno de Android.....	48
	Conexiones posteriores con wifi.....	50
7.	CASOS DE USO	52
	Tablas de Caso de uso	53
	Graficos de caso de uso	66
	Usuarios	66
	DIAGRAMA DE SECUENCIAS	67
8.	MÓDULOS	70
	8.1. Módulo de Autenticación de Usuario	70
	8.2. Módulo de Registro de Usuarios.....	70
	8.3. Módulo de Consulta de Edificaciones	71
	8.4. 8. Módulo de Registro de Edificios	71
9.	MODELO DE ENTIDAD RELACIÓN	74
10.	DICCIONARIO DE DATOS DEL MODELO ENTIDAD RELACIÓN.....	75
	1. Autenticación y seguridad.....	75
	2. Formulario.....	77
	3. Resultados	83
	4. Historial y Auditoría	86
11.	Diagrama de flujos	88
12.	PROTOTIPOS DE PANTALLAS DEL APLIACTIVO	90

13.	Anexo de sugerencias y soluciones a problemas	91
	Problemas de Sdk: Revisar la parte de variables de entorno.....	91
	Problemas con la compilación:	91

Tabla 1: Requisitos de Software	8
Tabla 2:Requerimiento mínimo del dispositivo móvil	8
Tabla 3 : Requerimiento mínimo de la servidor	9
Tabla 4:Requerimiento mínimo de la estación de desarrollo.....	9
Tabla 5: CU_01.....	53
Tabla 6: CU_02.....	54
Tabla 7: CU_03.....	55
Tabla 8: CU_04.....	56
Tabla 9: CU_05.....	57
Tabla 10: CU_06.....	58
Tabla 11: CU_07.....	59
Tabla 12: CU_08.....	60
Tabla 13: CU_09.....	61
Tabla 14: CU_10.....	62
Tabla 15: CU_11.....	63
Tabla 16: CU_12.....	64
Tabla 17: CU_13.....	65
Tabla 18: Usuarios	76
Tabla 19 Tokens de recuperación	77
Tabla 20: Formulario	79
Tabla 21: Inspecciones.....	80
Tabla 22: archivos_inspeccion.....	81
Tabla 23: comentarios_inspeccion.....	81
Tabla 24: catalogo_inspeccion.....	82
Tabla 25: matriz_puntuacion	83
Tabla 26: : resultados De lainspeccion	85
Tabla 27: Historial_auditoria	87

1. INTRODUCCIÓN

La evaluación de la vulnerabilidad sísmica de edificaciones es un proceso fundamental dentro de la gestión del riesgo, especialmente en zonas de alta sismicidad como las parroquias Olmedo y Bolívar de Guayaquil. Tradicionalmente, este proceso se realiza de manera manual, lo que implica una recopilación lenta de datos, una mayor probabilidad de errores y limitaciones al momento de consolidar resultados.

Para superar estas dificultades, se plantea el desarrollo de una **aplicación móvil denominada SismosApp**, basada en la metodología **FEMA P-154 (2015)** para la inspección rápida de edificaciones. El proyecto busca **digitalizar el formulario FEMA P-154**, permitiendo a los inspectores capturar información en campo de manera más ágil, precisa y segura, reduciendo la carga operativa y facilitando la generación de reportes estandarizados.

La solución propuesta no se limita únicamente a la digitalización de formularios, sino que integra un ecosistema tecnológico completo que combina:

- **Una aplicación móvil en Flutter** (Android/iOS), para la captura y visualización de datos en campo.
- **Un back-end desarrollado en Node.js con Express.js**, que actúa como intermediario entre la app y los servicios en la nube.
- **Supabase** como plataforma de base de datos relacional (PostgreSQL), autenticación segura mediante JSON Web Tokens (JWT), y almacenamiento de objetos (fotografías y evidencias gráficas).

Este enfoque garantiza no solo la **eficiencia en la recolección de datos**, sino también la **consistencia, escalabilidad y seguridad** de la información, aspectos esenciales para un sistema que busca apoyar decisiones críticas en la gestión de riesgos sísmicos.

La aplicación incorpora un **motor de cálculo (SL1 Engine)** encargado de procesar automáticamente los parámetros establecidos en la metodología FEMA P-154, entregando resultados cuantificables sobre el nivel de vulnerabilidad de las edificaciones. Estos resultados pueden convertirse en **reportes PDF/Excel**, generados en segundo plano mediante procesos automáticos (Background Workers), listos para ser consultados por usuarios o instituciones.

El proyecto representa un **avance significativo en la transformación digital de la evaluación de vulnerabilidad sísmica**, aportando una herramienta moderna, confiable y de impacto directo en la resiliencia urbana de Guayaquil.

El alcance del proyecto comprende:

- El diseño y desarrollo de una aplicación móvil con **formularios dinámicos** basados en FEMA P-154.
- La implementación de un **sistema de autenticación y roles de usuario** a través de Supabase Auth.
- El almacenamiento seguro y escalable de la información mediante **PostgreSQL** y Supabase Storage.
- La integración de un **motor de cálculo (SL1 Engine)** para procesar automáticamente las puntuaciones de vulnerabilidad.
- La generación de **reportes digitales en formato PDF/Excel**, gestionados mediante procesos en segundo plano.
- La disponibilidad de un **back-end RESTful** que permita la comunicación fluida entre la aplicación móvil y la base de datos.

Objetivo General

Desarrollar una aplicación móvil multiplataforma para la **evaluación rápida de vulnerabilidad sísmica de edificaciones** en Guayaquil, basada en la metodología FEMA P-154, que permita **digitalizar, almacenar, procesar y reportar datos estructurales y no estructurales** mediante un ecosistema tecnológico moderno y escalable.

Objetivos Específicos

1. **Digitalizar** el formulario FEMA P-154 en una aplicación móvil multiplataforma (Flutter).
2. **Implementar un sistema seguro de usuarios y roles** mediante Supabase Auth y JWT.
3. **Diseñar la base de datos en PostgreSQL** para gestionar de manera estructurada la información de inspecciones.
4. **Integrar un motor de cálculo (SL1 Engine)** que ejecute automáticamente las reglas de vulnerabilidad definidas por FEMA.
5. **Optimizar la captura de datos en campo** con soporte para fotografías y almacenamiento en Supabase Storage.
6. **Desarrollar un módulo de generación de reportes digitales (PDF/Excel)** en procesos asíncronos gestionados por Background Workers.
7. **Facilitar la consulta y análisis de resultados** a través de la app móvil y la API RESTful.
8. **Validar la aplicación en campo** en edificaciones seleccionadas de las parroquias Olmedo y Bolívar, asegurando su usabilidad y precisión.

2. REQUERIMIENTOS TÉCNICOS (borrador)

Requerimientos mínimos de software

Componente	Mínimo	Recomendado
SO móvil	Android 10 / iOS 13	Android 12 / iOS 15
Framework móvil	Flutter 3.x (Dart SDK)	Última versión estable de Flutter
IDE	Android Studio Flamingo / VS Code	Android Studio Giraffe + extensiones
Backend	Node.js 16.x + Express	Node.js 18.x LTS + Express
Base de datos	PostgreSQL 12 (Supabase)	PostgreSQL 14+ (Supabase)
Autenticación	Supabase Auth con JWT	Supabase Auth + MFA
Almacenamiento	Supabase Storage	Supabase Storage + backup automático
Reportes	pdfkit / exceljs	pdfkit / exceljs + workers en contenedores
Control de versiones	Git	GitHub/GitLab con CI/CD

Tabla 1: Requisitos de Software

Requerimientos mínimos de hardware

2. Requisitos de hardware

a) Dispositivo móvil (usuarios en campo)

Requisito	Mínimo	Recomendado
Procesador	Quad-core 1.8 GHz	Octa-core $\geq$ 2.0 GHz
RAM	3 GB	6 GB
Almacenamiento	32 GB	128 GB
Cámara	8 MP	12 MP+
Conectividad	3G / Wi-Fi	4G LTE o superior
Pantalla	5.5" HD	6"+ Full HD

Tabla 2: Requerimiento mínimo del dispositivo móvil

b) Servidor / Back-end

Requisito	Mínimo	Recomendado
CPU	2 núcleos	4 núcleos
RAM	2 GB	8 GB
Almacenamiento	20 GB SSD	50 GB SSD
SO	Ubuntu 20.04 LTS	Ubuntu 22.04 LTS
Servicios	Supabase (Auth, Storage, Postgres)	Supabase + backups automáticos

*Tabla 3 : Requerimiento mínimo de la servidor***c) Estación de desarrollo**

Requisito	Mínimo	Recomendado
CPU	Intel i3 8va gen / Ryzen 3	Intel i5 10ma gen / Ryzen 5
RAM	8 GB	16 GB
Disco	128 GB SSD	256 GB SSD+
GPU	Integrada	Dedicada o integrada con soporte emulador
SO	Windows 10 / Ubuntu 20.04 / macOS 11	Windows 11 / Ubuntu 22.04 / macOS 13
Extras	SDK Android, Node.js, Git	SDK Android + Docker + PostgreSQL client

Tabla 4:Requerimiento mínimo de la estación de desarrollo

3. ARQUITECTURA DEL SOFTWARE

Arquitectura técnica propuesta para el desarrollo de un sistema de evaluación de vulnerabilidad sísmica. La solución se basa en una arquitectura de microservicios y se compone de tres elementos principales: una aplicación móvil como front-end, un back-end RESTful en la nube, y una plataforma de servicios de bases de datos y almacenamiento.

3.1. Arquitectura de Alto Nivel

La arquitectura general del sistema se divide en tres capas interconectadas:

1. **Capa de Presentación (Front-end):** Una aplicación móvil desarrollada con Flutter, compatible con iOS y Android. Su función principal es la captura de datos a través de formularios dinámicos y la presentación de información al usuario.
2. **Capa de Lógica de Negocio (Back-end):** Un servidor de aplicaciones desarrollado con Node.js y Express.js, que expone una REST API para la comunicación con el front-end. Esta capa contiene el motor de cálculo de vulnerabilidad y gestiona las tareas en segundo plano.
3. **Capa de Datos y Almacenamiento:** Se utiliza Supabase como plataforma de servicios.
 - a. PostgreSQL: Almacena los datos estructurados, incluyendo la información de los levantamientos y los reportes generados.
 - b. Supabase Storage: Un servicio de almacenamiento de objetos diseñado para guardar archivos binarios, como las fotos capturadas durante las inspecciones.
 - c. Supabase Auth: Un servicio de autenticación que gestiona la seguridad y el acceso de los usuarios mediante JSON Web Tokens (JWT).

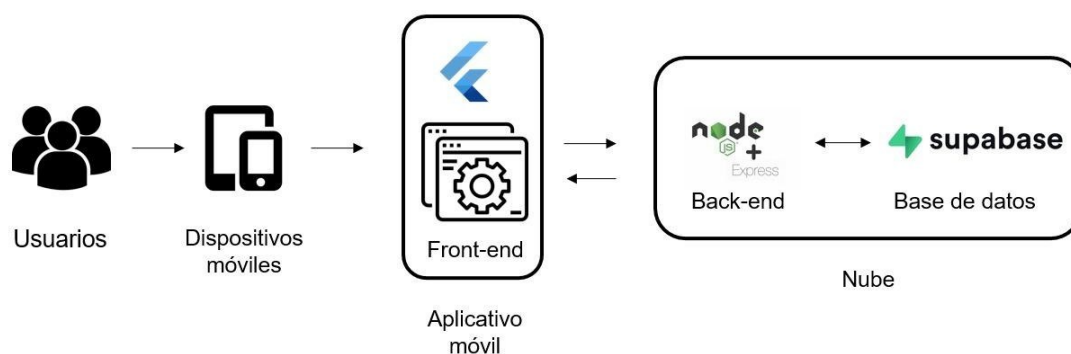


Ilustración 1: Arquitectura de del proyecto

3.2. Componentes y Tecnologías

Front-end:

1. Tecnología: Flutter
2. Función:
 - a. Interfaz de usuario (UI) para la interacción del usuario.

- b. Form Builder para la creación y gestión de formularios de levantamiento de datos.
- c. Cache para el almacenamiento temporal de datos y mejora del rendimiento.
- d. Manejo de la autenticación y autorización del usuario a través de JWT.

Back-end:

1. Tecnología: Node.js, Express.js
2. Función:
 - a. REST API: Sirve como punto de entrada para todas las solicitudes del front-end.
 - b. SL1 Engine: Un módulo de lógica de negocio que implementa las reglas de evaluación de vulnerabilidad sísmica FEMA P-154.
 - c. Report Worker API: Un servicio de API para gestionar la cola de tareas de generación de reportes.
 - d. Background Workers: Procesos que se ejecutan en segundo plano para tareas que requieren tiempo, como la generación de archivos PDF/Excel.

Infraestructura de Datos:

1. Tecnología: Supabase
2. Servicios Utilizados:
 - a. Postgres: Base de datos principal para la persistencia de datos relacionales.
 - b. Storage: Para almacenar las fotos asociadas a los levantamientos.
 - c. Auth: Provee un sistema de autenticación de usuarios y gestión de tokens.

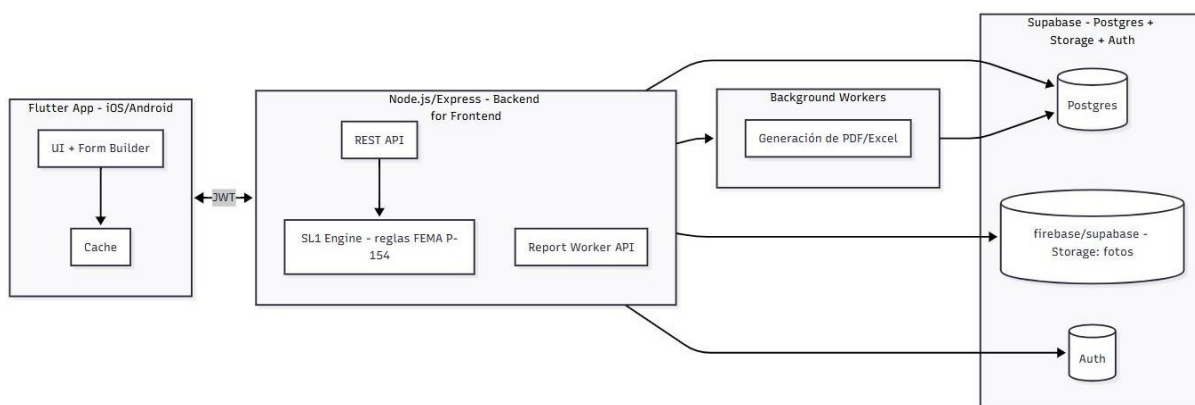


Ilustración 2: Componentes y Tecnologías

3.3. Flujo de Procesos

El siguiente diagrama de secuencia ilustra el flujo de trabajo para la captura de datos y la generación de reportes:

Detalle del Flujo de Peticiones:

1. Autenticación: El usuario se autentica a través de la API de Auth para obtener un JWT.
2. Envío de Datos: La aplicación móvil envía los datos del formulario (metadata) a la API de encuestas (/surveys). El back-end inserta estos datos en la tabla de encuestas en Postgres.
3. Subida de Fotos: Las fotos se suben directamente a Supabase Storage, y la API del back-end actualiza las URLs de las fotos en el registro de la encuesta.
4. Cálculo de Puntuación: La aplicación móvil solicita el cálculo de la puntuación SL1. El back-end ejecuta el SL1 Engine y actualiza la base de datos con el resultado.
5. Generación de Reporte: El usuario solicita un reporte, lo que encola una tarea en los Background Workers. Estos workers recuperan la información de Postgres y Storage, generan el reporte (PDF/Excel) y lo almacenan. El back-end actualiza el registro de la encuesta con el ID del reporte.
6. Consulta de Reportes: La aplicación móvil puede obtener los reportes o estadísticas a través de la API, que a su vez consulta la base de datos Postgres.

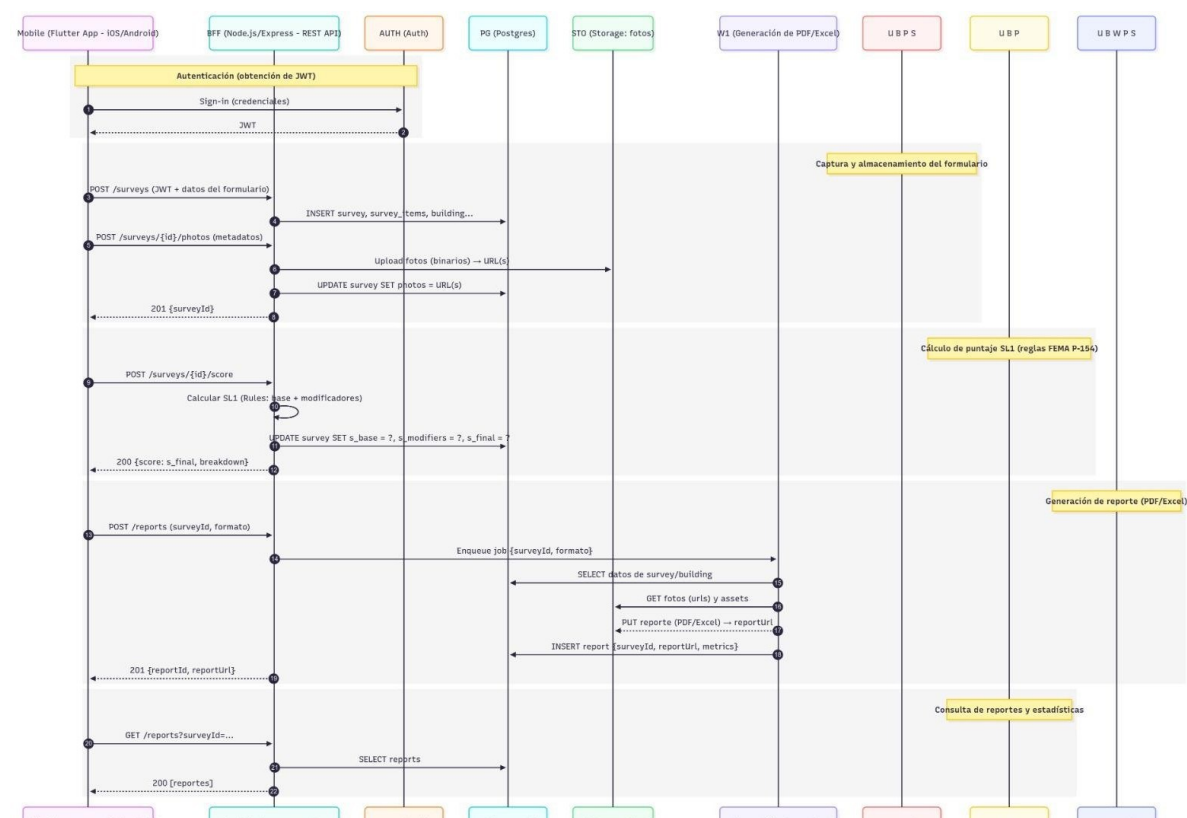


Ilustración 3.: Flujo de Procesos

4. HERRAMIENTAS UTILIZADAS PARA EL DESARROLLO

Android studio 2024.3.2

Android Studio es el entorno de desarrollo integrado (IDE) oficial para el desarrollo de aplicaciones Android. Desarrollado por Google, ofrece herramientas poderosas y características para crear aplicaciones móviles y dispositivos Android. Basado en IntelliJ IDEA, permite a los desarrolladores diseñar, probar y depurar aplicaciones para teléfonos, tabletas, wearables, televisores y dispositivos de auto.

¿Por qué usarlo?

- **Desarrollo optimizado para Android:** Es la herramienta más completa para desarrollar aplicaciones Android, con características específicas para este sistema operativo.
- **Compatibilidad con lenguajes modernos:** Soporta Java, Kotlin y otros lenguajes usados para aplicaciones Android.
- **Integración con herramientas de Google:** Está diseñado para funcionar de manera fluida con las APIs y servicios de Google, como Firebase y Google Maps.
- **Diseño eficiente de interfaces:** Permite crear interfaces de usuario mediante un editor visual (Layout Editor) y proporciona acceso a plantillas de diseño.

Beneficios

- Emuladores de dispositivo Android: Facilita las pruebas de aplicaciones sin necesidad de un dispositivo físico.
- Herramientas de depuración: Ofrece herramientas avanzadas para depurar y optimizar el código, como el Android Debugger (ADB) y análisis de rendimiento.
- Plantillas de código y ejemplos: Proporciona plantillas preconfiguradas para crear rápidamente aplicaciones Android con diseños y funcionalidades comunes.
- Compatibilidad con Kotlin: Es compatible de forma nativa con Kotlin, el lenguaje preferido para el desarrollo Android.
- Soporte para pruebas unitarias y de interfaz: Facilita la ejecución de pruebas para asegurar la calidad de las aplicaciones.
- Actualizaciones constantes: Recibe actualizaciones frecuentes con nuevas características, herramientas y mejoras de seguridad.

Flutter

Flutter es un framework de desarrollo de aplicaciones creado por Google. Te permite crear aplicaciones móviles, web, de escritorio y embebidas usando una sola base de código.

¿Por qué usarlo?

- Una sola base de código para múltiples plataformas (¡no tienes que programar separado para Android e iOS!).
- Interfaz de usuario muy fluida y personalizable.
- Hot Reload: puedes ver los cambios en tiempo real mientras desarrollas.
- Respaldo por Google, usado en apps como Google Pay y eBay Motors.

¿Cómo funciona?

- Usa Dart para la lógica.
- Tiene su propio motor de renderizado gráfico (no usa componentes nativos como React Native, sino que dibuja todo desde cero).

Supabase

Supabase es una plataforma de desarrollo backend como servicio (BaaS) de código abierto. Te permite crear la infraestructura de base de datos, autenticación, almacenamiento y APIs sin tener que programar todo desde cero.

¿Por qué usarlo?

- Todo en uno: Base de datos, autenticación, almacenamiento y APIs listos para usar.
- Autenticación integrada: Manejo de usuarios con seguridad mediante JWT.
- PostgreSQL gestionado: Base de datos robusta y escalable sin tener que instalar nada local.
- Storage: Guardado de archivos (fotos, PDFs, videos) vinculado a la base de datos.
- Rápida implementación: En minutos tienes un backend funcional.
- Código abierto y respaldado por la comunidad.

¿Cómo funciona?

- Base de datos (Postgres): Permite crear tablas y relaciones con control de acceso por usuario (RLS).
- Auth: Sistema de autenticación (correo, redes sociales, SSO).
- Storage: Almacenamiento de archivos con control de acceso.
- APIs automáticas: Cada tabla tiene una API REST lista para consultar y modificar datos.
- SDKs: Tiene librerías para integrarse fácilmente con apps en Flutter, Node.js, Python, etc.
-

JAVA

Java 17 es una versión LTS del lenguaje y plataforma Java, altamente estable, segura y optimizada, fundamental para la compilación nativa de aplicaciones Android en Flutter y Gradle.

¿Por qué usarlo?

- **LTS oficialmente soportado:** Java 17 es la versión Long-Term Support vigente, con actualizaciones de seguridad garantizadas hasta 2029, asegurando estabilidad en entornos de producción.
- **Compatibilidad total con Flutter 3.x, Android Studio y AGP 8.x:** Evita incompatibilidades de compilación, alinear la toolchain de Flutter con el ecosistema Android moderno.
- **Optimización de rendimiento:** Introduce recolectores de basura de baja latencia (ZGC y Shenandoah), que reducen stop-the-world pauses y mejoran la experiencia en procesos intensivos como builds y pruebas automatizadas.
- **Nuevas características del lenguaje:** Incorpora Records, Sealed Classes y Pattern Matching, que disminuyen boilerplate y permiten una modelación más expresiva y segura de datos.
- **Seguridad reforzada:** Mitiga vulnerabilidades críticas (CVEs) presentes en versiones anteriores como Java 8 y 11, fortaleciendo la capa de ejecución en sistemas sensibles.
- **Interoperabilidad Kotlin/Java:** Garantiza plena integración con plugins nativos de Android y librerías legacy escritas en Java, coexistiendo con el ecosistema Kotlin en Android Studio.

¿Cómo funciona?

- **JVM 17 ejecuta bytecode multiplataforma:** Utiliza JIT (Just-In-Time) y soporte para AOT (Ahead-Of-Time) en Android ART, optimizando bytecode en tiempo de ejecución y en instalación.
- **Gradle + Android Studio:** La compilación se alinea mediante la propiedad `org.gradle.java.home=jdk-17`, garantizando que la toolchain use el mismo JDK en entornos locales y de CI/CD.
- **AGP 8.x con D8/R8:** Convierte el bytecode Java/Kotlin en DEX (Dalvik Executable) y aplica optimización/minificación, reduciendo tamaño final del APK/AAB y mejorando tiempo de carga.
- **JVM TI y Flight Recorder:** Proporcionan capacidades avanzadas de depuración, profiling y monitoreo en Android Studio Profiler, permitiendo análisis de rendimiento granular en tiempo real.
- **Configuración centralizada:** La definición de la versión de Java se establece en `gradle.properties` y variables de entorno (`.bashrc`, `.zshrc`, pipelines CI/CD), unificando la toolchain para desarrollo y despliegue continuo.

GITHUB

GitHub es una plataforma colaborativa basada en Git que centraliza control de versiones, integración continua (CI/CD) y gestión de proyectos, garantizando trazabilidad, seguridad y automatización en equipos de desarrollo.

¿Por qué usarlo?

- **Control de versiones distribuido:** Basado en Git, permite mantener el historial completo de cambios en repositorios locales y remotos, soportando ramas paralelas y revertibilidad sin dependencia de un servidor central activo.

- **Flujo colaborativo escalable:** Los Pull Requests, revisiones de código (Code Review) e Issues facilitan la integración continua y la trazabilidad de decisiones técnicas, asegurando calidad en proyectos multi-equipo.
- **CI/CD nativo con GitHub Actions:** Proporciona pipelines automatizados para compilar proyectos Flutter, ejecutar pruebas unitarias/integración y desplegar artefactos (APK, AAB, IPA) en entornos controlados. (Nota: la generación y firma de IPA requiere runners macOS o integración con servicios externos como Codemagic o Fastlane).
- **Gestión de secretos y entornos:** Permite configurar variables cifradas (tokens de acceso, credenciales de keystore Android, certificados iOS) garantizando seguridad en la ejecución de workflows de CI/CD.
- **Integración con ecosistemas externos:** A través de Webhooks y GitHub Actions, se puede notificar a servicios como Supabase Edge Functions, Slack o Discord, desencadenando despliegues automáticos o alertas en tiempo real.
- **Comunidad y transparencia:** Issues públicos, wikis, GitHub Projects (tableros Kanban) y discusiones permiten la gestión ágil de tareas y la documentación colaborativa con visibilidad global.

¿Cómo funciona?

- **Repositorio Git remoto en la nube:** El flujo estándar incluye commits en local y sincronización mediante git push origin main, replicando cambios, etiquetas (tags) y versiones en GitHub.
- **Workflows YAML en GitHub Actions:** Definidos en .github/workflows/, permiten orquestar pipelines que incluyen comandos como flutter build apk --release, ejecución de tests automatizados y análisis estático con SonarCloud.
- **Protección de ramas y PR checks:** Se configuran reglas de branch protection que exigen pasar pruebas (lint, tests, build) antes de permitir merges a ramas críticas como main o release.
- **Releases y artefactos:** GitHub Releases almacena paquetes versionados (APK, AAB, IPA) y notas de versión. Los artefactos pueden ser firmados digitalmente con SHA-256 a través de workflows configurados en Actions.
- **Webhooks REST y APIs:** GitHub expone eventos HTTP POST para integrarse con sistemas externos. Esto permite, por ejemplo, que al cerrar un PR o fallar un pipeline, se ejecute una función en Supabase, un microservicio backend o una notificación en un canal de monitoreo.

NODE.JS

Node.js es un entorno de ejecución de JavaScript del lado del servidor, construido sobre el motor V8 de Google Chrome. Permite ejecutar código JavaScript fuera del navegador, habilitando el desarrollo de aplicaciones backend, APIs y servicios web.

¿Por qué usarlo?

- **Lenguaje unificado:** Utiliza JavaScript tanto en frontend como backend, simplificando el desarrollo y mantenimiento del código.

- **Arquitectura asíncrona:** Event-driven y non-blocking I/O, optimizado para aplicaciones que requieren manejar múltiples conexiones simultáneas como las inspecciones sísmicas en tiempo real.
- **Ecosistema NPM rico:** Acceso a miles de librerías especializadas para integración con bases de datos, procesamiento de archivos, generación de reportes y servicios en la nube.
- **Rendimiento escalable:** Manejo eficiente de múltiples inspectores trabajando concurrentemente en diferentes edificaciones.
- **Compatibilidad cloud:** Integración nativa con servicios modernos como Supabase, facilitando el despliegue y escalabilidad.

¿Cómo funciona?

Node.js ejecuta JavaScript del lado del servidor mediante el motor V8, procesando peticiones HTTP de la aplicación móvil Flutter, gestionando conexiones con la base de datos PostgreSQL de Supabase, y coordinando procesos en segundo plano como la generación de reportes PDF/Excel. Su arquitectura basada en eventos permite manejar múltiples operaciones simultáneas sin bloquear el hilo principal de ejecución.

EXPRESS.JS

Express.js es un framework web minimalista y flexible para Node.js que proporciona herramientas robustas para crear APIs RESTful y aplicaciones web. Es la biblioteca más utilizada para desarrollo de servidores HTTP en el ecosistema Node.js.

¿Por qué usarlo?

- **Sistema de routing avanzado:** Gestión eficiente de rutas HTTP (GET, POST, PUT, DELETE) para diferentes endpoints de la API del sistema de inspecciones sísmicas.
- **Middleware robusto:** Procesamiento de peticiones en capas secuenciales, permitiendo implementar autenticación JWT, validación de datos FEMA P-154, logging de auditoría y manejo de errores.
- **Flexibilidad arquitectónica:** Permite integrar múltiples bibliotecas especializadas según las necesidades específicas del proyecto, como el motor de cálculo SL1 Engine.
- **Manejo de archivos:** Procesamiento eficiente de uploads de fotografías y documentos adjuntos a las inspecciones.
- **Gestión de errores centralizada:** Sistema unificado para capturar, procesar y reportar errores de manera consistente.

¿Cómo funciona?

Express.js actúa como el núcleo del servidor, recibiendo peticiones HTTP de la aplicación Flutter, procesándolas a través de middleware personalizado para validación y autenticación, ejecutando la lógica de negocio correspondiente (cálculos FEMA, almacenamiento de datos), interactuando con Supabase para persistencia de información, y devolviendo respuestas estructuradas en formato JSON. Su diseño modular permite separar claramente las responsabilidades entre routing, middleware, controladores y servicios.

Visual Studio Code

Visual Studio Code (VS Code) es un editor de código fuente gratuito y de código abierto desarrollado por Microsoft. Es multiplataforma (Windows, macOS, Linux) y está optimizado para el desarrollo web moderno y aplicaciones en la nube.

¿Por qué usarlo?

Ventajas específicas para Node.js/Express.js:

- **Soporte nativo para JavaScript/TypeScript:** Incluye IntelliSense, autocompletado y detección de errores en tiempo real para Node.js.
- **Terminal integrado:** Permite ejecutar comandos del servidor (npm i, node --watch server.js) sin salir del editor.
- **Depuración avanzada:** Configuración sencilla de breakpoints y debugging para aplicaciones Node.js.
- **Extensiones especializadas:**
 - Node.js Extension Pack
 - REST Client para probar APIs
 - Thunder Client como alternativa a Postman
- **Control de versiones integrado:** Git incorporado para gestionar el repositorio del backend.
- **Gestión de proyectos:** Workspace que permite manejar múltiples carpetas (frontend Flutter + backend Node.js) simultáneamente.

5. INSTALACIÓN

CONFIGURACIÓN DEL APLICATIVO

Descarga de Android Studio

1. Ingresar y descargar el instalador de Android Studio desde <https://developer.android.com/studio/archive>
2. Seleccionar la versión recomendada (Android Studio 2024.3.2)
3. Descargar el archivo de instalación.

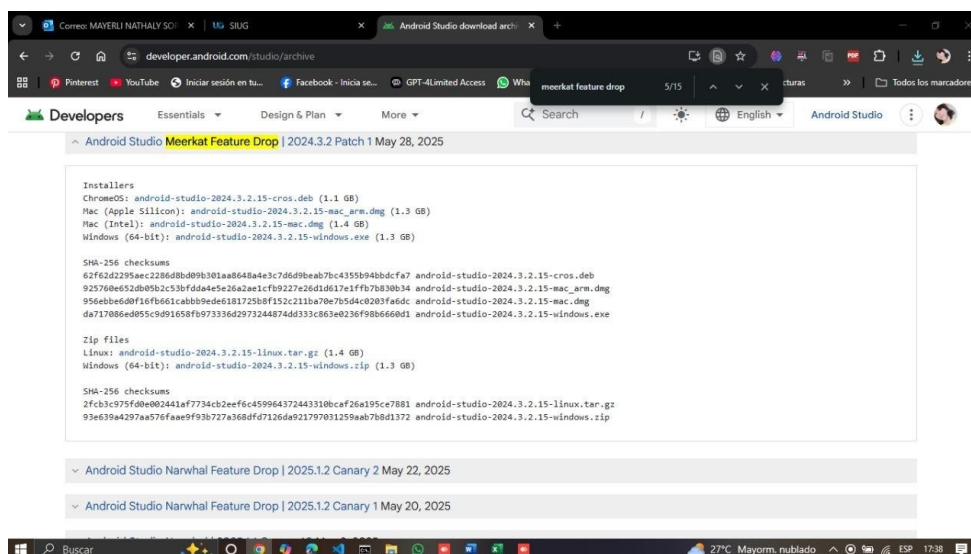


Ilustración 4: Descarga de Android Studio

Instalador de Android Studio

1. Ejecutar el instalador descargado.
2. Seleccionar la ruta por defecto.

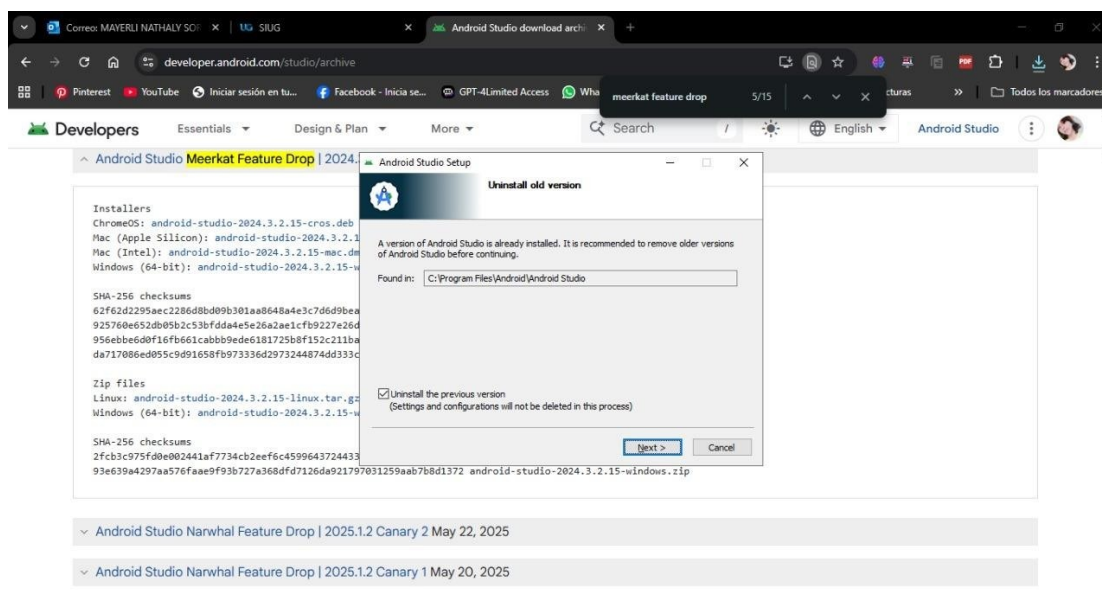


Ilustración 5. Instalación de : Android Studio

3. Asegurarse de marcar:

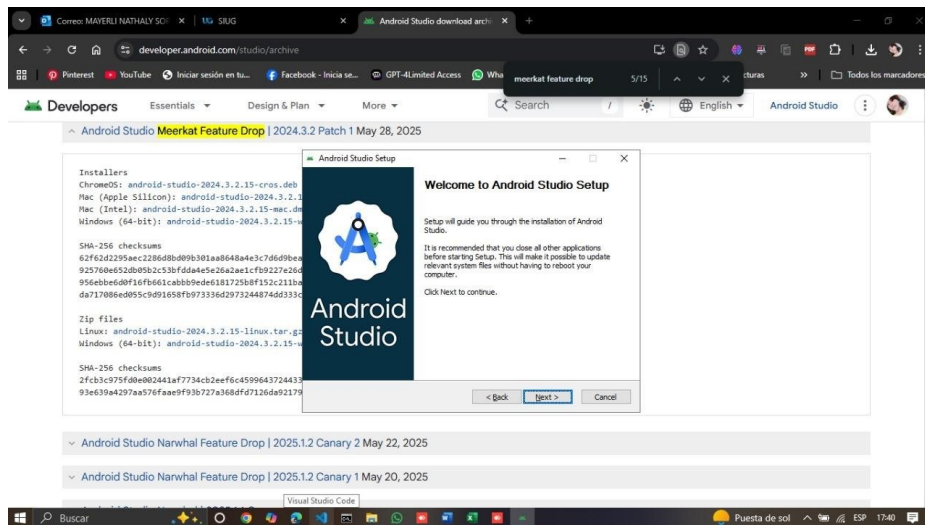


Ilustración 6: Instalación de : Android Studio

- **Android SDK**
- **AVD (Android Virtual Device)**

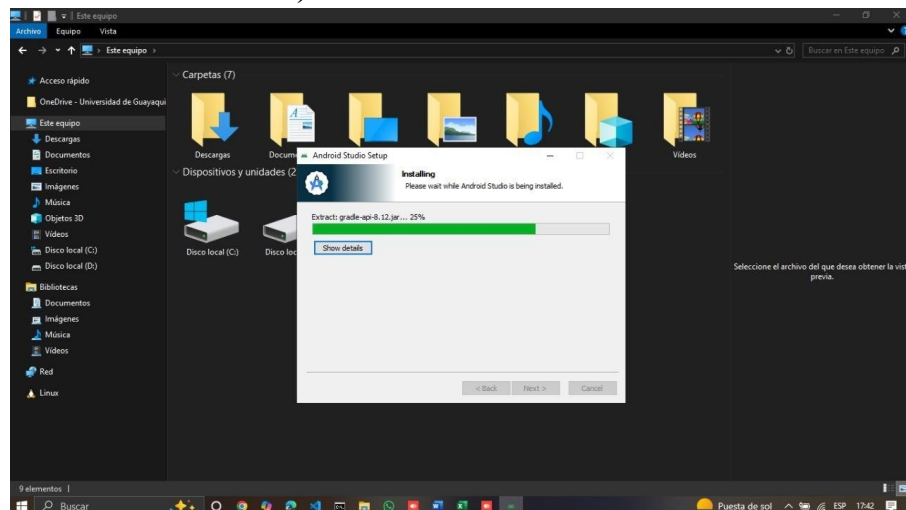
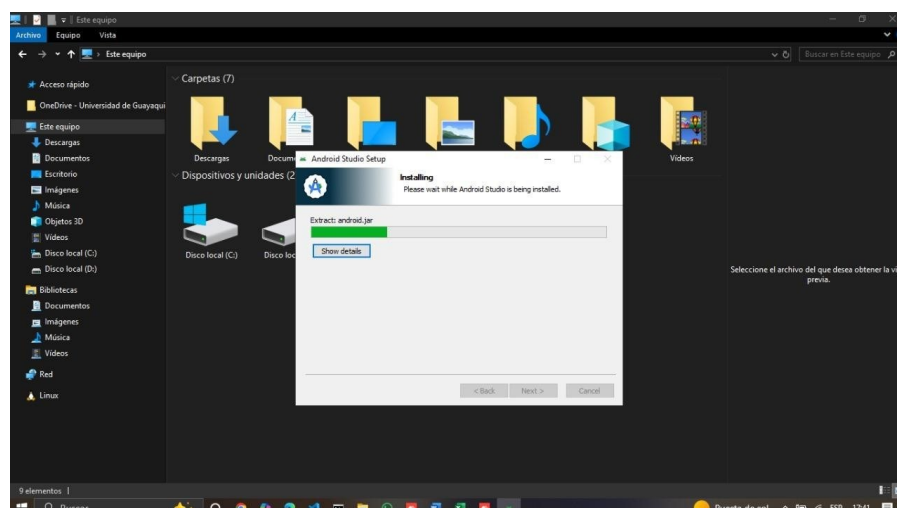
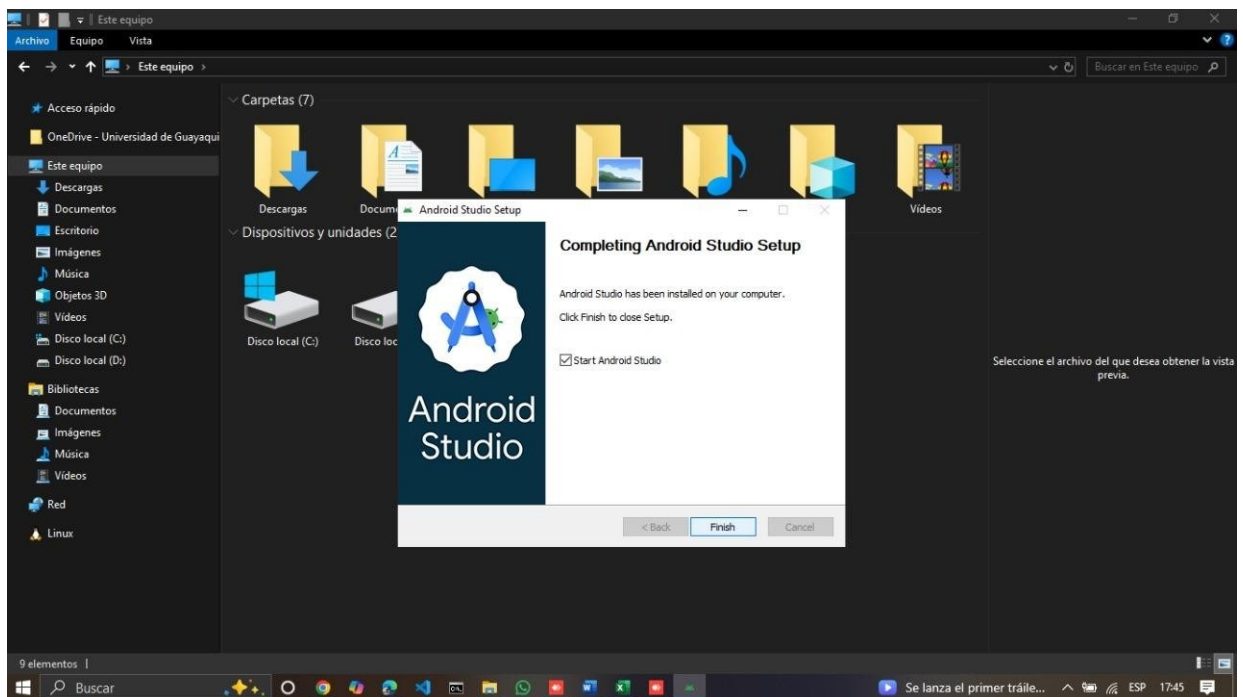
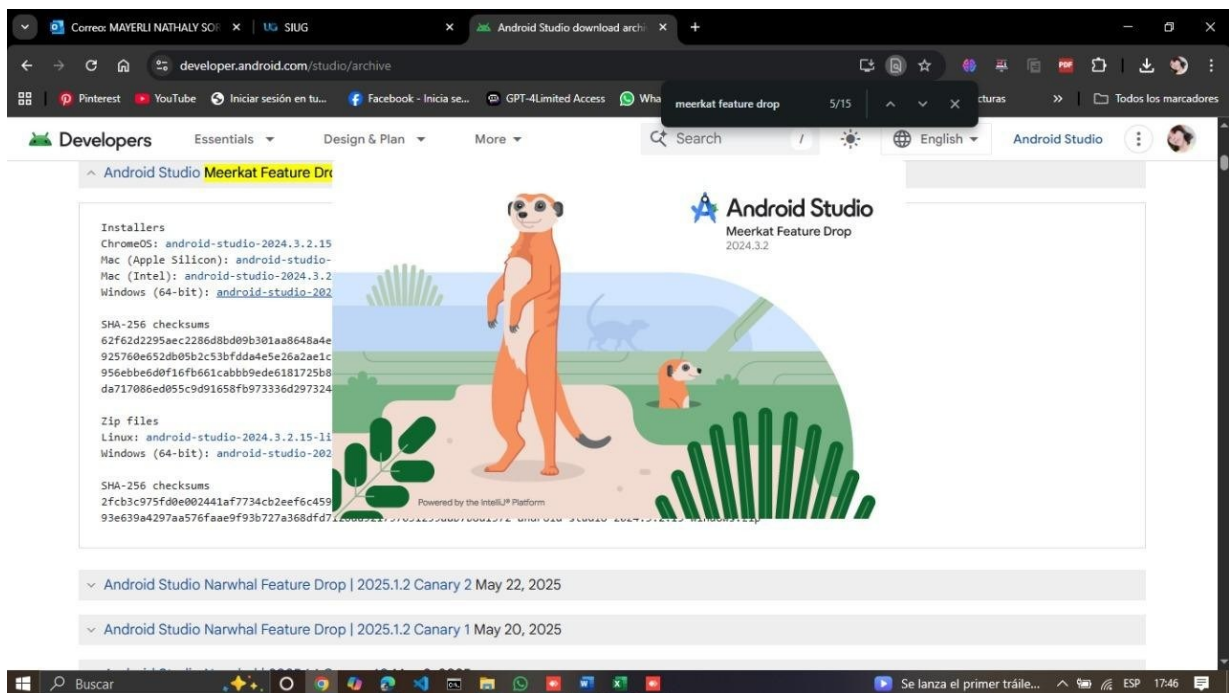


Ilustración 7: Instalación de : Android Studio

- **Herramientas de compilación**

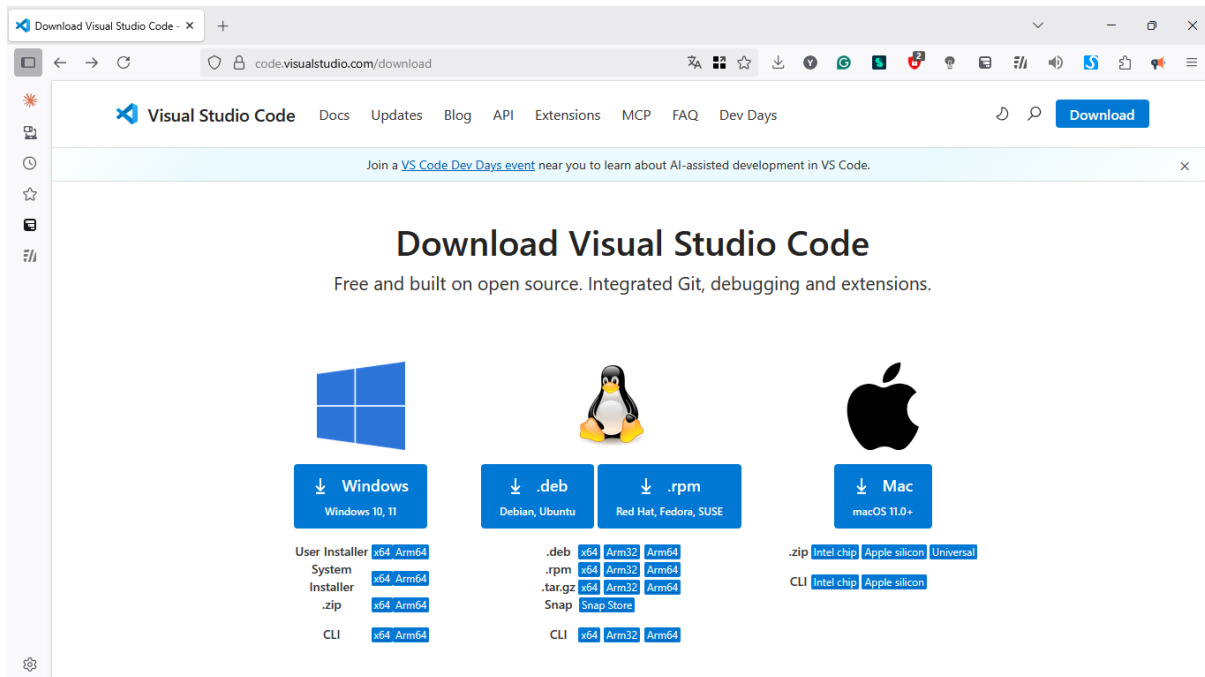


1. Finalizar la instalación y abrir Android Studio.

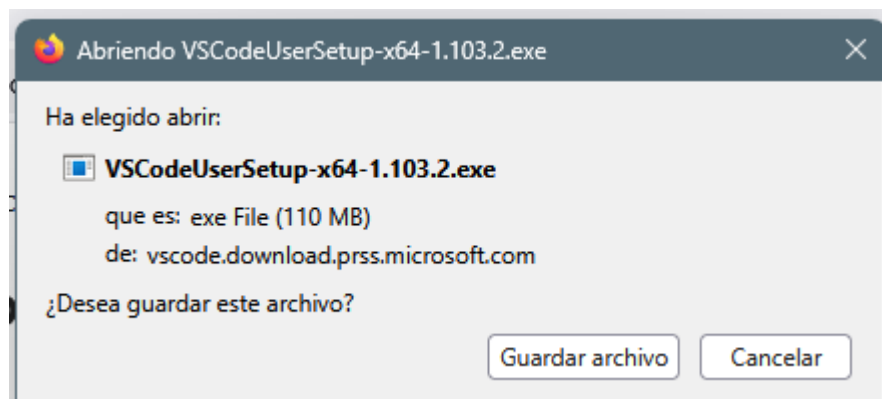


Descarga Visual Studio Code

1. Descarga la versión más actual del aplicativo



2. Ejecuta el .exe



4. Acepta y sigue hasta que se instale

Instalación e implemetacion de Flutter

FASE 1: Descargar la carpeta Flutter

Se descarga la carpeta Flutter, esta acción se puede realizar de diversas formas según su página oficial <https://docs.flutter.dev> algunas de ellas se explicaran a continuación:

- a. Opción 1: Descarga del repositorio de Flutter a través de Git.

Advertencia: Se debe tener instalado Git : <https://git-scm.com/>

Paso 1: Abre GIT BASH..

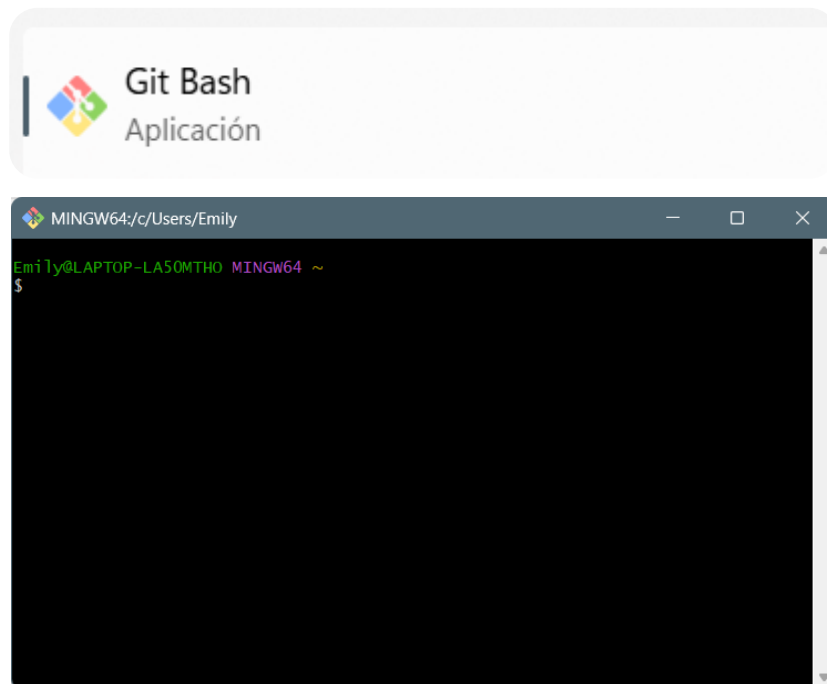


Ilustración 8: Instalacion Flutter

Paso 2: Una vez abierta, se debe colocar el siguiente comando en la pantalla:

`$ git clone -b main https://github.com/flutter/flutter.git`

y luego se debe ejecutar (se puede hacer ENTER para ejecutarlo). Se debe esperar a que se descargue completamente, debe ser similar a la siguiente pantalla:

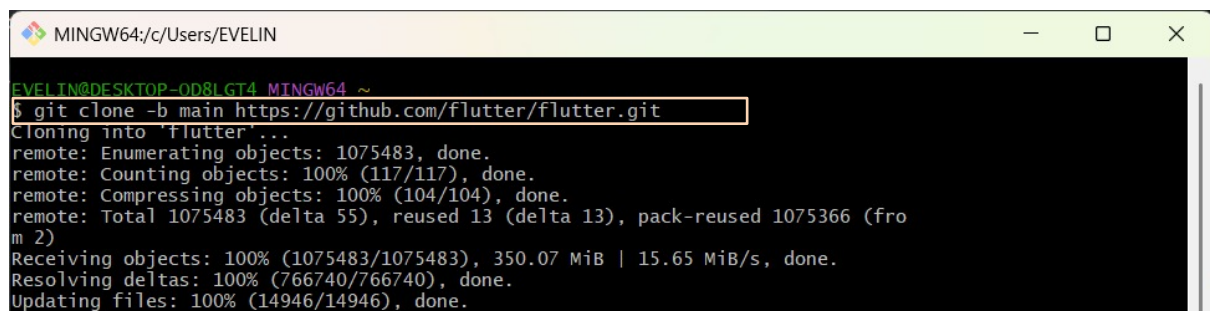


Ilustración 9: Instalacion Flutter

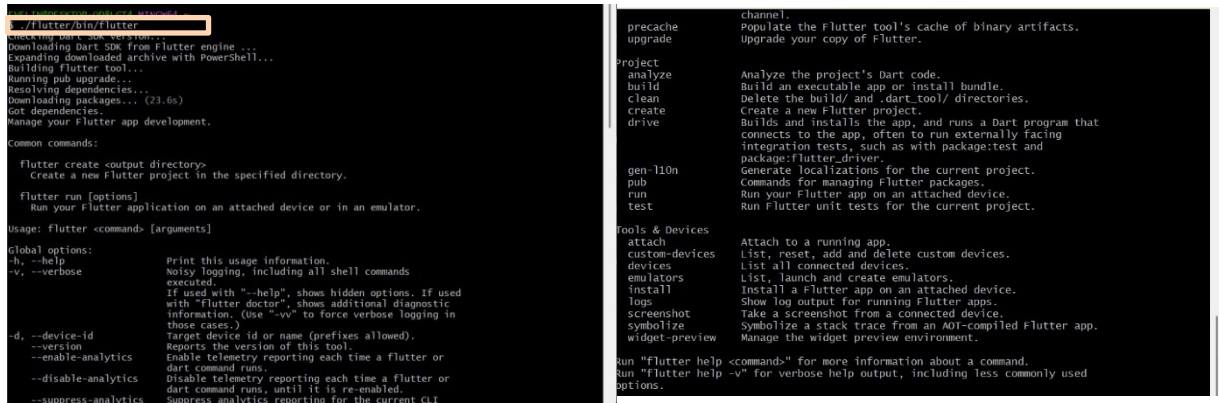
Este repositorio se descargará de manera automática en el directorio:

C:\Users\(\nombre del usuario)

Paso 3: Verifica que **Dart se haya configurado correctamente**, similar a la acción anterior se coloca este comando en el cmd y se ejecuta:

`$ ./flutter/bin/flutter`

La información que se debe presentar en la pantalla debe ser similar a las imágenes a continuación:



```

$ ./flutter/bin/flutter
Downloading Dart SDK from Flutter engine ...
Expanding downloaded archive with PowerShell...
Building Flutter tool...
Running pub upgrade...
Resolving dependencies...
Downloading packages... (23.6s)
Got dependencies.
Manage your Flutter app development.

Common commands:

Flutter create <output directory>
  Create a new Flutter project in the specified directory.

Flutter run [options]
  Run your Flutter application on an attached device or in an emulator.

Usage: flutter <command> [arguments]

Global options:
-h, --help          Print this usage information.
-v, --verbose       Noisy logging, including all shell commands
                    executed.
                    If used with "--help", shows hidden options. If used
                    with "flutter doctor", shows additional diagnostic
                    information. (Use "-vv" to force verbose logging in
                    those cases.)
-d, --device-id     Target device id or name (prefixes allowed).
                    Reports the version of this tool.
--enable-analytics  Enable telemetry reporting each time a flutter or
                    dart command runs.
--disable-analytics Disable telemetry reporting each time a flutter or
                    dart command runs, until it is re-enabled.
--suppress-analytics Suppress analytics reporting for the current CLI

Commands:
channel
  Populate the Flutter tool's cache of binary artifacts.
  Upgrade your copy of Flutter.

Project
analyze
  Analyze the project's Dart code.
build
  Build an executable app or install bundle.
clean
  Delete the build/ and .dart_tool/ directories.
create
  Create a new Flutter project.
drive
  Builds and installs the app, and runs a Dart program that
  connects to the app, often to run externally facing
  integration tests, such as with package:test and
  package:flutter_driver.
gen-l10n
  Generate localizations for the current project.
pub
  Commands for managing Flutter packages.
run
  Run your Flutter app on an attached device.
test
  Run Flutter unit tests for the current project.

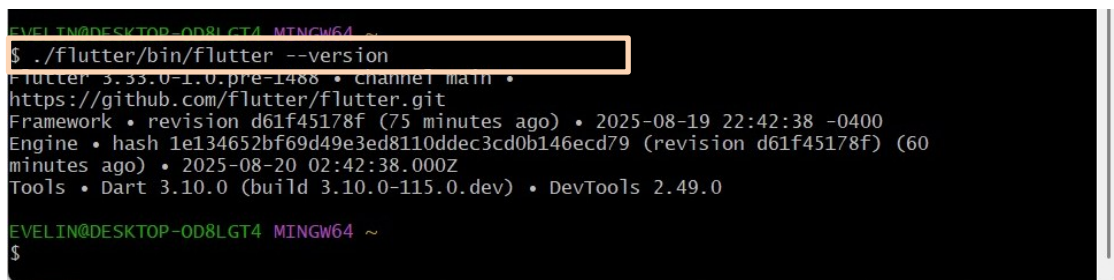
Tools & Devices
attach
  Attach to a running app.
custom-devices
  List, reset, add and delete custom devices.
devices
  List all connected devices.
emulators
  List, launch and create emulators.
install
  Install a Flutter app on an attached device.
logs
  Show log output for running Flutter apps.
screenshot
  Take a screenshot from a connected device.
symbolize
  Symbolize a stack trace from an AOT-compiled Flutter app.
widget-preview
  Manage the widget preview environment.

Run "flutter help <command>" for more information about a command.
Run "flutter help -v" for verbose help output, including less commonly used
options.
  
```

Ilustración 10: Instalación Flutter

Paso 4: Si se desea **verificar que versión** de Flutter y Dart se tiene, se puede colocar el siguiente comando:

`$ ./flutter/bin/flutter --version`



```

EVELIN@DESKTOP-OD8LGT4 MINGW64 ~
$ ./Flutter/bin/flutter --version
Flutter 3.33.0-1.0.pre-1468 • Channel main •
https://github.com/flutter/flutter.git
Framework • revision d61f45178f (75 minutes ago) • 2025-08-19 22:42:38 -0400
Engine • hash 1e134652bf69d49e3ed8110dddec3cd0b146ecd79 (revision d61f45178f) (60
minutes ago) • 2025-08-20 02:42:38.000Z
Tools • Dart 3.10.0 (build 3.10.0-115.0.dev) • DevTools 2.49.0

EVELIN@DESKTOP-OD8LGT4 MINGW64 ~
$
  
```

Ilustración 11: Instalación Flutter

- b. **Opción 2:** Descarga de la carpeta de Flutter a través de la página oficial <https://docs.flutter.dev>

Paso 1: En el apartado de **Install manually** de la página oficial se va la sección de **Install and set up Flutter**:

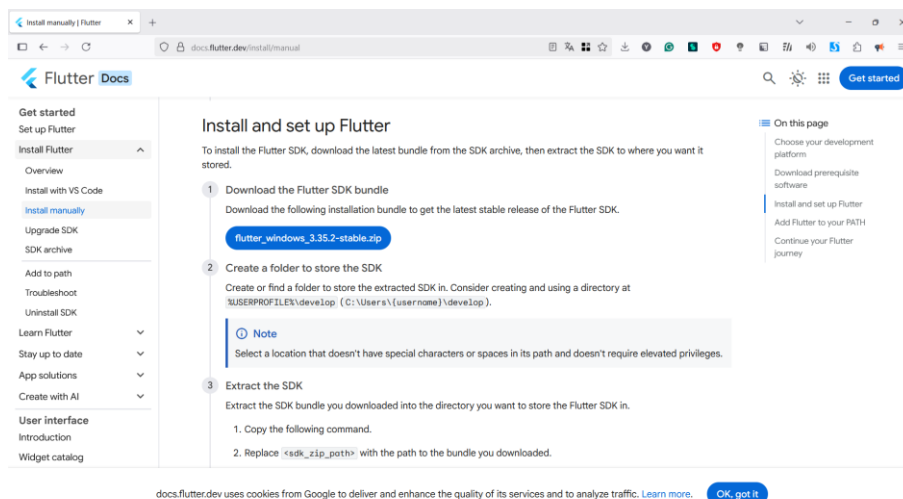


Ilustración 12: Instalación Flutter

Se realiza clic en `flutter_windows_3.35.2-stable.zip`, para descargar la versión más estable (actual) de manera automática.

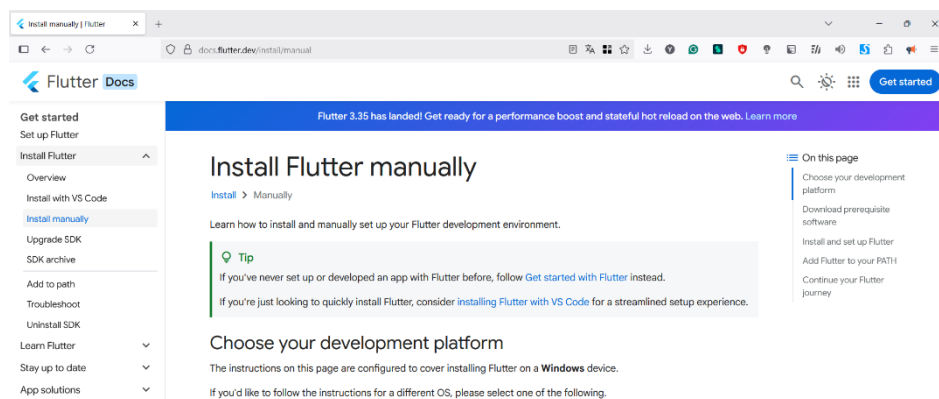


Ilustración 13: Instalación Flutter

Paso 2: Una vez descargada, extráela en la carpeta que desees; es recomendable que se coloque en el directorio del usuario en el disco C:

C:\Users*(nombre del usuario)*

Dado que se extraerá una carpeta llamada **flutter** y puede que en ciertos casos generar problemas con encontrar la carpeta.

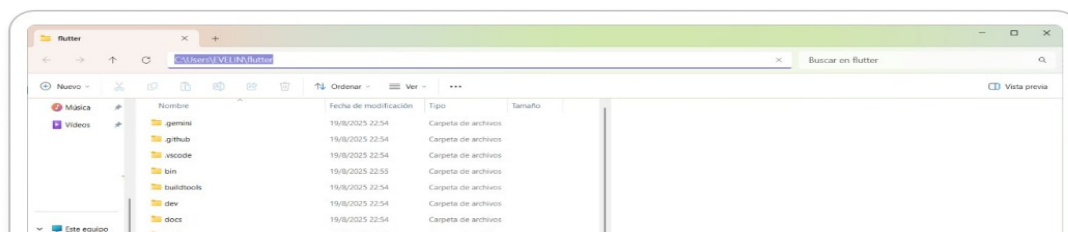
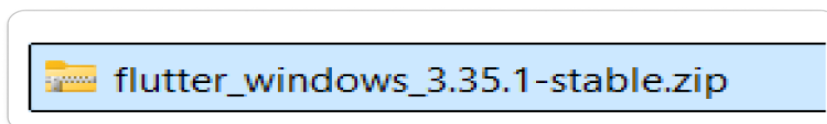


Ilustración 15: Instalación Flutter

Con eso se debe se puede establecer que Flutter se instaló, pero en ciertas circunstancias puede que las herramientas no la detecten (en ambas instalaciones) por lo que es recomendable realizar las siguientes acciones:

Paso 1: Verificamos donde está la carpeta de **flutter/bin**

Paso 2: Nos dirigimos a las propiedades del sistema específicamente a las variables de entorno:

Opción 1: Busca en tu computador “Editar Variables de entorno”

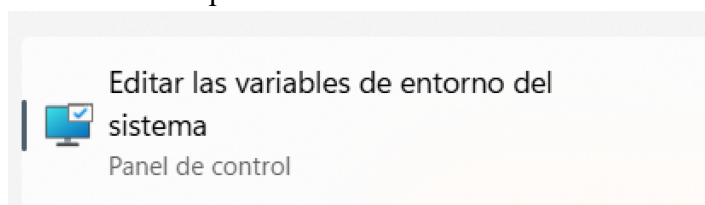


Ilustración 16: Instalación Flutter

Opción 2: Se realiza la siguiente secuencia:

Abre el Panel De Control, nos dirigimos a la sección de Sistema y seguridad. Luego hacemos clic en la parte de Sistema y buscamos la opción de Configuraciones avanzadas del sistema. Una vez encontrada se realiza clic en ella.

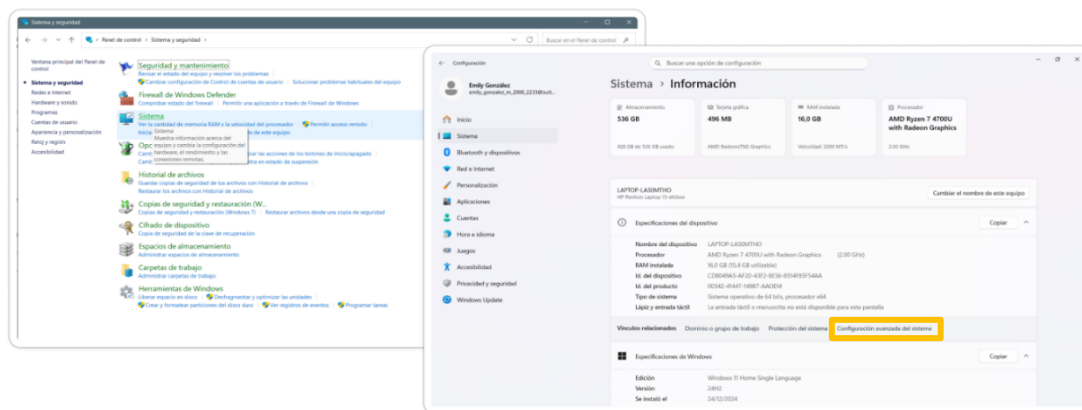


Ilustración 18: Instalación Flutter

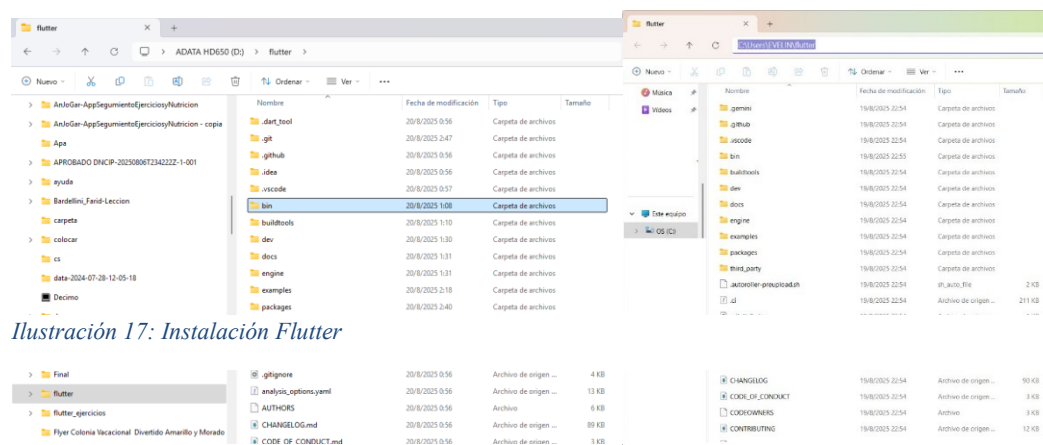


Ilustración 17: Instalación Flutter

Paso 3: Debe aparecer la siguiente pantalla y hacemos clic en el botón de Variables de entorno

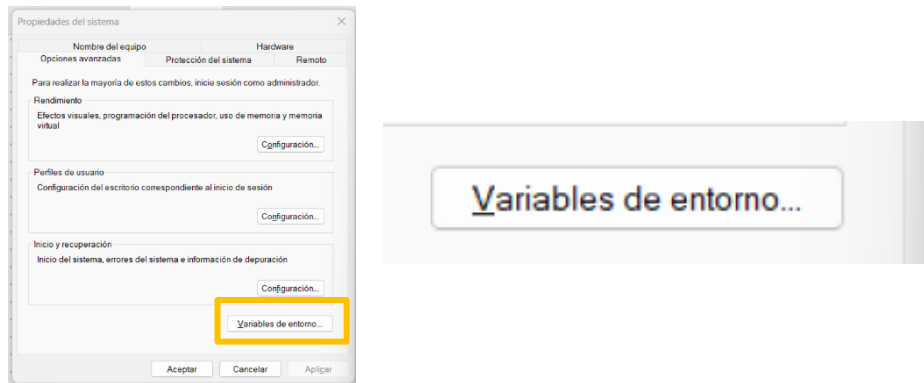


Ilustración 19: Instalación Flutter

Paso 4: La siguiente pantalla es la que aparece, buscamos la variable PATH y hacemos doble clic en ella

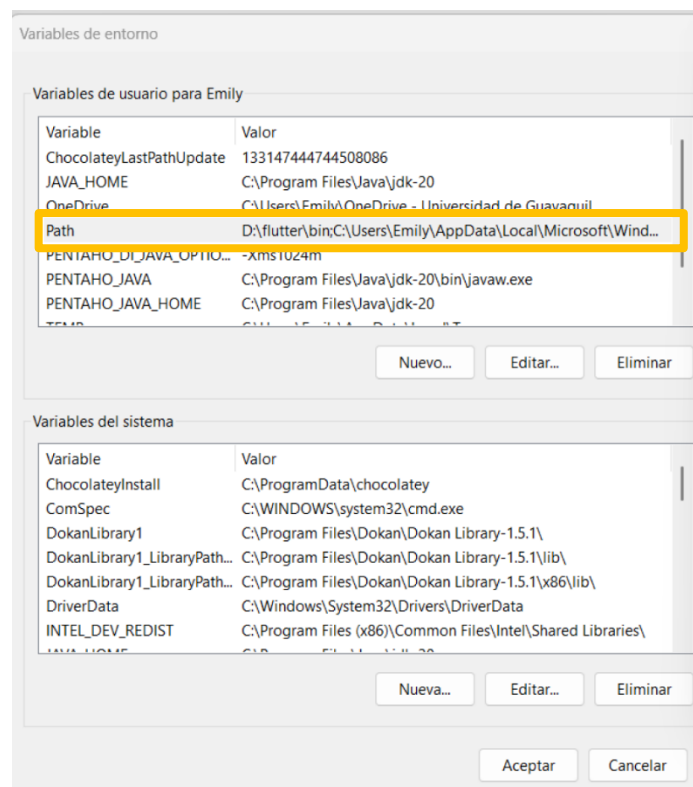


Ilustración 20: Instalación Flutter

Paso 5: En esta pantalla, se debe seguir los siguientes pasos:

1. Hacemos clic en el botón Nuevo
2. Colocamos la ruta donde esta la carpeta flutter/bin
3. Luego subimos al inicio
4. Debe quedar como esta en la imagen

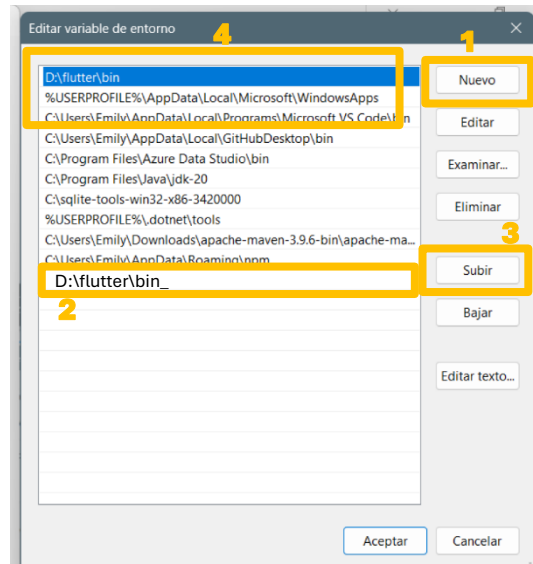


Ilustración 21: Instalación Flutter

Paso 6: Aceptamos todo y para mayor seguridad reiniciamos el dispositivo.

FASE 2: Implementación de Flutter en el entorno de trabajo

A continuación, se explicará de manera detallada de la implementación de las extensiones o plugins de Flutter en el entorno de trabajo, ya sean en Android Studio o Visual Studio.

Visual Studio

Paso 1: Iniciamos el programa

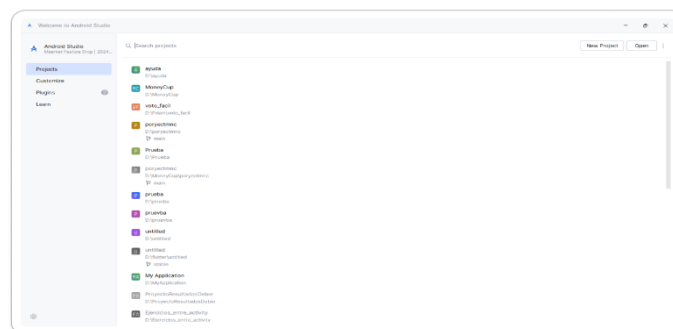


Ilustración 22: Instalación Flutter

Paso 2: Nos dirigimos a la sección de Pluggins, en la parte de MARKETPLACE

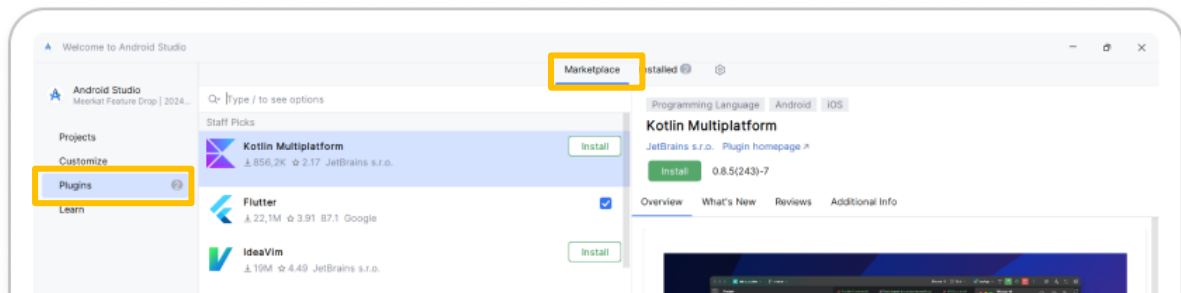


Ilustración 23: Instalación Flutter

Paso 3: Escribimos “DART” en la barra de búsqueda y realizamos la instalación

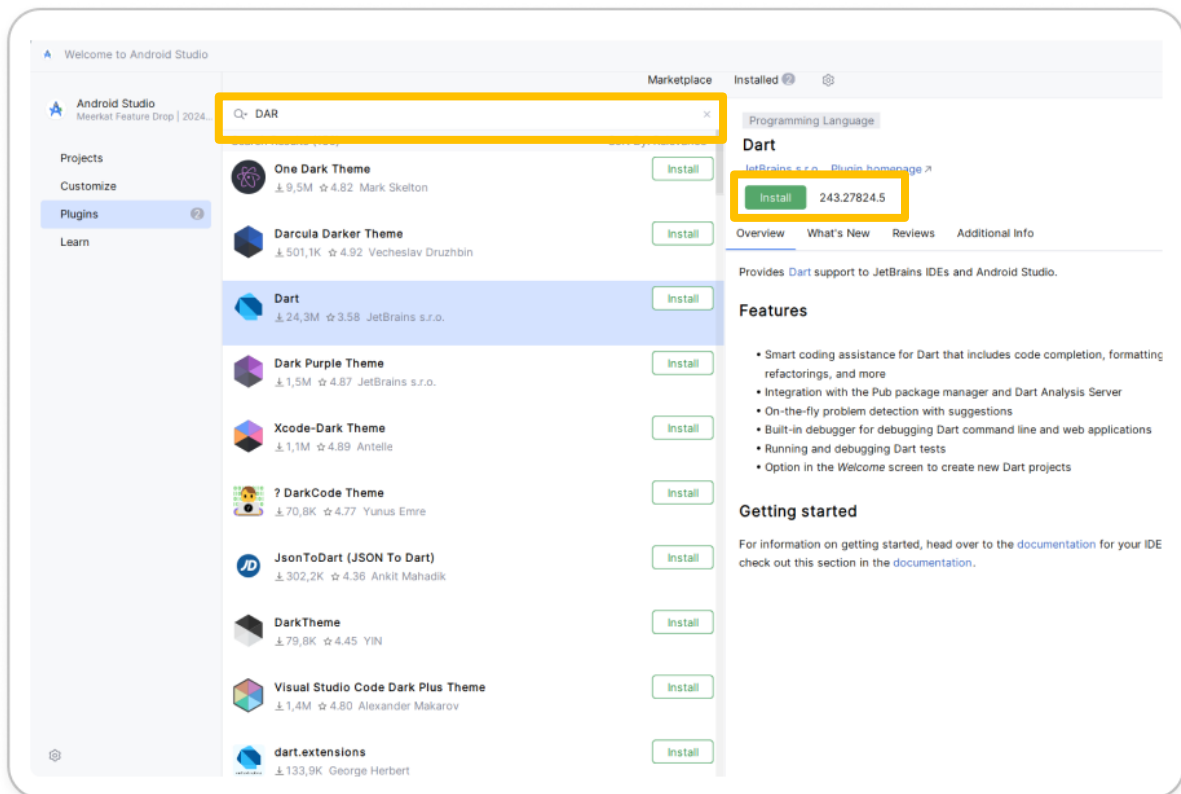


Ilustración 24: Instalación Flutter

Hacemos lo mismo con Flutter:

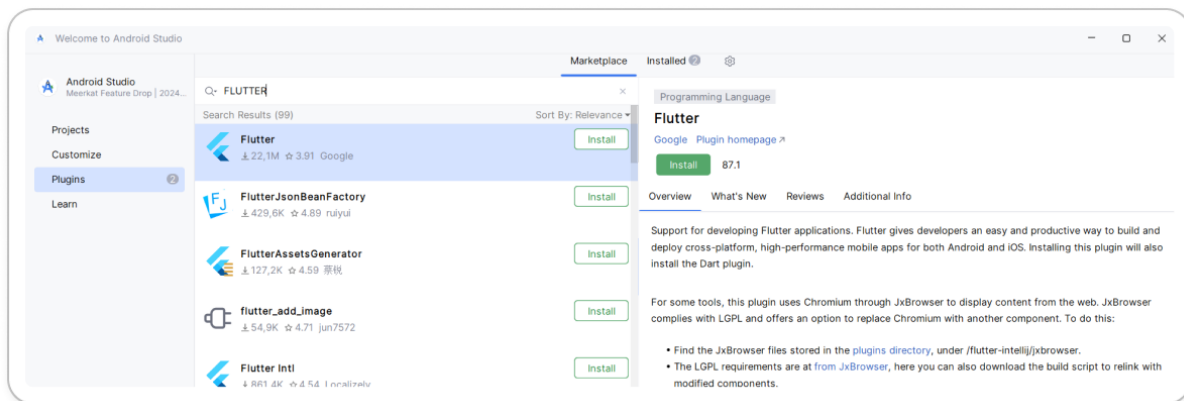


Ilustración 25: Instalación Flutter

Paso 4: Una vez instalado, se tiene que reiniciar el aplicativo.

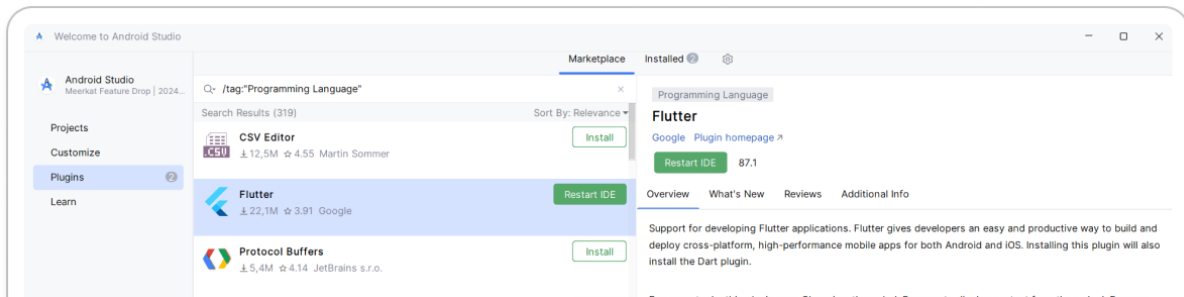


Ilustración 26: Instalación Flutter

Paso 5: Para verificar que se ha instalado de manera correcta el plugin; se debe observar el botón de New Flutter Project al iniciar el aplicativo y los plugins deben estar en la sección de Installed.

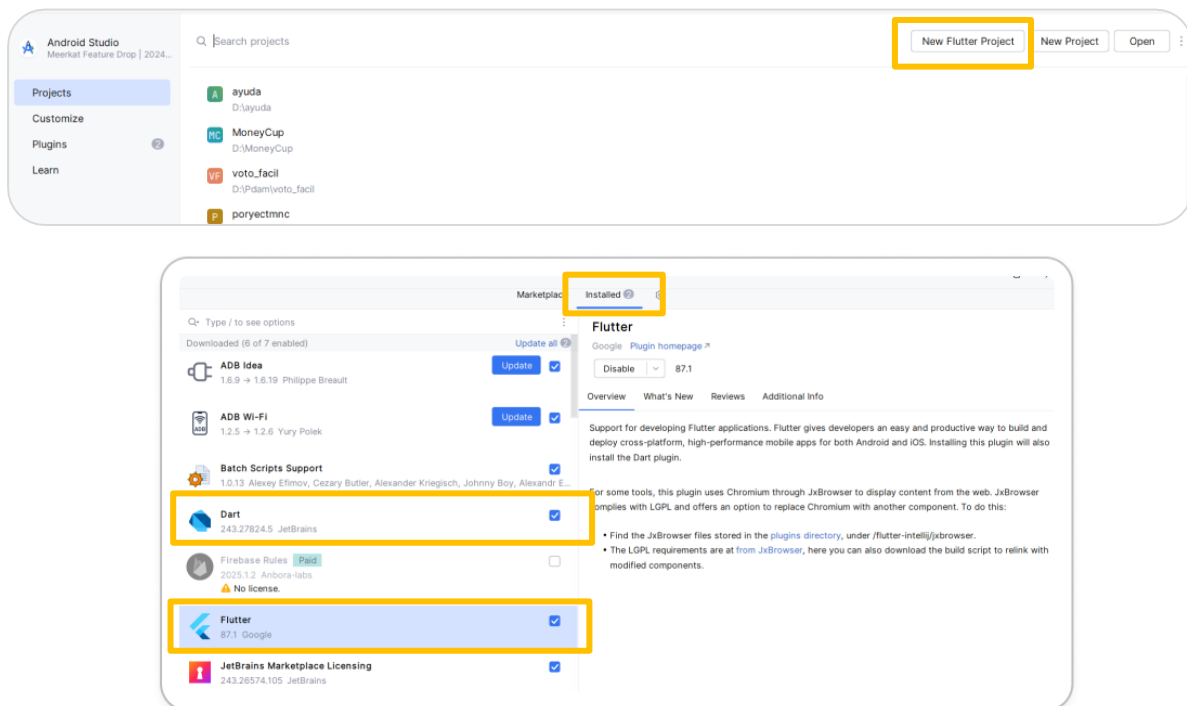


Ilustración 27: Instalación Flutter

Visual Studio

Paso 1: Abrimos el Visual Studio y nos dirigimos a las Extensiones colocada en un parte lateral

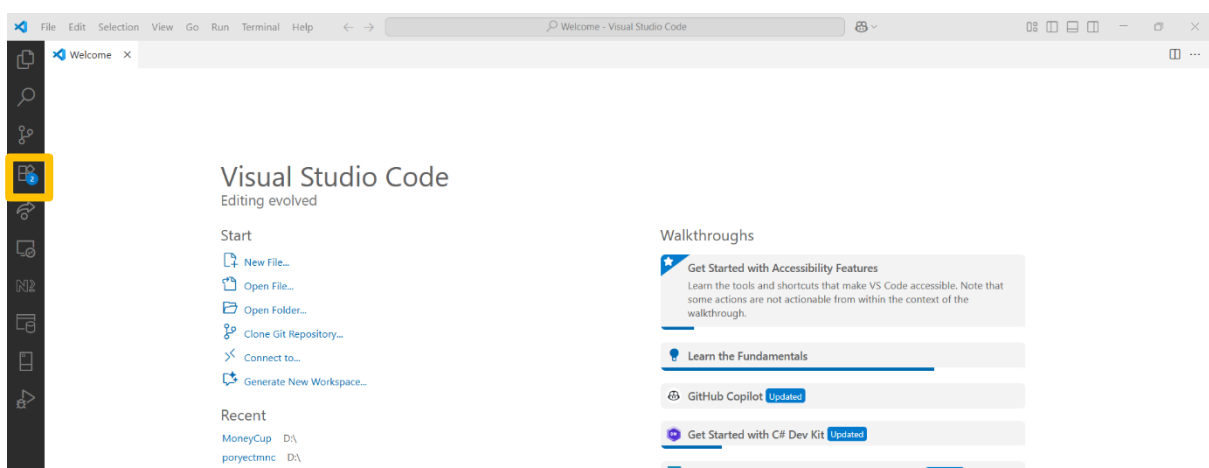


Ilustración 28: Instalación Flutter

Paso 2: Escribimos DART o FLUTTER en la barra de búsqueda e instalamos. Un aviso SI buscas e instalas Flutter primero te sale un aviso que las dos extensiones (FLUTTER Y DART) se van instalar

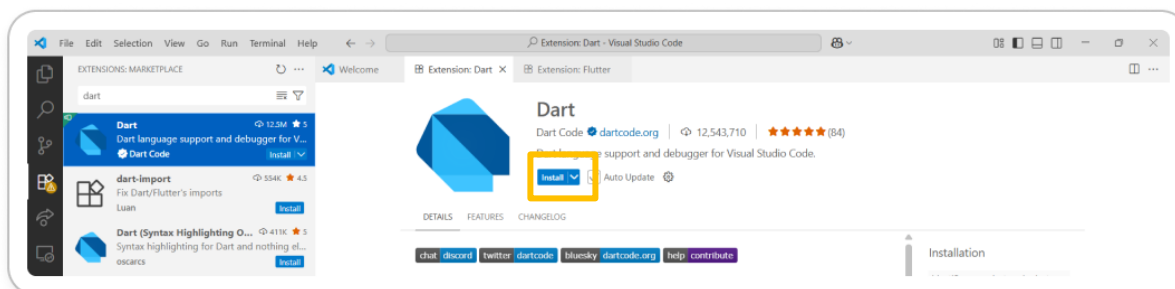


Ilustración 30: Instalación Flutter

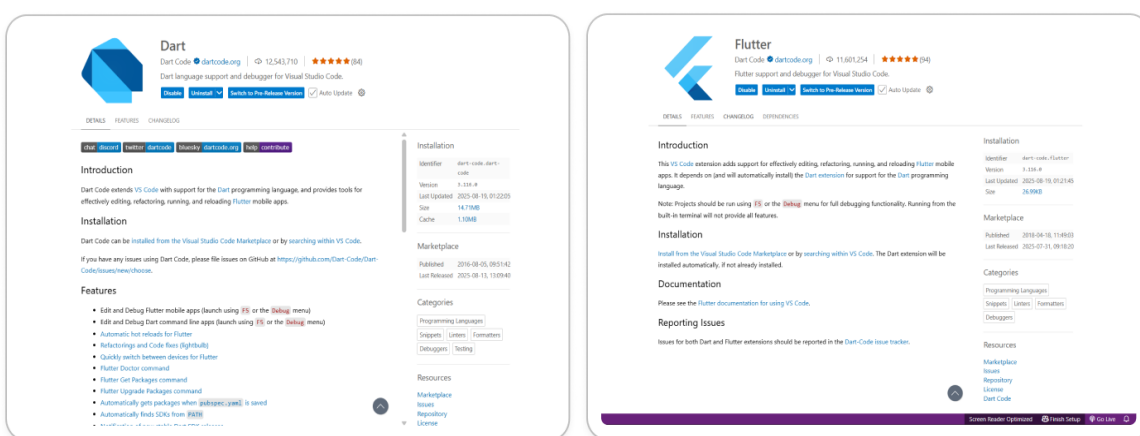


Ilustración 29: Instalación Flutter

Paso 3: Verificamos la instalación de ambas extensiones.

No es necesario reiniciar el aplicativo, pero es recomendable.

Paso 4: Para verificar si se ha implantado la extensión, nos dirigimos al área de búsqueda y colocamos >flutter y si sale **Flutter: New Project**, se ha realizado con éxito.

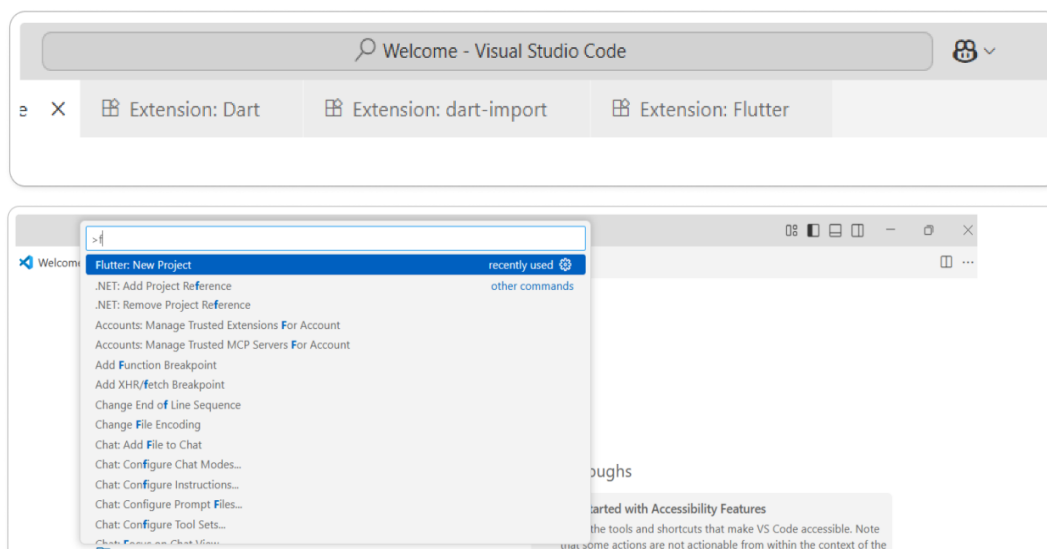


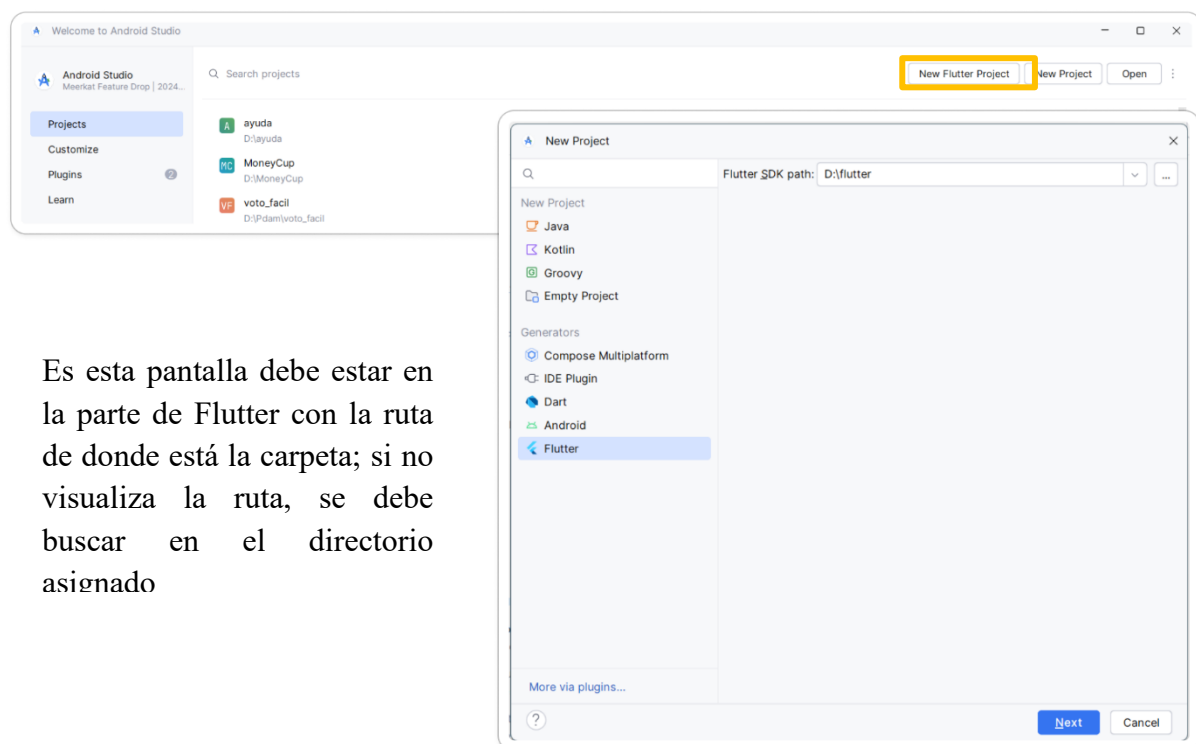
Ilustración 31: Instalación Flutter

Fase 3: Verificación e instalación.

En esta parte verificaremos que todo este correctamente implementado para la generación y compilación de cualquier proyecto Flutter.

Android Studio

Paso 1: Hacemos clic en botón de New Flutter Project, se debe abrir la ventana de New Project



Es esta pantalla debe estar en la parte de Flutter con la ruta de donde está la carpeta; si no visualiza la ruta, se debe buscar en el directorio asignado

Ilustración 32: Instalación Flutter

Paso 2: Si después de hacer el clic en el botón de Next aparece la siguiente ventana, entonces todo esta correctamente implementado.

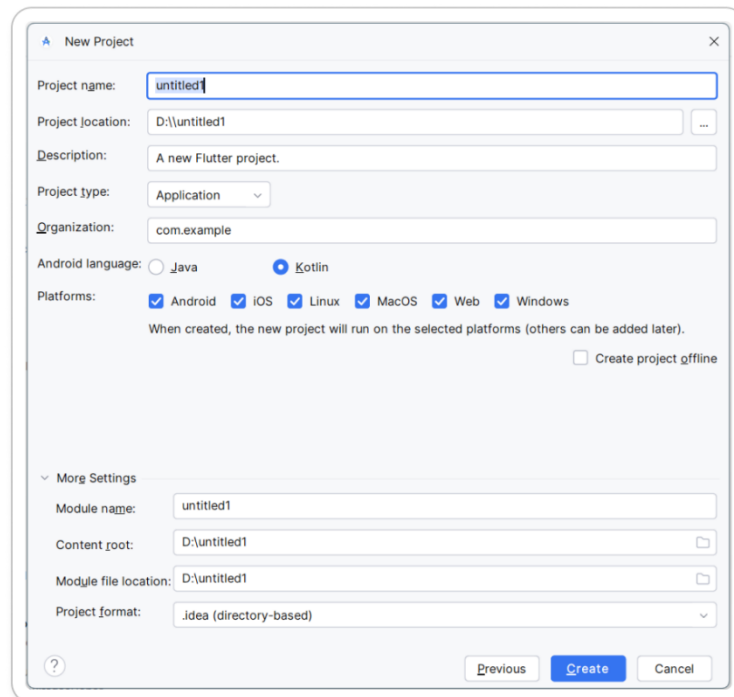
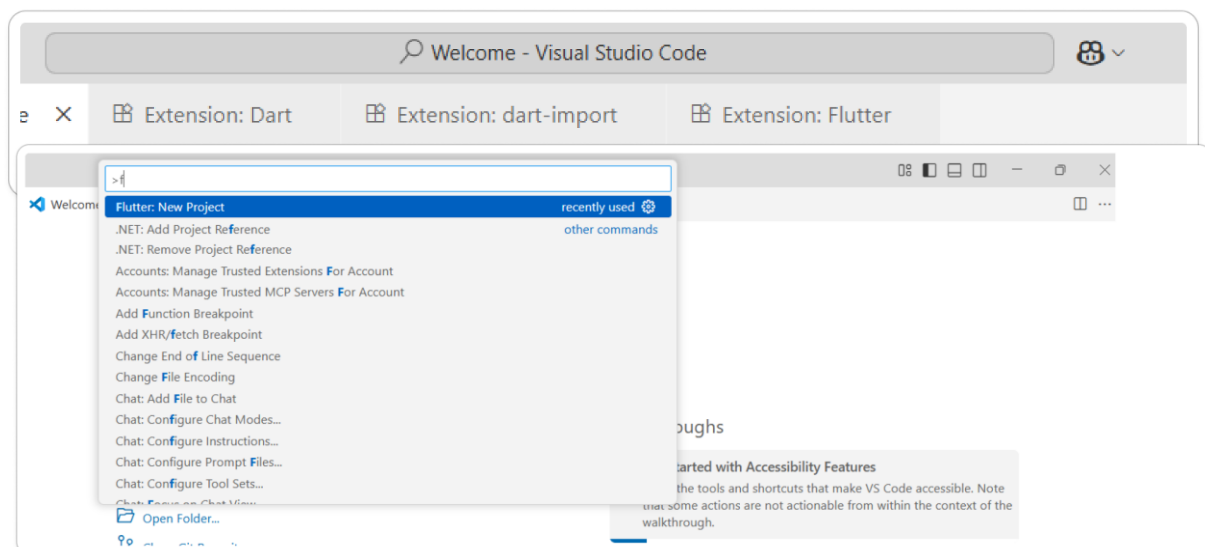


Ilustración 33: Instalación Flutter

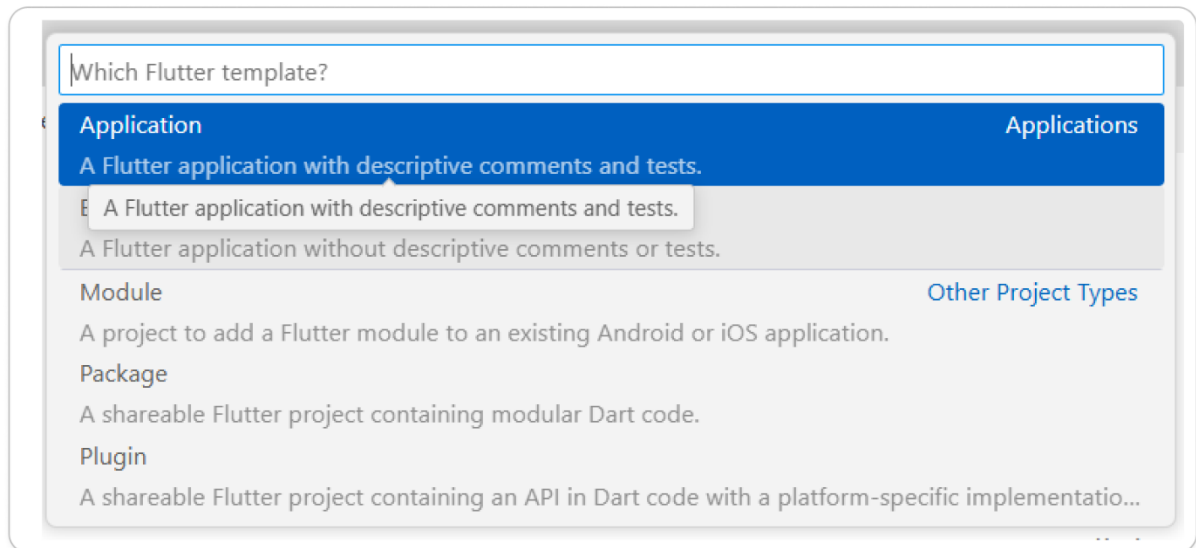
Si sale alguna advertencia o no parece la pantalla verificar las variables de entorno.

Visual studio

Paso 1: nos dirigimos al área de búsqueda y colocamos **>flutter** y si sale **Flutter: New Project**, y lo seleccionamos



Paso 2: Al seleccionar **Flutter: New Project** debe aparecer lo siguiente:

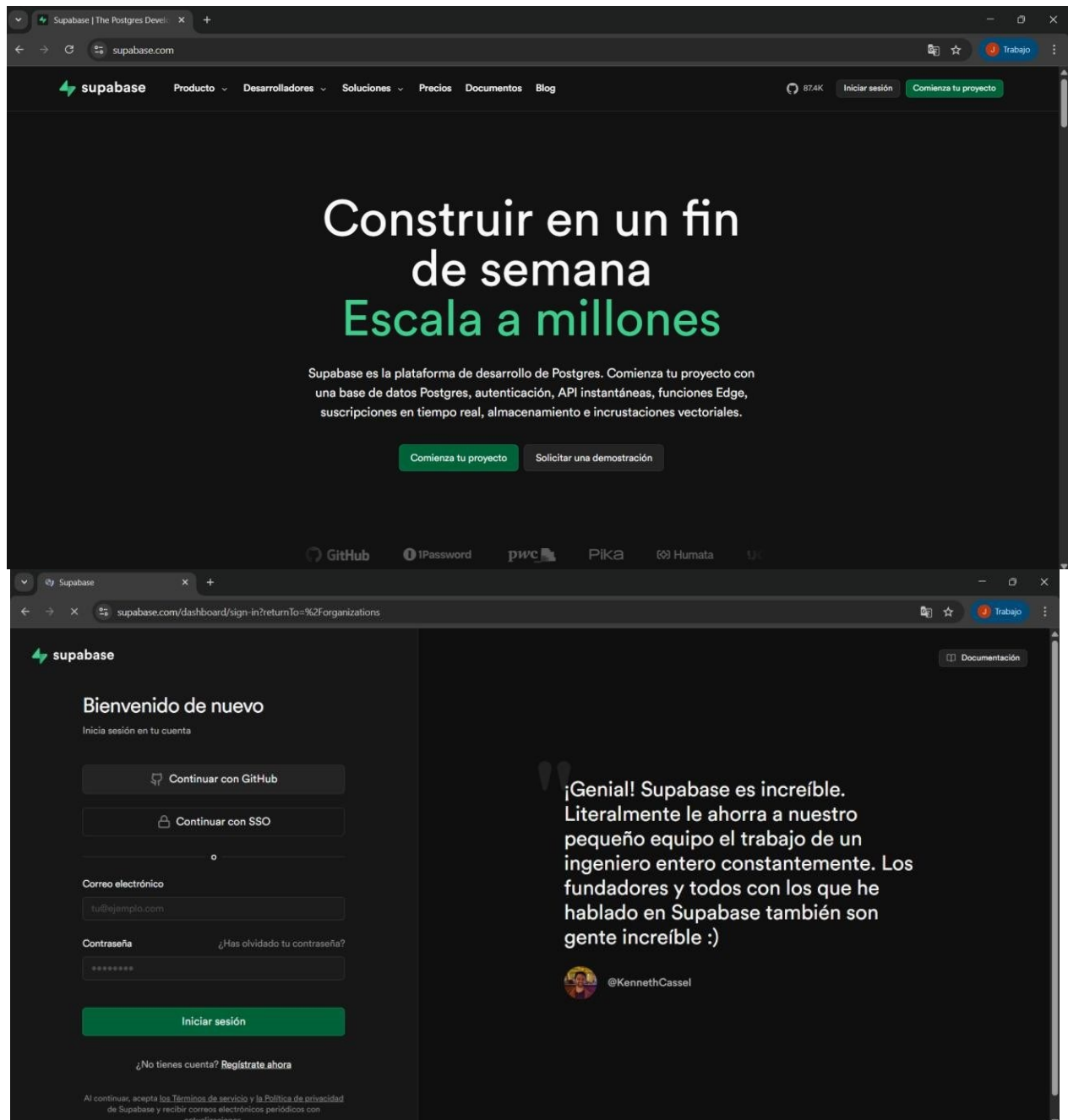


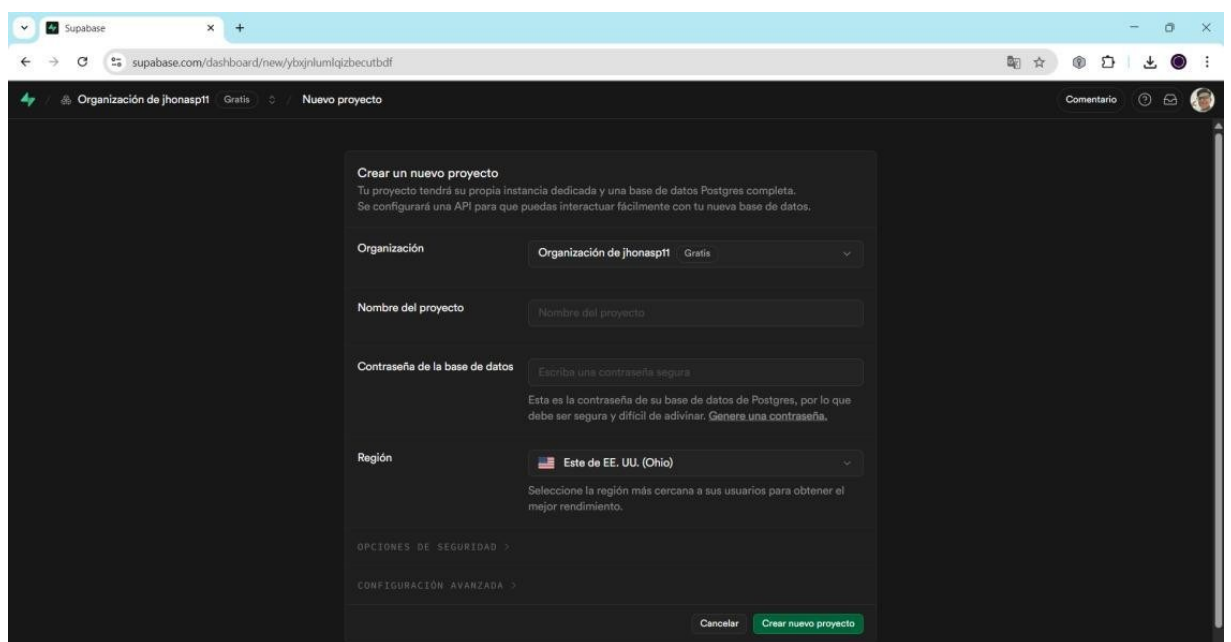
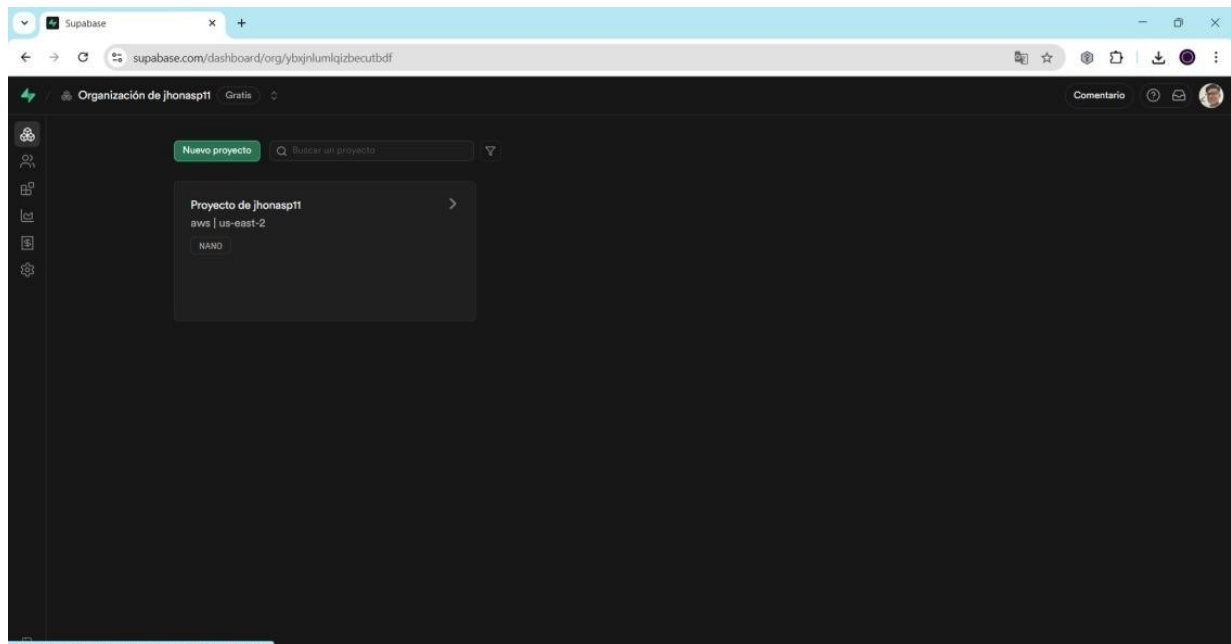
Si sale alguna advertencia o no parece la pantalla verificar las **Variables de entorno**.

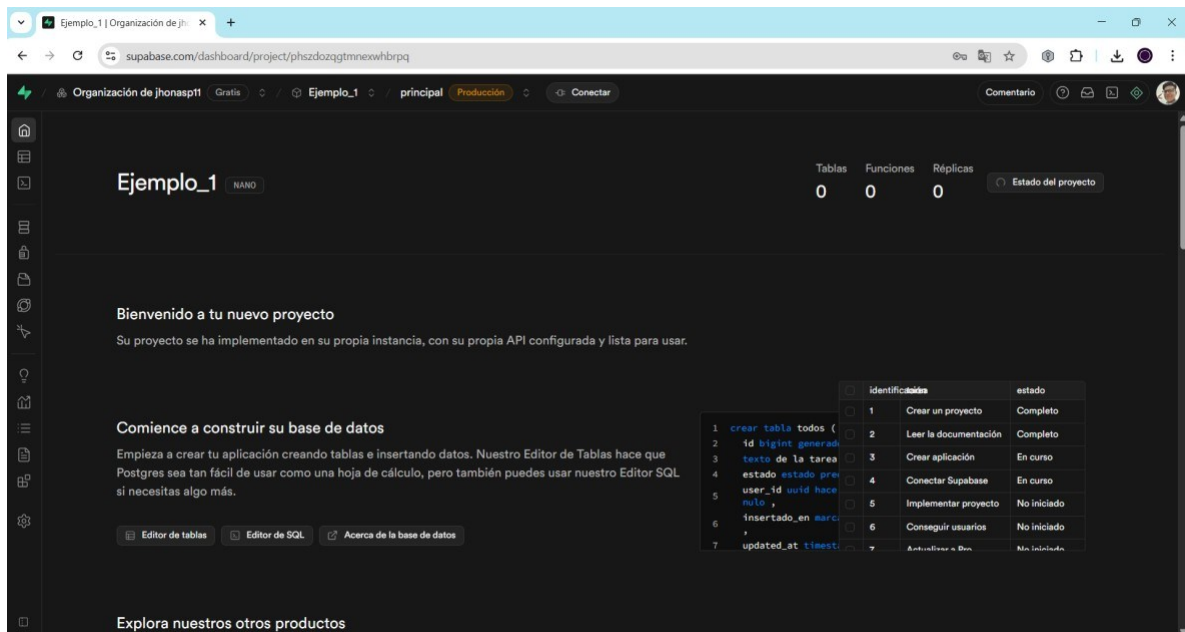
Creación de nuevo proyecto con Supabase

Pasos a seguir:

1. Ingresar a <https://supabase.com/>
2. Registrarse o iniciar sesión con Google/GitHub.
3. Crear un **nuevo proyecto** → indicar nombre, región y contraseña de la base de datos.
4. Guardar la **URL del proyecto** y la **API Key** (necesarias para integrarlo con Flutter).



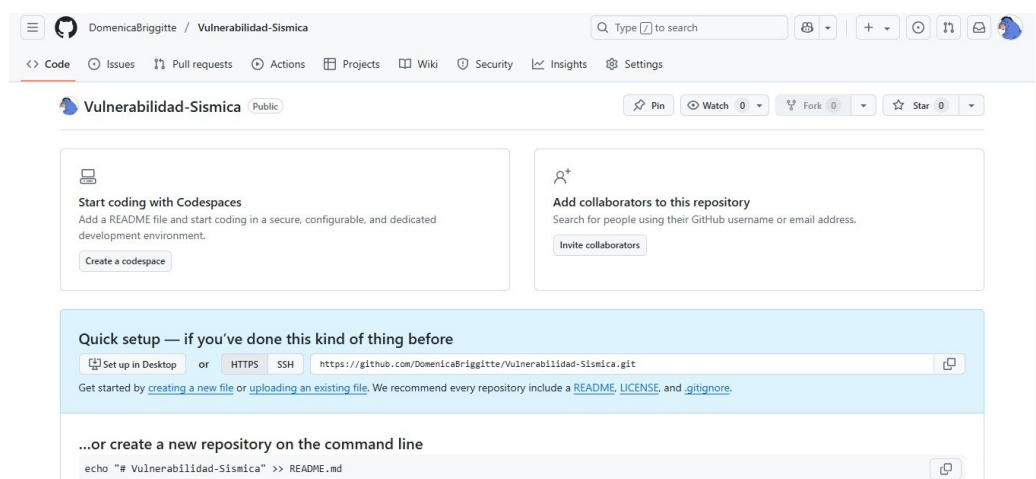




Creación del repositorio en github

Pasos a seguir:

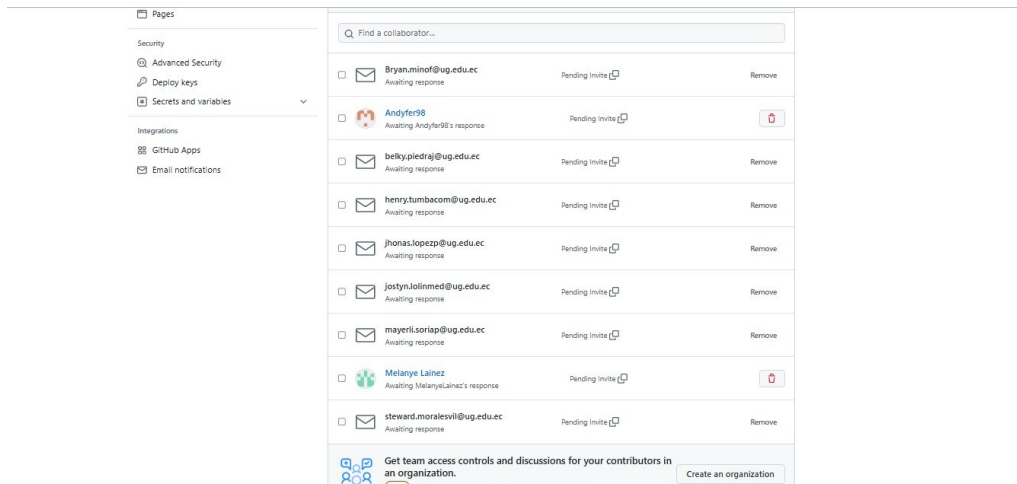
1. Ingresar a GitHub y hacer clic en **New Repository**.
2. Definir nombre del repositorio (ejemplo: **SismoApp**).
3. Elegir visibilidad (Privado o Público).
4. Marcar la opción **Initialize with README**.
5. Crear el repositorio.



Colaboradores en Github

Pasos a seguir:

1. Entrar al repositorio.
2. Ir a **Settings > Collaborators**.
3. Agregar los usuarios de tu equipo.
4. Cada colaborador recibirá una invitación en su cuenta GitHub.



Proyecto en Github

Terminal de Android Studio mostrando comandos Git:

1. Abrir la terminal de Android Studio en el proyecto.
2. Inicializar Git:

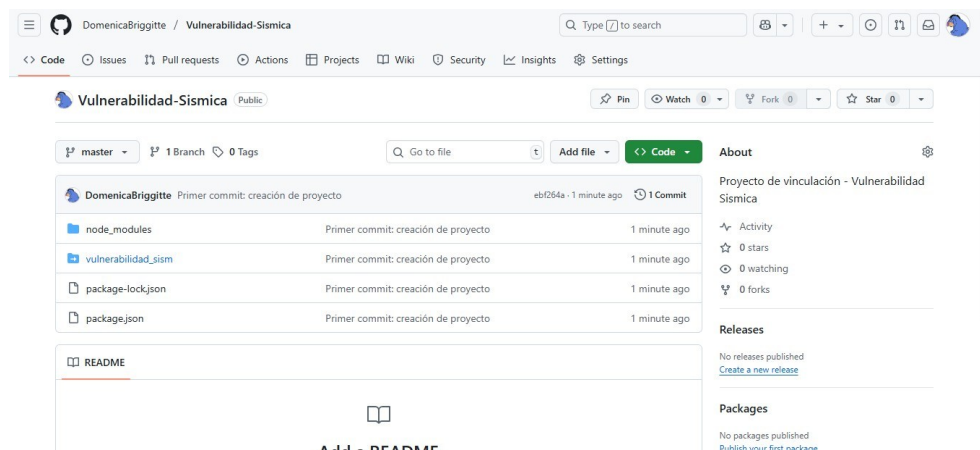

```
git init
```

```
git remote add origin https://github.com/usuario/SismoApp.git
```
3. Subir cambios iniciales:


```
git add .
```

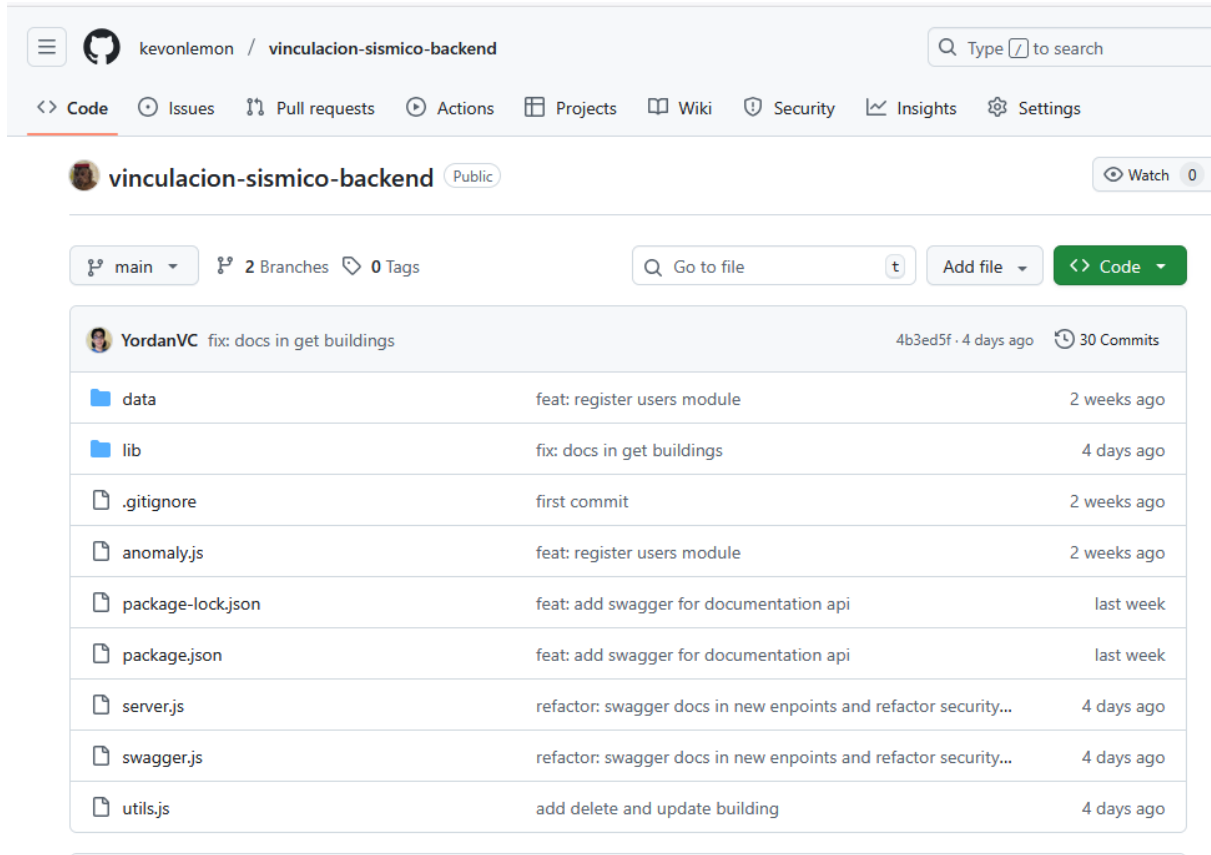
```
git commit -m "Proyecto inicial"
```

```
git push -u origin main
```
4. Verificar en GitHub que el proyecto aparece cargado.



Descarga del servidor para conexión de la base de datos

1. Descargar el repositorio del back end

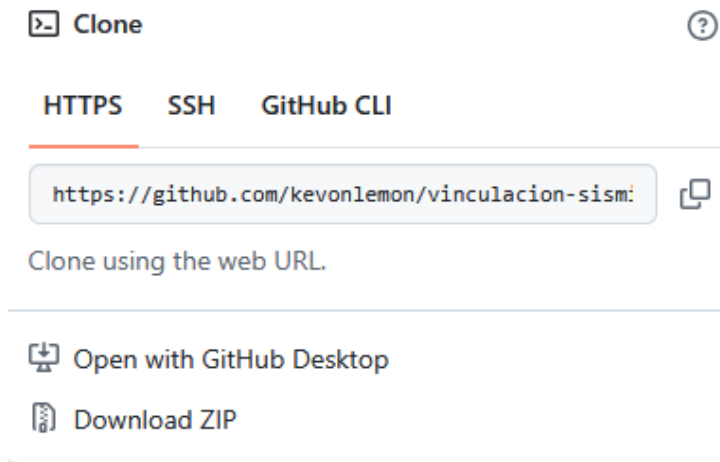


The screenshot shows the GitHub repository page for 'vinculacion-sismico-backend' by user 'kevonlemon'. The repository is public. The main branch is 'main'. There are 2 branches and 0 tags. The repository has 30 commits. The files and folders listed are:

File/Folder	Commit Message	Time Ago
data	feat: register users module	2 weeks ago
lib	fix: docs in get buildings	4 days ago
.gitignore	first commit	2 weeks ago
anomaly.js	feat: register users module	2 weeks ago
package-lock.json	feat: add swagger for documentation api	last week
package.json	feat: add swagger for documentation api	last week
server.js	refactor: swagger docs in new endpoints and refactor security...	4 days ago
swagger.js	refactor: swagger docs in new endpoints and refactor security...	4 days ago
utils.js	add delete and update building	4 days ago

Backend: <https://github.com/kevonlemon/vinculacion-sismico-backend>

Se puede usar cualquiera forma para obtener el repositorio, el que sea más conveniente para el usuario.

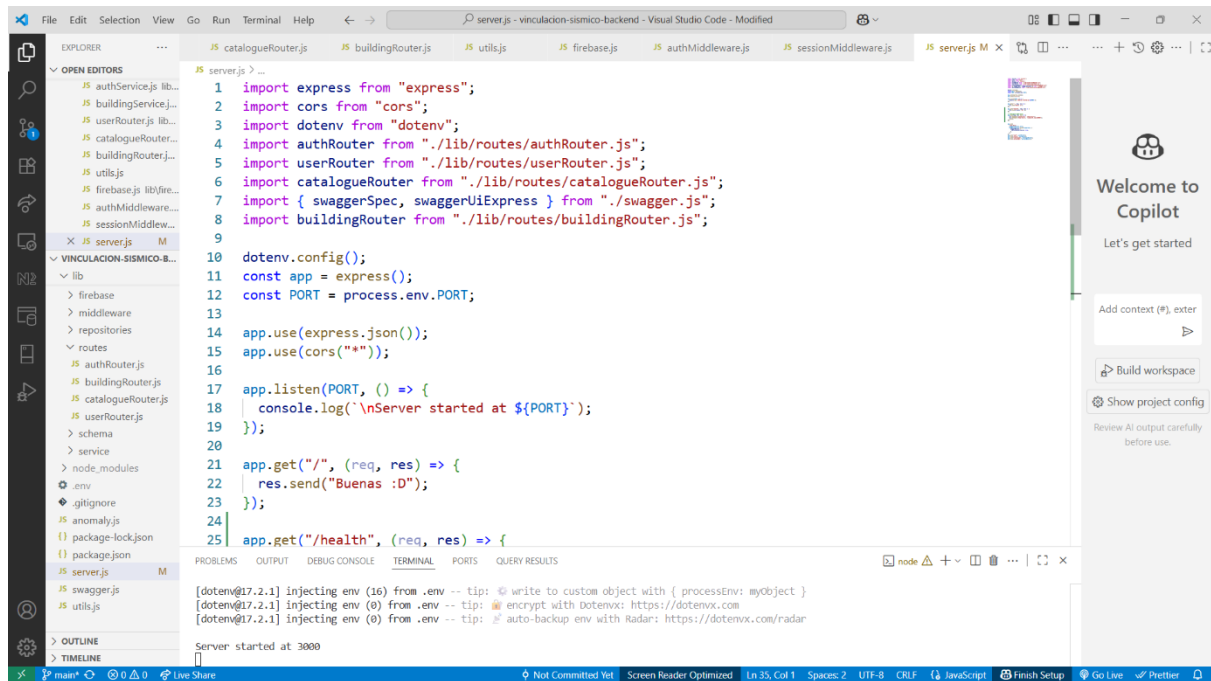


The screenshot shows the 'Clone' section of the GitHub repository page. It displays the HTTPS URL for cloning the repository:

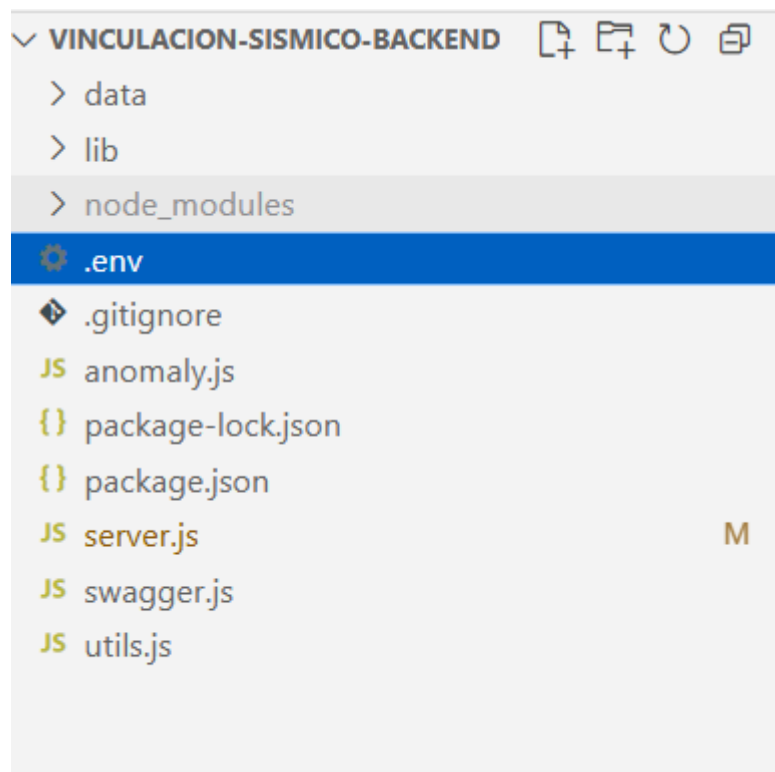
```
https://github.com/kevonlemon/vinculacion-sismico-backend.git
```

Below the URL, it says 'Clone using the web URL.' There are also options to 'Open with GitHub Desktop' and 'Download ZIP'.

2. Una vez teniendo la carpeta del repositorio del proyecto se debe abrir en Visual Studio Code



3. Crear un archivo **.env** en la ruta principal del proyecto (imagen)



Colocamos en el .env el siguiente texto, que son las credenciales de la base de datos

```
DB_TEST_SCHEMA=public
DB_HOST=aws-0-us-east-2.pooler.supabase.com
DB_USER=postgres.gevboecwurmleirqlxxh
DB_PORT=5432
DB_PASSWORD=1234
DB=postgres
KNEX_CLIENT=pg
NODE_ENV=development
PORT=3000
JWT_ACCESS_SECRET= ejemploToken
JWT_REFRESH_SECRET= ejemplorefreshToken
DB_TEST_SCHEMA=public

//CREDENCIALES DE FIREBASE
FIREBASE_PRIVATE_KEY="-----BEGIN PRIVATE KEY-----
nMIIEvAIBADANBgkqhkiG9w0BAQEFAASCBywggSiAgEAAoIBAQCnTZ/VA3qhVklz/n14pVe7koAejm35sHRQjKbajxQE66VtUmVGBjcfS8iuBbtqeAplL+x90r2PMOVZR/nQQeAVEAQ/GPa
IBi3Gur/yclpdeCegys/ZxGMheXoRdUea2bw/2QKIs9TeVPDJu/nRLxByP4NZ1bGgLYnR9sYcymXcBLIO4prmuQXH1qw8YNkcU336GS2hXg04iFvRwPnqHIOUssARYc6osinKmsY3xJr922kDbb
CXve9118qasPMDI3Whoyo7IF8Df1QAzu/nDNNPkPFZa0Wgzki/jfUUm/cYGIxGAg0c7/JtSw0DO1398acbnEt34TReJzj25Qh/nw8t7sg4FAGMBAAECggEASo768GzU/UqdY+cmJBN3wB8dwqobBkH
boY9mT2GH1F43nLpZCwH68ATxo4SaE/n3LRYicC53QPWaa+E4cGB0hEpu07uK07PMnzCBs1Q9jdMq4inesQdy2gu3PizE3gliRbujQ17TNZGn5RwTeLqEuhAKsxjQ3/tq0hvP4ZJ9TJGnMna9JZQ
PuqU1teh9F2nLnrc3XQTPyCu4m0Lz7U5w6vttU29fa2pvk3cTB4YV93L9n+5Eg+UDgW5XlmUovoKzNuUeljry6JWdTnVtQszHy5ux6Aomp7BlwPnde/6fHkoUnvTpVQXiyd8x6mkp9udEGRqdWa
w8EHCF1PkNW0/DRbwKBgQDVGrxaFEAfcYi4LyYnBttugH+89L3hnvoT4arJqCVYd0kR7jo5xUsMJApBrOOGQnLYKaGphNJTEo158yTnMpT6ZCNbdYuTqek+EulcJdDtEaf7HN3suNS4VCZIW
Mlw9NGsZ8GmeCfK/FKkWiF/n3NidTKXS61w46h0lWRGj9C/YrwKBgQDI+sDIBp6sWVFRbRSkC+BLfEQRn0bq2Ji/nOes/remwztUyQCBSVehxLp+ScptsVWwx+nsdUVd5m2hOV914JkMqzxtsTf
oUgxs/nnD1kFE5C8GsCFROc5WzLEDE4kWF6cyuRvOm3JD0UnqzL6m20rMmQ6pQnFTdw8XnSb/L7krJiwKBgG6pI4XjXij12/Pu8ABCD1CiibKlpLS0yUlp7ZgCoSV7OvSuYmm/nWgJD9A8Ne9
AeU/Ge8GL1zIDC/3Pi7KjDa3la5FmmMNISljiYKqBOKZirkhIBTYnBN5zVC3dbT2q4/sTKocXoP6LUFFYkUlmHgO10bxj5SezoZpvVim9c9shAoGAJYxTnhXZbJq/5NVE/zY6DWBGB1TOlwmnL
5rKjLcdN8jZjVdkVVje+HOBjPlN04uTjg/nTeELGKP4T2q8nvU7ltdL2N0oqLR12kJPylcAb2RAG6Xn3RtgVkf4YJxClajbv/nR10ep/Tl7mQRMrM9okPmLVEwgv13Yk5w+ISmPEkCgYAnE9KXv
URIGUDyJzhg0V5CnrgLPTqY9XYw7H7ZnUV+2TTIQLDeoK9eoUhlZJS83gNxywXwvSB7jltT3VtVnNsFa/n1saxoljeMtwX625iTY6jMkXWXejsetSy1xc3Z46L8fyOGAaDwBkryvBtCQkMwZ5\
nPxow8X7hnx6gT28Oq020lw=n-----END PRIVATE KEY-----"
FIREBASE_CLIENT_EMAIL=firebase-adminsdk-fbsvc@sistema-vulnerabilidad-sismica.iam.gserviceaccount.com
FIREBASE_PROJECT_ID=sistema-vulnerabilidad-sismica

DOMAIN_URL_FIREBASE_STORAGE=https://storage.googleapis.com
STORAGE_BUCKET=sistema-vulnerabilidad-sismica.firebaseiostorage.app
```

4. Abrir terminal del proyecto (back end Visual Studio) y colocar

npm i

Este comando nos instalar las dependencias para el servidor. Además, se tiene que realizar con cada cambio que tenga el servidor.

```
Restarting 'server.js'
● PS D:\vinculacion-sismico-backend> npm i

up to date, audited 312 packages in 3s

35 packages are looking for funding
run `npm fund` for details

found 0 vulnerabilities
○ PS D:\vinculacion-sismico-backend> █
```

Una vez terminado el primer comando, escribimos y ejecutamos

node --watch server.js

Este comando es para correr el servidor de manera local

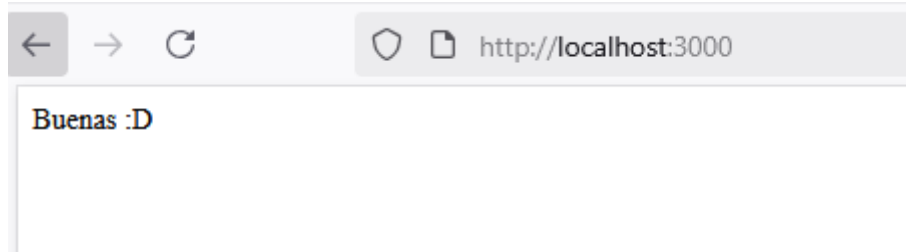
```
Server started at 3000
○ PS D:\vinculacion-sismico-backend> node --watch server.js
[dotenv@17.2.1] injecting env (16) from .env -- tip: 📖 version env with Radar: https://dotenvx.com/radar
[dotenv@17.2.1] injecting env (0) from .env -- tip: 📖 auto-backup env with Radar: https://dotenvx.com/radar
█
```

Una vez establecido verificamos que se haya establecido el server que debe aparecer esto en la terminal

```
Restarting 'server.js'
[dotenv@17.2.1] injecting env (16) from .env -- tip: 🐿 version env with Radar: https://dotenvx.com/radar
[dotenv@17.2.1] injecting env (0) from .env -- tip: 🐿 specify custom .env file path with { path: '/custom/path/.env' }
[dotenv@17.2.1] injecting env (0) from .env -- tip: 🐿 write to custom object with { processEnv: myObject }

Server started at 3000
□
```

O colocamos el localhost:3000 en nuestro navegador actual



En server se agrega

```
app.get("/health", (req, res) => {
  res.json({ status: "ok" });
});

// Configurar keep-alive
app.use((req, res, next) => {
  res.setHeader('Connection', 'keep-alive');
  res.setHeader('Keep-Alive', 'timeout=5, max=100000');
  next();
});
```

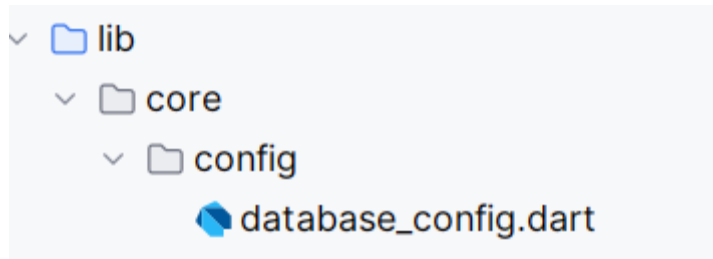
Tipo así:



6. COMPILADOR DEL PROYECTO

Advertencia: Se debe tener descargado el back end y el front end desde el github

Abrimos el proyecto del front -end (Visual Studio o Android Studio) y nos dirigimos a database_config.dart

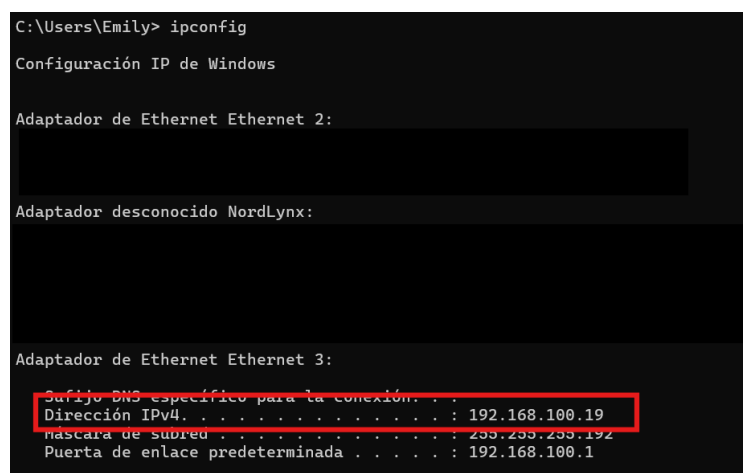


Y lo abrimos



Aquí solo cambiaremos la ip de que se presenta que es donde se localiza el servidor para fuentes externas, pero **DENTRO DE LA MISMA RED WIFI.**

Usando el comando ipconfig (en Windows) nos visualiza la dirección que necesitamos



Una vez terminado eso podemos compilar el proyecto

Opcion 1: Emulador de Android Studio

Si el proyecto no es demasiado pesado, el proyecto se puede ejecutar de manera directa con el Emulador que viene por defecto en Android Studio:

- **Paso 1:** Busca la herramienta de Device Manager y crea el emulador adecuado para el proyecto

Opcion 2: Sacar APK del Programa

Paso 1: Abrimos la terminal del proyecto colocamos los siguientes comandos

`flutter build apk --release`

Y esperamos que se ejecute hasta que aparezca el mensaje de :

✓ Built build\app\outputs\flutter-apk\app-release.apk (45.6MB)



Paso 2: Escribimos luego el siguiente comando:

`cd build\app\outputs\flutter-apk`

```

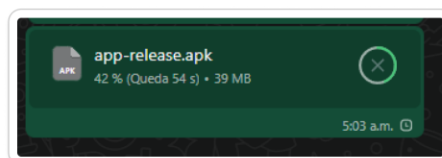
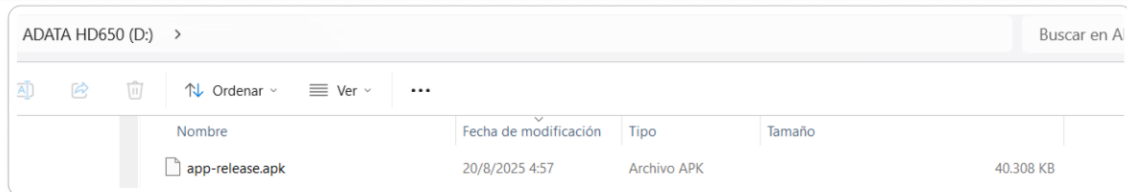
PS D:\prueba2025> cd build\app\outputs\flutter-apk
PS D:\prueba2025\build\app\outputs\flutter-apk>
  
```

Paso 3: Luego el siguiente:

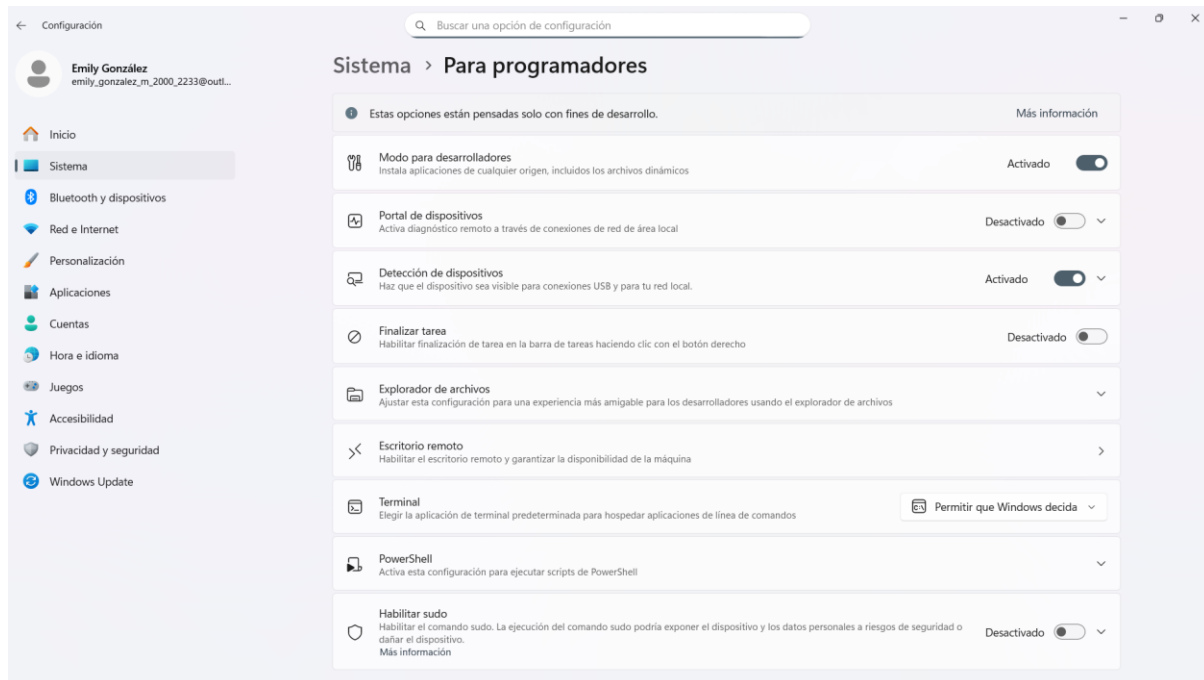
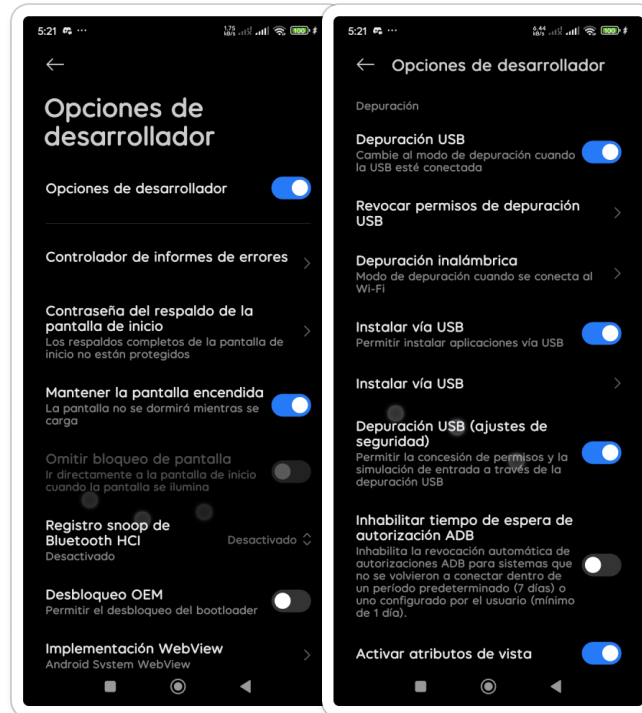
`copy app-release.apk D:\` ← donde vas a copiar el apk

```

PS D:\prueba2025\build\app\outputs\flutter-apk> copy app-release.apk D:\
PS D:\prueba2025\build\app\outputs\flutter-apk>
  
```



Para las siguientes formas necesitas tener configurado el dispositivo para la depuración USB o WIFI (tanto desde el computador como para el teléfono)



Opción 3: Terminal de Flutter para dispositivo físico

Paso 1: Conecta tu teléfono con cable USB

Paso 2: Abre la terminal del proyecto y coloca el siguiente comando:

`flutter devices`

Y esperamos que se visualicen los dispositivos:

```
PS C:\flutterproyectos\Examen_2p_Domenica_Ortiz\vulnesis\prueba> flutter devices
• >> # Esto usa ADB internamente
Found 5 connected devices:
SM S918B (mobile) • R5CM711M44E • android-arm64 • Android 15 (API 35)
SM S918B (mobile) • 192.168.3.27:5555 • android-arm64 • Android 15 (API 35)
Windows (desktop) • windows • windows-x64 • Microsoft Windows [Versi#n 10.0.26100.4946]
Chrome (web) • chrome • web-javascript • Google Chrome 139.0.7258.139
Edge (web) • edge • web-javascript • Microsoft Edge 139.0.3405.111
```

Paso 3: Se coloca el comando de

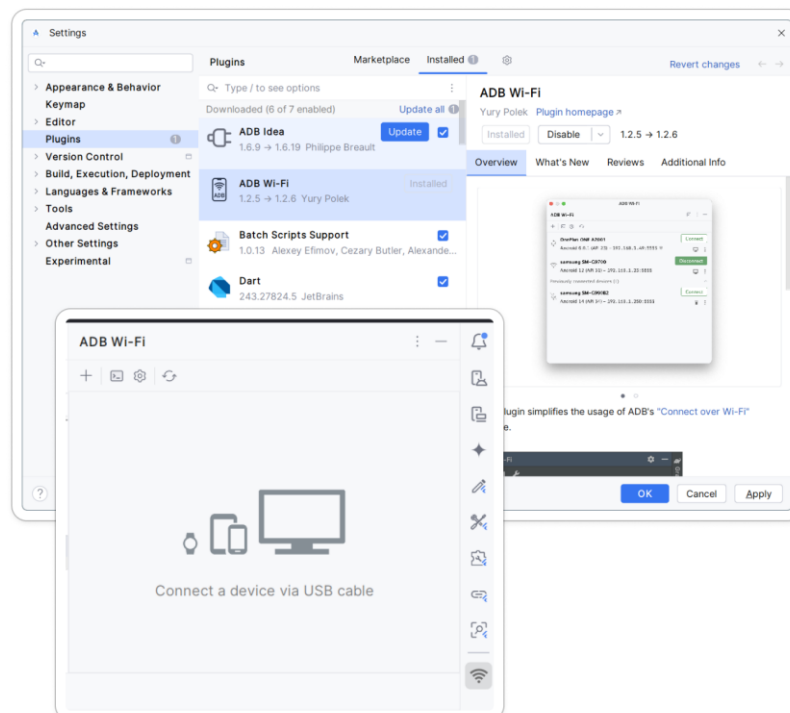
`flutter run -d (nombre del dispositivo)`

hacemos Enter y esperamos que se ejecute

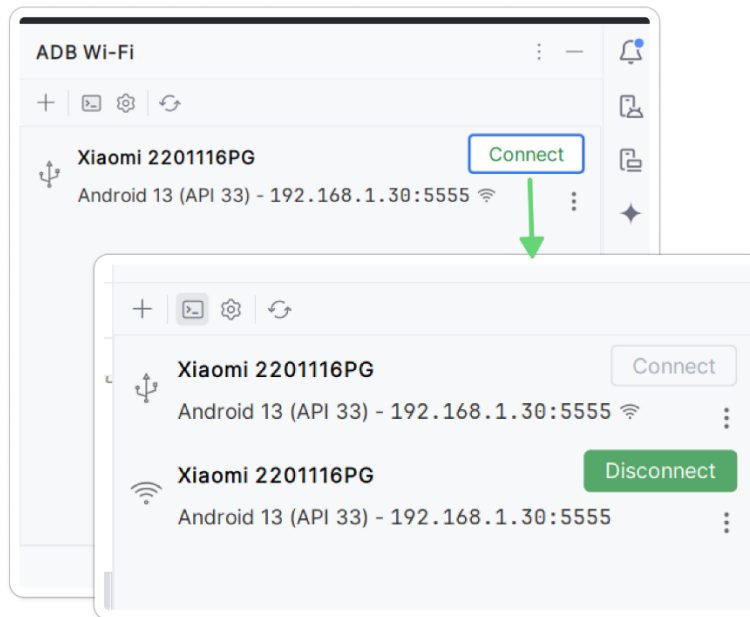
```
If you expected another device to be detected, please run "flutter doctor" to diagnose potential issues. You may also try increasing the time to wait for
connected devices with the "--device-timeout" flag. Visit https://flutter.dev/setup/ for troubleshooting tips.
❖ PS C:\flutterproyectos\Examen_2p_Domenica_Ortiz\vulnesis\prueba> # Ejecutar en el dispositivo por USB (más rápido)
>> flutter run -d R5CM711M44E
>>
>> # O ejecutar en el dispositivo por WiFi
>> flutter run -d 192.168.3.27:5555
Launching lib\main.dart on SM S918B in debug mode...
Running Gradle task 'assembleDebug'...
```

Opción 4: Extensión ADB desde el entorno de Android

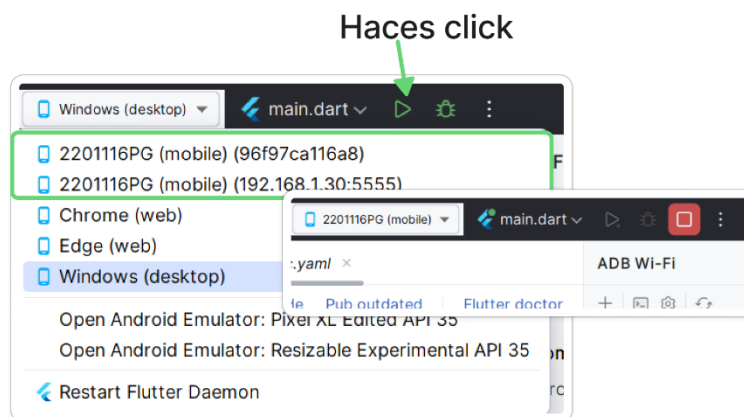
Paso 1: Descargar plugin ADB wifi en Android Studio



Paso 2: Conectar el cable y realiza la conexión:



PASO 3: Vamos al compilador en la parte superior y escogemos el dispositivo. Finalmente ejecutamos:



Paso 4: Aceptamos los permisos que nos pidan:

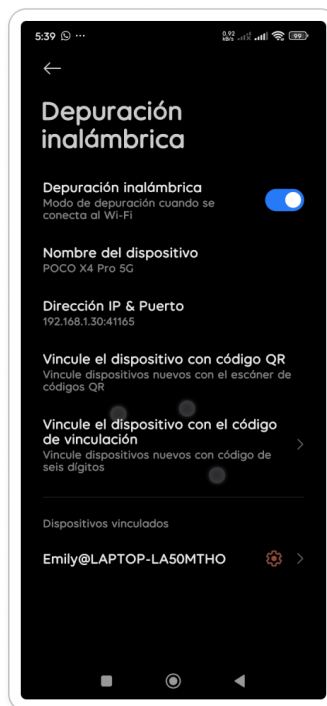


Conexiones posteriores con wifi

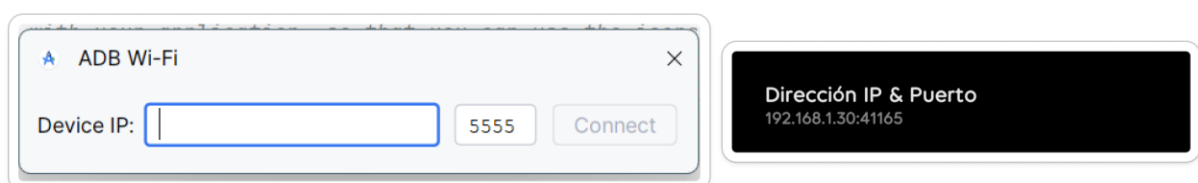
Paso 1: Realizamos una conexión:



Paso 2: Nos dirigimos en nuestro dispositivo y vamos a la parte de Depuración Inalámbrica y vemos el Id & puerto

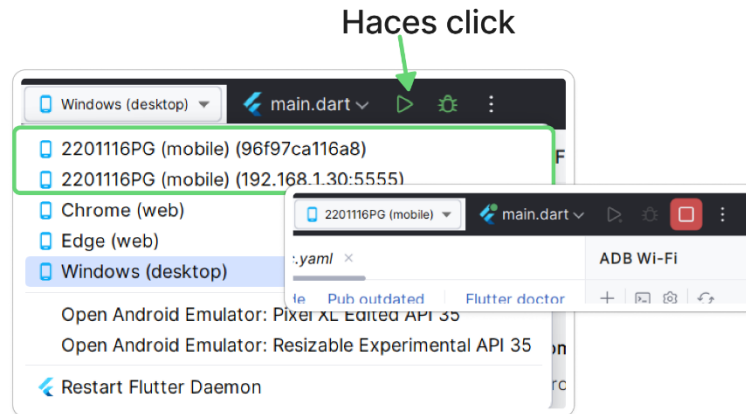


Paso 2: Colocamos el id en la parte izquierda y el puerto en el derecho



Realizamos la conexión, si llega la notificación del dispositivo acerca de la conexión, puedes cerrar la ventana de ADB wifi.

Paso 3: Vamos al compilador en la parte superior y escogemos el dispositivo. Finalmente ejecutamos:



7. CASOS DE USO

Se presenta una lista inicial de casos de uso del sistema "SismosApp", basada en el diagrama de flujo general, los wireframes y las funcionalidades descritas en el proyecto. Esta lista es preliminar y se enfoca en las interacciones principales de los usuarios según sus roles (Administrador, Inspector y Ayudante).

Los casos de uso serán desarrollados en detalle posteriormente por todo el equipo, incluyendo diagramas, actores involucrados, precondiciones, postcondiciones y escenarios alternativos.

1. Registro de usuario: El usuario ingresa datos como nombre, rol, correo, teléfono y contraseña para crear una cuenta nueva.
2. Inicio de sesión: El usuario ingresa correo y contraseña para acceder al sistema, redirigiendo según su rol.
3. Recuperación de contraseña: El usuario ingresa correo o teléfono para restablecer la contraseña mediante un enlace o código.
4. Visualización y edición de perfil: El usuario ve y actualiza sus datos personales (nombre, correo, rol, etc.).
5. Cierre de sesión: El usuario finaliza su sesión activa.
6. Visualización de panel de estadísticas (para Administrador): Muestra resúmenes de usuarios registrados, activos, inactivos, edificios evaluados y gráficos de tendencias.
7. Registro de nuevo edificio (para Inspector): Ingreso de datos del edificio como nombre, dirección, código postal, uso, latitud, longitud, inspector asignado, fecha, número de pisos, área total, año de construcción/ampliación, ocupación y tipo de suelo.
8. Evaluación de vulnerabilidad sísmica (para Inspector): Completar formulario FEMA con detalles como irregularidades verticales/planta, código sísmico, matriz de resultados, extensión de revisión, peligros geológicos y acciones requeridas.
9. Búsqueda de edificios o informes: Realizar búsquedas por inspector, edificio o código de informe.
10. Visualización de detalles de edificio o informe: Ver información completa de un edificio, incluyendo inspectores, fecha, resultados y opciones para ver detalles o resultados.
11. Consulta de informes FEMA (para Todos): Acceso a pantallas de consulta para ver resultados de evaluaciones existentes.
12. Visualización de resultados de evaluación: Mostrar matriz de resultados, índice sísmico y recomendaciones (e.g., evaluación estructural detallada).
13. Subida de archivos gráficos/fotos: Adjuntar imágenes o gráficos durante el registro o evaluación de edificios.

Tablas de Caso de uso

Identificador de Caso Uso	CU_01	
Nombre	Registro de usuario	
Descripción	El usuario ingresa sus datos personales para crear una cuenta nueva en el sistema.	
Actores	Usuario	
Secuencia normal		
Actor	Software	
1. El usuario selecciona la opción 'Registrarse'.		
	2. El sistema muestra formulario con campos: nombre, rol, correo, teléfono y contraseña.	
3. El usuario completa el formulario.		
	4. El sistema valida la información.	
	5. El sistema crea la cuenta y confirma el registro.	
Excepciones	Software	
	6. Si el correo ya está registrado, muestra error y solicita un correo diferente.	
CU relacionados	CU_02, CU_04	
Precondición	El usuario no debe estar registrado previamente.	
Post condición	La nueva cuenta queda activa en el sistema.	

Tabla 5: CU_01

Identificador de Caso Uso	CU_02	
Nombre	Inicio de sesión	
Descripción	El usuario ingresa correo y contraseña para acceder al sistema.	
Actores	Usuario	
Secuencia normal		
Actor	Software	
1. El usuario selecciona 'Iniciar sesión'.		
	2. El sistema solicita correo y contraseña.	
3. El usuario ingresa sus credenciales.		
	4. El sistema valida credenciales.	
	5. El sistema redirige al panel correspondiente según rol.	
Excepciones	Software	
	6. Si credenciales inválidas, muestra error y permite reintentar.	
CU relacionados	CU_01, CU_05	
Precondición	El usuario debe estar registrado.	
Post condición	El usuario accede al sistema con su rol asignado	

Tabla 6: CU_02

Identificador de Caso Uso	CU_03	
Nombre	Recuperación de contraseña	
Descripción	El usuario puede recuperar su contraseña mediante correo o teléfono.	
Actores	Usuario	
Secuencia normal		
Actor		Software
1. El usuario selecciona 'Olvidé mi contraseña'		
		2. El sistema solicita correo o teléfono.
3. El usuario ingresa el dato requerido.		
		4. El sistema valida que exista una cuenta asociada
		5. El sistema envía un enlace o código para restablecer la contraseña.
Excepciones		Software
		6. Si no se encuentra cuenta asociada, muestra mensaje de error.
CU relacionados	CU_02	
Precondición	El usuario debe tener una cuenta registrada.	
Post condición	El usuario recibe un medio para restablecer la contraseña	

Tabla 7: CU_03

Identificador de Caso Uso	CU_04	
Nombre	Visualización y edición de perfil	
Descripción	El usuario puede ver y modificar sus datos personales.	
Actores	Usuario	
Secuencia normal		
Actor	Software	
1. El usuario accede a 'Mi perfil'.		
	2. El sistema muestra la información actual del usuario.	
3. El usuario edita los datos que desee actualizar.		
	4. El sistema valida y guarda los cambios.	
	5. El sistema confirma la actualización.	
Excepciones	Software	
	6. Si datos inválidos, muestra error y solicita corrección.	
CU relacionados	CU_01	
Precondición	El usuario debe haber iniciado sesión	
Post condición	Los datos del usuario quedan actualizados.	

Tabla 8: CU_04

Identificador de Caso Uso	CU_05	
Nombre	Cierre de sesión	
Descripción	El usuario finaliza su sesión activa en el sistema.	
Actores	Usuario	
Secuencia normal		
Actor		Software
1. El usuario selecciona la opción 'Cerrar sesión'.		
		2. El sistema invalida el token de sesión.
		3. El sistema redirige a la pantalla de inicio.
Excepciones		Software
		4. Si ocurre error en cierre, se fuerza invalidación del token.
CU relacionados	CU_02	
Precondición	El usuario debe tener sesión activa.	
Post condición	El usuario queda desconectado del sistema.	

Tabla 9: CU_05

Identificador de Caso Uso	CU_06	
Nombre	Visualización de panel de estadísticas	
Descripción	El administrador visualiza métricas sobre usuarios y edificios evaluados.	
Actores	Usuario	
Secuencia normal		
Actor		Software
1. El administrador accede al panel de estadísticas. '		
		2. El sistema consulta la base de datos.
		3. El sistema muestra resúmenes: usuarios registrados, activos, inactivos, edificios evaluados, tendencias gráficas.
Excepciones		Software
		4.. Si no hay datos, se muestra panel vacío con mensaje informativo.
CU relacionados	CU_07, CU_08	
Precondición	El usuario debe tener rol de Administrador.	
Post condición	El administrador obtiene un panorama estadístico del sistema.	

Tabla 10: CU_06

Identificador de Caso Uso	CU_07	
Nombre	Registro de nuevo edificio	
Descripción	El inspector ingresa información completa de un edificio.	
Actores	Inspector	
Secuencia normal		
Actor		Software
1. El inspector selecciona 'Registrar edificio'.		
		2. El sistema muestra formulario con campos requeridos.
3. El inspector completa datos como nombre, dirección, código postal, uso, ubicación, área, ocupación, tipo de suelo.		
		4. El sistema valida datos ingresados.
		5. El sistema guarda el edificio en la base de datos.
Excepciones		Software
		6. Si algún campo es inválido, muestra error y retorna al formulario.
CU relacionados	CU_08, CU_10	
Precondición	El inspector debe estar autenticado.	
Post condición	El edificio queda registrado en la base de datos.	

Tabla 11: CU_07

Identificador de Caso Uso	CU_08	
Nombre	Evaluación de vulnerabilidad sísmica	
Descripción	El inspector completa el formulario FEMA P-154 sobre un edificio registrado.	
Actores	Inspector	
Secuencia normal		
Actor		Software
1. El inspector selecciona un edificio registrado.		
		2. El sistema muestra formulario FEMA.
3. El inspector completa detalles de vulnerabilidad, irregularidades y riesgos.		
		4. El sistema calcula índice sísmico y guarda los resultados.
		5. El sistema confirma el registro de evaluación.
Excepciones		Software
		6. Si faltan campos, se notifica y no se guarda evaluación.
CU relacionados	CU_07, CU_11	
Precondición	Debe existir un edificio registrado.	
Post condición	Los resultados de vulnerabilidad quedan almacenados.	

Tabla 12: CU_08

Identificador de Caso Uso	CU_09	
Nombre	Búsqueda de edificios o informes	
Descripción	El usuario realiza búsquedas por inspector, edificio o código de informe.	
Actores	Usuario	
Secuencia normal		
Actor		Software
1. El usuario accede a la opción de búsqueda.		
		2. El sistema muestra campos de búsqueda.
3. El usuario ingresa criterios de búsqueda.		
		4. El sistema muestra resultados coincidentes.
Excepciones		Software
		5. Si no se encuentran resultados, muestra mensaje de 'sin coincidencias'.
CU relacionados	CU_10, CU_11	
Precondición	El usuario debe estar autenticado.	
Post condición	Se muestran resultados de búsqueda.	

Tabla 13: CU_09

Identificador de Caso Uso	CU_10	
Nombre	Visualización de detalles de edificio o informe	
Descripción	El usuario consulta información completa de un edificio o informe.	
Actores	Usuario	
Secuencia normal		
Actor		Software
1. El usuario selecciona un resultado de búsqueda.		
		2. El sistema muestra detalles completos: datos del edificio, inspecciones, resultados.
3. El usuario navega entre pestañas de detalles.		
Excepciones		Software
		4. Si datos incompletos, muestra advertencia.
CU relacionados	CU_09, CU_08	
Precondición	El edificio o informe debe existir en el sistema.	
Post condición	El usuario obtiene información completa.	

Tabla 14: CU_10

Identificador de Caso Uso	CU_11		
Nombre	Consulta de informes FEMA		
Descripción	El lector usuario puede acceder a informes de evaluaciones previas.		
Actores	Lector		
Secuencia normal			
Actor		Software	
1. El lector ingresa a la sección de informes.			
		2. El sistema muestra listado de informes FEMA disponibles.	
3. El lector selecciona un informe			
		4. El sistema muestra los resultados de la evaluación FEMA.	
Excepciones		Software	
		5. Si no hay informes, muestra mensaje de 'sin datos disponibles'.	
CU relacionados	CU_08, CU_10		
Precondición	El lector debe estar autenticado.		
Post condición	El lector obtiene información de evaluaciones existentes.		

Tabla 15: CU_11

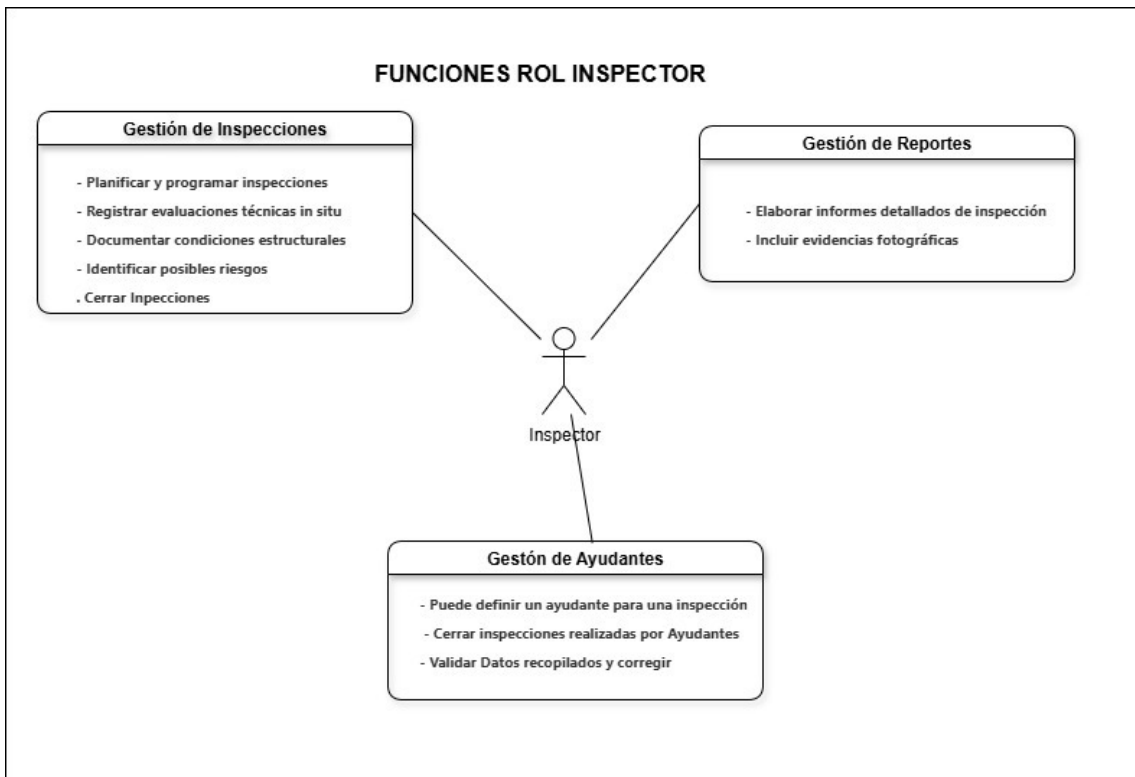
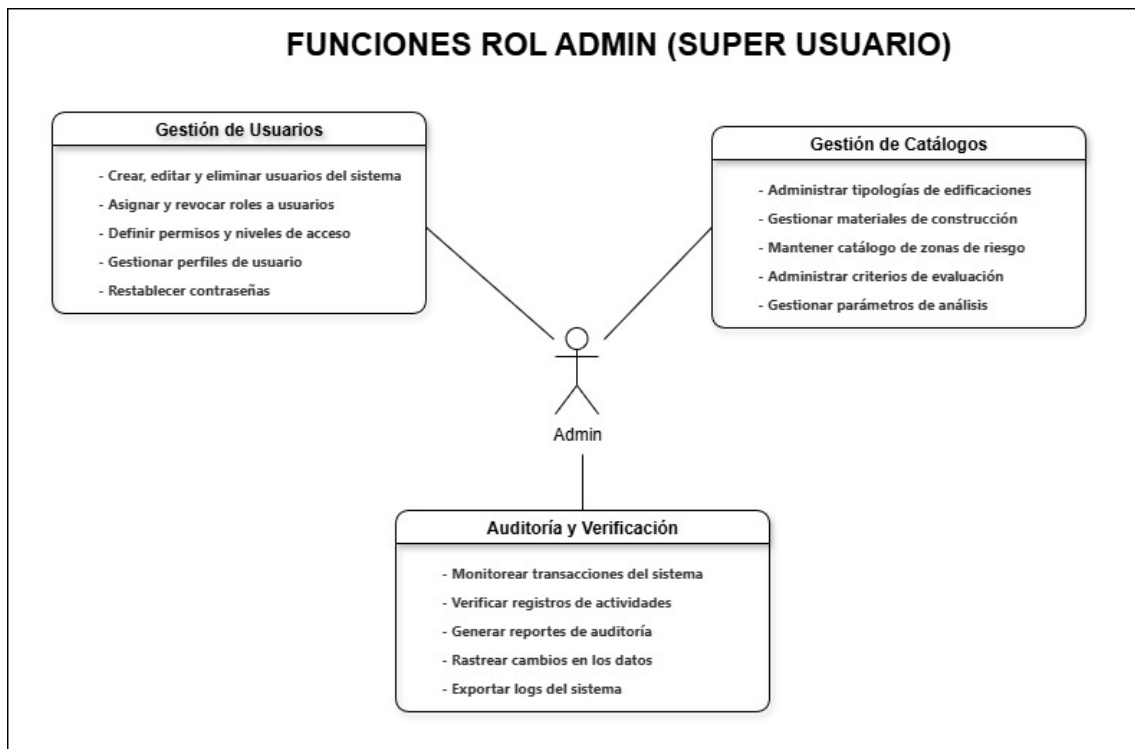
Identificador de Caso Uso	CU_12		
Nombre	Visualización de resultados de evaluación		
Descripción	Permite a los usuarios consultar los resultados detallados de una evaluación sísmica.		
Actores	Usuario, Inspector, Sistema.		
Secuencia normal			
Actor		Software	
1. El usuario selecciona un informe o evaluación registrada.			
		2. El sistema carga y muestra la matriz de resultados SL1 , el índice sísmico calculado y las recomendaciones derivadas. El sistema solicita correo o teléfono.	
3. El usuario navega entre secciones (matriz, observaciones, recomendaciones).			
Excepciones		Software	
		4. Si los resultados no han sido calculados, el sistema muestra un mensaje “Evaluación pendiente de cálculo”.	
CU relacionados	CU_09. CU_07		
Precondición	Debe existir un registro de evaluación con resultados almacenados.		
Post condición	El usuario visualiza correctamente los resultados de la evaluación.		

Tabla 16: CU_12

Identificador de Caso Uso	CU_13
<i>Tabla 17: CU_13</i>	Subida de archivos gráficos/fotos
Nombre	
Descripción	Permite al inspector adjuntar fotografías o gráficos durante el registro o evaluación de edificios.
Actores	Inspector, Sistema.
Secuencia normal	
Actor	Software
1. inspector adjunta fotografías o gráficos durante el registro o evaluación de edificios.	
	2. El sistema permite elegir uno o varios archivos desde la galería o cámara.
3. El inspector confirma la subida.	
	4. El sistema guarda los archivos en Supabase Storage y asocia sus URLs al edificio/evaluación correspondiente
Excepciones	Software
	5. Si el archivo excede el tamaño permitido, el sistema muestra un mensaje de error.
	6. Si el archivo no tiene un formato válido (ejemplo: .jpg, .png, .pdf), se rechaza la subida
CU relacionados	CU_07, CU_08
Precondición	El usuario debe estar autenticado como inspector y tener un edificio/evaluación en proceso.
Post condición	Los archivos quedan almacenados y asociados al registro correspondiente.

Graficos de caso de uso

Usuarios



FUNCIONES ROL AYUDANTE

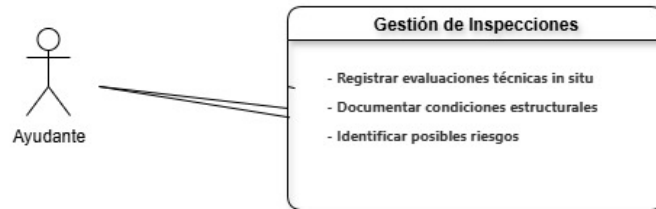


DIAGRAMA DE SECUENCIAS

Diagrama de Secuencia para Registro de Edificios

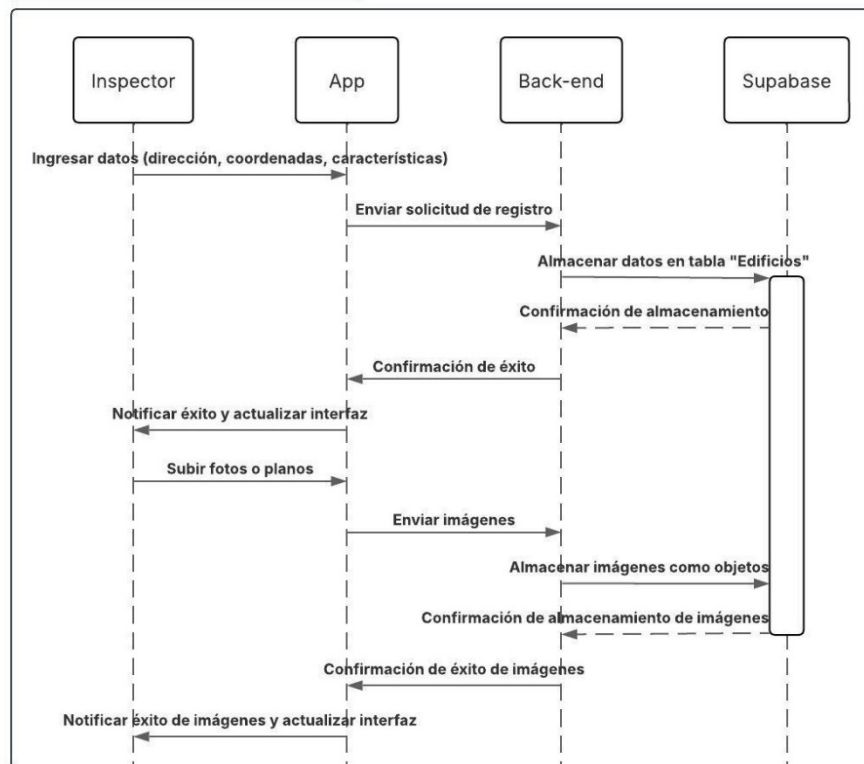


Diagrama de Secuencia para Generación de Reportes

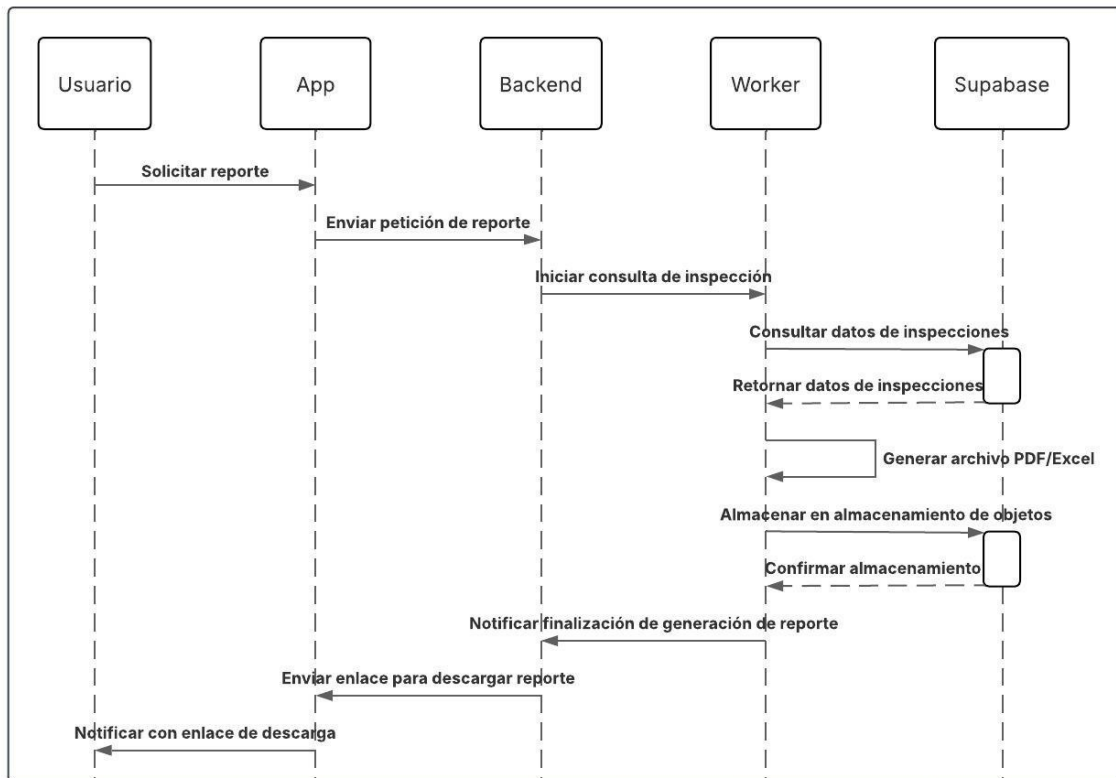


Diagrama de Secuencia de Autenticación de Usuario

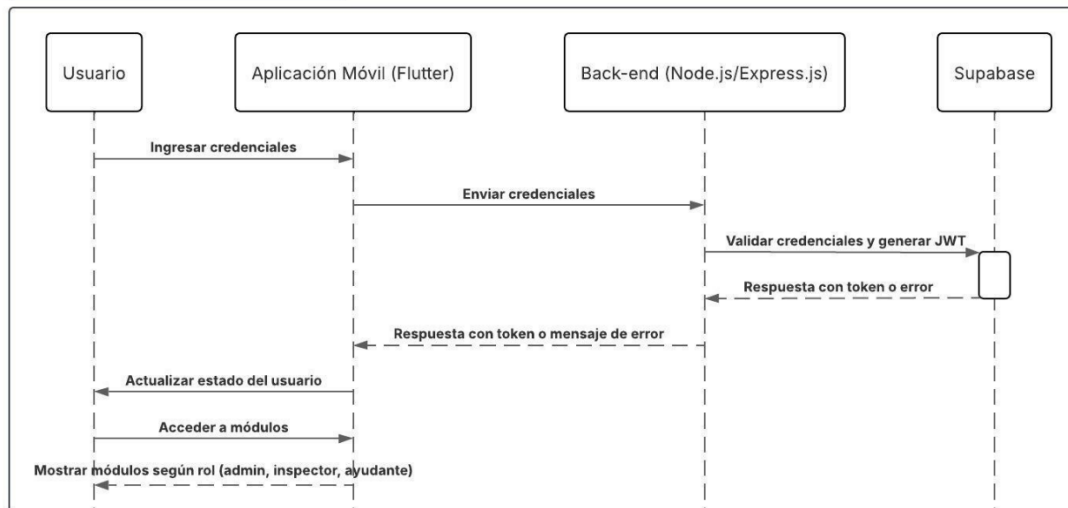
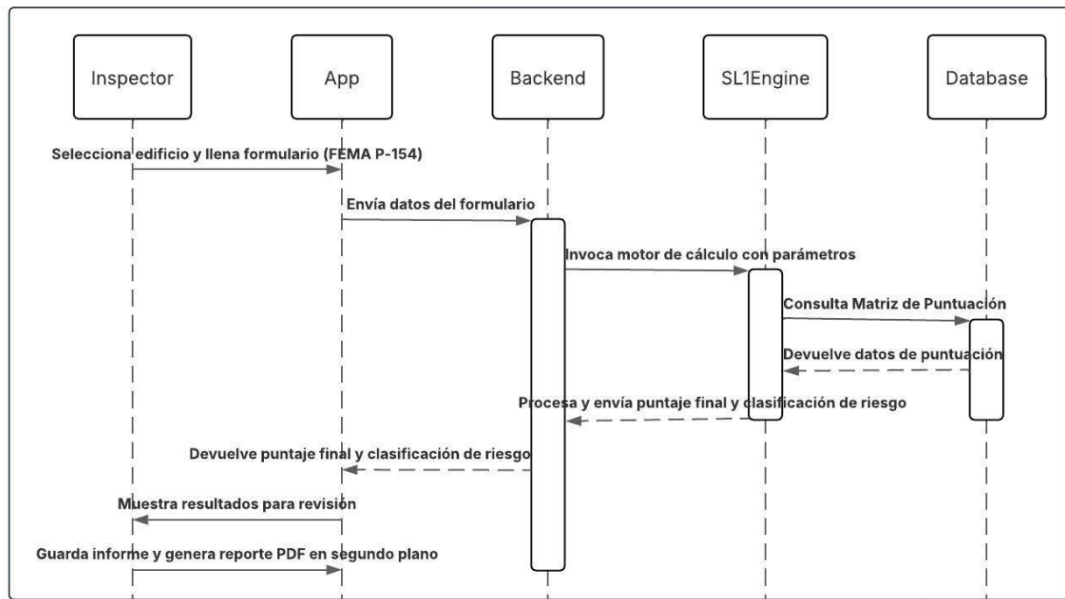


Diagrama de Secuencia para Evaluación de Vulnerabilidad Sísmica



8. MÓDULOS

8.1. Módulo de Autenticación de Usuario

Este módulo gestiona el acceso de los usuarios al sistema. Su función principal es validar la identidad del usuario a través de la autenticación de credenciales, como el **nombre de usuario** y la **contraseña**.

Además de las funciones básicas de inicio de sesión, el módulo incluye las siguientes opciones para mejorar la experiencia de usuario y la seguridad:

- **¿Olvidaste tu contraseña?:** Permite a los usuarios restablecer su contraseña si la han olvidado. Esto puede redirigirlos a un formulario de recuperación o a un proceso de envío de enlace al correo electrónico.
- **¿Aún no tienes una cuenta?:** Dirige a los usuarios nuevos al proceso de registro.
- **Registrarse:** Inicia el flujo de creación de una nueva cuenta de usuario, donde se solicitará información como nombre, correo electrónico y una contraseña.

8.2. Módulo de Registro de Usuarios

Este módulo se encarga de gestionar la creación de nuevas cuentas de usuario en el sistema. Su propósito es capturar y almacenar de manera segura la información necesaria para que un nuevo usuario pueda acceder y utilizar las funcionalidades del proyecto.

El proceso de registro solicita los siguientes datos al usuario:

- **Usuario:** Nombre de usuario único que servirá como identificador para iniciar sesión.
- **Rol:** Define los permisos y el nivel de acceso que tendrá el usuario dentro del sistema.
- **Correo:** Dirección de correo electrónico del usuario, utilizada para verificaciones de cuenta y comunicaciones importantes.
- **Teléfono:** Número de contacto del usuario.
- **Contraseña y Confirmar contraseña:** Permite al usuario establecer una contraseña segura para su cuenta, la cual debe ingresarse dos veces para asegurar que no haya errores de tipeo.

Una vez completado el registro, el módulo valida la información y crea una nueva entrada en la base de datos, lo que permite al usuario iniciar sesión en el **Módulo de Autenticación de Usuario**.

8.3. Módulo de Consulta de Edificaciones

Este módulo provee una interfaz para buscar y visualizar la información detallada de todas las edificaciones evaluadas o programadas para ser evaluadas. Su objetivo es facilitar el acceso a los datos de las inspecciones sísmicas, permitiendo a los usuarios generar informes completos de cada estructura.

El módulo ofrece dos criterios de búsqueda principales:

- **Búsqueda por Inspector:** Permite a los usuarios encontrar edificios basándose en el inspector asignado.
- **Búsqueda por Edificio:** Permite buscar una edificación específica usando su nombre, código u otro identificador.

Una vez que se ha realizado la búsqueda, se muestra un listado de edificaciones que coinciden con los criterios. Al seleccionar un edificio del listado, el sistema despliega un resumen del informe de inspección que incluye el código de informe, la fecha de la evaluación y el nombre del inspector. Este resumen también puede mostrar imágenes relevantes de la edificación.

Finalmente, el módulo incluye dos opciones para obtener más detalles:

- **Ver Detalle del Informe:** Muestra el informe completo de la evaluación, que contiene datos técnicos, observaciones y recomendaciones del inspector.
- **Ver Resultados:** Presenta los resultados de la evaluación de vulnerabilidad sísmica, incluyendo la calificación o el nivel de riesgo asignado a la estructura.

8.4. 8. Módulo de Registro de Edificios

Este módulo gestiona la entrada de datos de las edificaciones que serán evaluadas. Su propósito es capturar de manera sistemática toda la información relevante de cada estructura, lo que servirá como base para los análisis de vulnerabilidad sísmica.

El proceso de registro incluye:

- **Archivos adjuntos:**

Gráfico: Archivo de la vista de la edificación (plano o diagrama).

Foto del edificio: Imagen fotográfica de la edificación.

- **Detalles básicos:**

Nombre del edificio: Nombre de la edificación a registrar.

Dirección: Ubicación completa de la estructura.

Código postal: Código postal asociado a la dirección.

Uso del edificio: Categorización del uso (ej. residencial, comercial, industrial, educativo, etc.).

Latitud y longitud: Coordenadas geográficas exactas de la edificación para su geolocalización.

- **Detalles técnicos:**

Número de pisos: Cantidad de pisos o niveles de la edificación.

Área total de piso (m2): Área construida total del edificio expresada en metros cuadrados.

Año de construcción: Año en el que se finalizó la construcción inicial del edificio.

Ampliación: Opción de **Sí** o **No**. Si se selecciona **Sí**, se debe ingresar el **año de ampliación**.

- **Información del registro:**

Inspector: Nombre del inspector a cargo de la evaluación.

Fecha y hora del registro: Momento exacto en que se ingresan los datos al sistema.

- **Tipo de ocupación:** Categorización del uso del edificio, incluyendo las opciones: asamblea, comercial, servicios empresariales, industria, oficina, escuela, almacén, residencial y herramientas.

- **Tipo de suelo:** Clasificación del suelo donde se encuentra la edificación. Los tipos se categorizan por letras:

A: Roca dura

B: Roca semi-dura

C: Suelo denso

D: Suelo rígido

E: Suelo blando

F: Suelo pobre

- **Matriz de Resultados SL1: Identificación y Nivel de Vulnerabilidad**

La matriz de resultados SL1 (Serviceability Level 1) es una herramienta clave para la identificación y clasificación inicial del nivel de vulnerabilidad sísmica de los edificios evaluados. Esta matriz integra dos componentes principales: la identificación del edificio y su resultado final de vulnerabilidad de Nivel 1.

- **Extensión de Revisión (Exterior/Interior)**

Esta opción determina el alcance físico de la inspección: Exterior/Interior.

Revisión de Planos: La revisión de planos es fundamental para validar la información recopilada en campo. Se debe indicar si se realizó o no.

Fuente de Tipo de Suelo: Con su respectivo estado

Fuente de Peligros Geológicos: Con su respectivo estado

Contacto de la Persona: Se debe registrar la información de contacto del responsable de la evaluación o de la persona que proporcionó el acceso al edificio, incluyendo su nombre. Esto es vital para futuras inspecciones o aclaraciones.

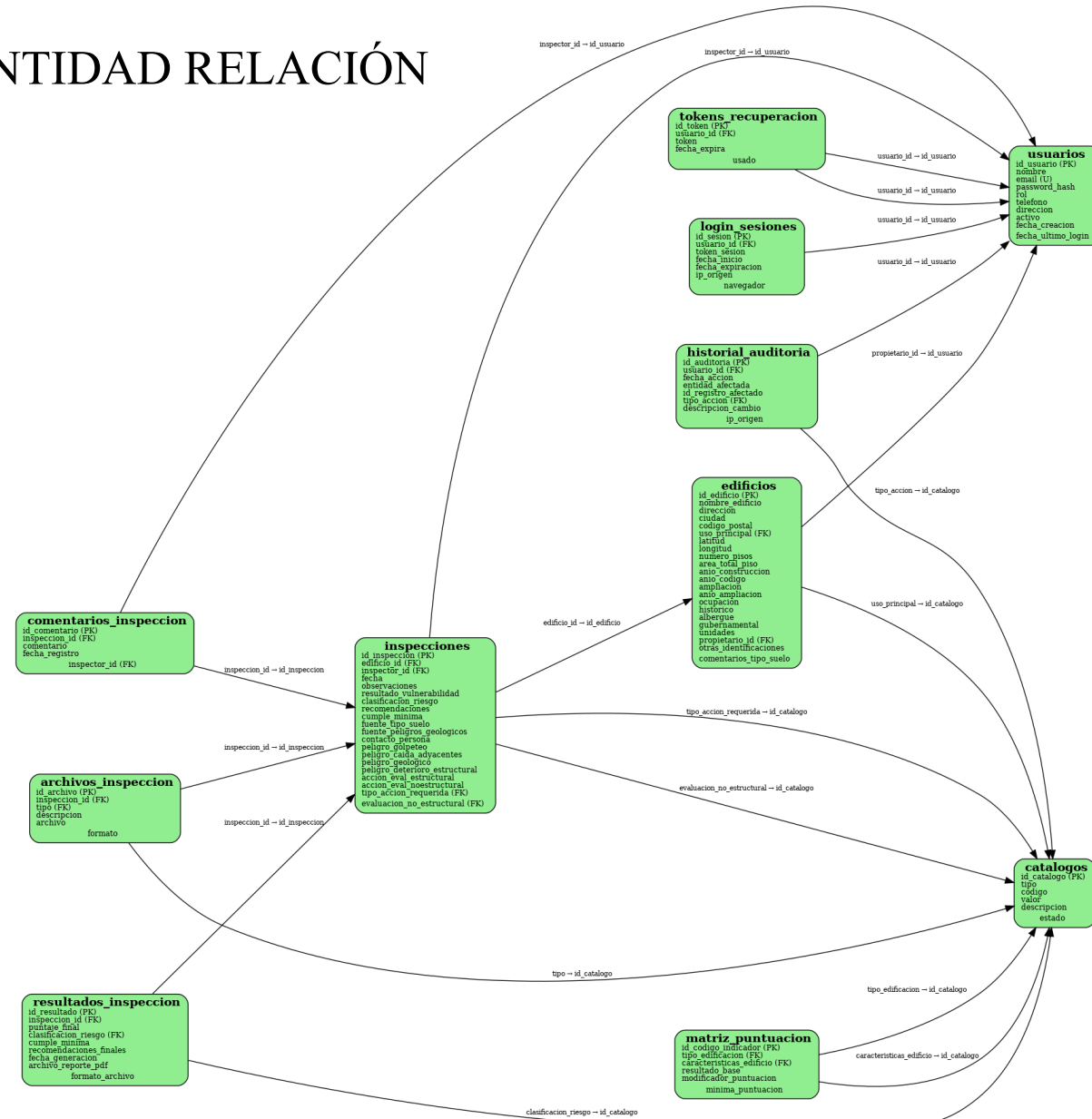
¿Requiere Inspección Nivel 2?

Esta pregunta es el punto de conexión con el siguiente módulo.

Sí: Se marca esta opción si la evaluación de Nivel 1 (SL1) arroja un resultado de **Vulnerable**.

No: Se marca si el resultado es **Vulnerable Bajo** o **Medio**, y no se observaron indicios de daño estructural grave que justifique un estudio más exhaustivo.

9. MODELO DE ENTIDAD RELACIÓN



10. DICCIONARIO DE DATOS DEL MODELO ENTIDAD RELACIÓN

1. Autenticación y seguridad

1.1. Usuarios

<u>Column Name</u>	<u>Column Description</u>	<u>Data Type</u>	<u>Length</u>	<u>PK</u>	<u>Nullable</u>	<u>Unique</u>	<u>FK</u>	<u>Restricciones / Ejemplo</u>
<u>id_usuario</u>	<u>Identificador único del usuario</u>	UUID	==	<u>true</u>	<u>false</u>	<u>true</u>	==	<u>Autoincremental</u>
<u>cedula</u>	<u>Número de cédula del usuario</u>	VARCHAR	10	<u>false</u>	<u>false</u>	<u>true</u>	==	<u>Solo números, longitud 10 dígitos ('0923456789')</u>
<u>nombre</u>	<u>Nombre completo del usuario</u>	VARCHAR	100	<u>false</u>	<u>false</u>	<u>false</u>	==	<u>Solo letras y espacios ('Carlos Pérez')</u>
<u>photo_perfil</u>	<u>Fotografía de perfil del usuario</u>	VARCHAR	255	<u>false</u>	<u>true</u>	<u>false</u>	==	<u>URL o ruta de la imagen ('/images/perfil/usuario1.png')</u>
<u>email</u>	<u>Correo electrónico para autenticación</u>	VARCHAR	150	<u>false</u>	<u>false</u>	<u>true</u>	==	<u>Formato válido de email ('usuario@correo.com')</u>
<u>password_hash</u>	<u>Contraseña cifrada del usuario</u>	VARCHAR	255	<u>false</u>	<u>false</u>	<u>false</u>	==	<u>Hash bcrypt o equivalente</u>

<u>rol</u>	<u>Tipo de usuario en el sistema</u>	<u>VARCHAR</u>	<u>20</u>	<u>false</u>	<u>false</u>	<u>false</u>	<u>==</u>	Valores: 'admin', 'inspector', 'ayudante' <u>Pendiente</u> . Debe existir al menos un admin. El rol inicial de un nuevo usuario es <u>Pendiente</u>
<u>telefono</u>	<u>Número telefónico</u>	<u>VARCHAR</u>	<u>20</u>	<u>false</u>	<u>true</u>	<u>false</u>	<u>==</u>	<u>Formato E.164 (+593987654321)</u>
<u>direccion</u>	<u>Dirección física del usuario</u>	<u>VARCHAR</u>	<u>255</u>	<u>false</u>	<u>true</u>	<u>false</u>	<u>==</u>	<u>Ej: 'Av. Central 123, Quito'</u>
<u>activo</u>	<u>Estado del usuario (activo/inactivo)</u>	<u>BOOLEAN</u>	<u>==</u>	<u>false</u>	<u>false</u>	<u>false</u>	<u>==</u>	<u>DEFAULT TRUE. Si es FALSE, no puede iniciar sesión</u>
<u>fecha_creacion</u>	<u>Fecha y hora de registro del usuario</u>	<u>timestampz</u>	<u>==</u>	<u>false</u>	<u>false</u>	<u>false</u>	<u>==</u>	<u>DEFAULT now()</u>
<u>fecha_ultimo_login</u>	<u>Fecha y hora del último inicio de sesión</u>	<u>timestampz</u>	<u>==</u>	<u>false</u>	<u>true</u>	<u>false</u>	<u>==</u>	<u>Se actualiza en cada inicio exitoso</u>

Tabla 18: Usuarios

Roles propuestos

• Admin (Super Admin / Coordinador General)

- Asigna roles (inspector, ayudante).
- Gestiona catálogos (tipos de suelo, materiales, tipologías).
- Ve auditoría completa.

• Inspector (rol operativo principal)

- Realiza inspecciones y genera reportes.
- Puede asignar ayudantes a sus inspecciones.

• Ayudante (rol de apoyo)

- Registra datos de campo, sube fotos, llena formularios.
- No puede cerrar ni validar inspecciones

1.2. Tokens de recuperación

Tokens para recuperación de contraseña

<u>Column Name</u>	<u>Column Description</u>	<u>Data Type</u>	<u>Length</u>	<u>PK</u>	<u>Nullable</u>	<u>Unique</u>	<u>FK</u>	<u>Restricciones / Ejemplo</u>
<u>id_token</u>	Identificador único del token	integer	=	true	false	true	=	Autoincremental
<u>usuario_id</u>	Usuario que solicitó el token	integer	=	false	false	false	usuarios.usuario_id	Debe existir en usuarios
<u>token</u>	Código único para recuperación	varchar	255	false	false	true	=	Hash aleatorio
<u>fecha_expira</u>	Fecha y hora de expiración	timestamp ptz	=	false	false	false	=	2025-08-12 10:00:00
<u>usado</u>	Indica si ya fue usado	boolean	=	false	false	false	=	Default FALSE

Tabla 19 Tokens de recuperación

2. Formulario

2.1. Entity Name: edificios

Entity Description: Información general y única de cada edificación registrada en el sistema.

Column Name	Column Description	Data Type	Length	PK	Nullable	Unique	FK	Restricciones / Ejemplo
id_edificio	Identificador único del edificio	UUID	-	true	false	true	-	Autoincremental
nombre_edificio	Nombre o referencia del edificio	VARCHAR	100	false	false	false	-	Ej: "Torre Central"
direccion	Dirección completa	VARCHAR	255	false	false	false	-	Ej: "Av. Central 123"
ciudad	Ciudad donde se ubica	VARCHAR	100	false	false	false	-	Ej: "Quito"
codigo_postal	Código postal de la ubicación	VARCHAR	10	false	true	false	-	Ej: "110101"
uso_principal	Tipo de uso principal del edificio	VARCHAR	50	false	false	false	catalogos.id_catalogo	Ej: "Residencial", "Comercial"
latitud	Coordenada geográfica norte-sur	DECIMAL	9,6	false	false	false	-	Ej: -0.225219
longitud	Coordenada geográfica este-oeste	DECIMAL	9,6	false	false	false	-	Ej: -78.524948
numero_pisos	Número total de pisos incluyendo subsuelos	SMALLINT	-	false	false	false	-	Ej: 12
area_total_piso	Área total construida en m²	NUMERIC	10,2	false	false	false	-	Ej: 1500.50
anio_construccion	Año en que se construyó el edificio	SMALLINT	-	false	false	false	-	Ej: 1998
anio_codigo	Año del código de construcción aplicado	SMALLINT	-	false	false	false	-	Ej: 2011
ampliacion	Indica si el edificio ha tenido ampliaciones	BOOLEAN	-	false	false	false	-	Valores: true/false
anio_ampliacion	Año en que se realizó la ampliación	SMALLINT	-	false	true	false	-	Solo aplica si ampliacion = true
ocupacion	Actividad principal	VARCHAR	50	false	false	false	-	Ej: "Oficinas"
historico	Indica si el edificio tiene valor histórico	BOOLEAN	-	false	false	false	-	Valores: true/false

albergue	Indica si es usado como albergue	BOOLEAN	-	false	false	false	-	Valores: true/false
gubernamental	Indica si pertenece a una entidad gubernamental	BOOLEAN	-	false	false	false	-	Valores: true/false
unidades	Número de unidades (viviendas/locales)	SMALLINT	-	false	false	false	-	Ej: 20
propietario_id	Usuario propietario o cliente	INT	-	false	false	false	usuarios.id_usuario	Debe existir en usuarios
otras_identificaciones	Identificadores adicionales	TEXT	-	false	true	false	-	Ej: "Parcela 45, Registro Catastral 1234"
comentarios_tipo_suelo	Comentarios adicionales sobre el tipo de suelo	TEXT	-	false	true	false	-	Ej: "Suelo denso con capa superficial arenosa"

Tabla 20: Formulario

2.2. Entity Name: inspecciones

Entity Description: Registro detallado de cada visita de inspección realizada a un edificio.

Column Name	Column Description	Data Type	Length	PK	Nullable	Unique	FK	Restricciones / Ejemplo
id_inspeccion	Identificador único de inspección	UUID	-	true	false	true	-	Autoincremental
edificio_id	Edificio inspeccionado	INT	-	false	false	false	edificios.id_edificio	Debe existir en edificios
inspector_id	Usuario inspector	INT	-	false	false	false	usuarios.id_usuario	Debe existir en usuarios
fecha	Fecha y hora de la inspección	TIMESTAMP	-	false	false	false	-	Ej: "2025-08-11 14:30:00"
observaciones	Observaciones generales	TEXT	-	false	true	false	-	Ej: "Desgaste en fachada"
resultado_vulnerabilidad	Puntaje de vulnerabilidad calculado (FEMA)	DECIMAL	5,2	false	false	false	-	Ej: 3.5
clasificacion_riesgo	Nivel de riesgo	VARCHAR	20	false	false	false	-	Valores: "Bajo", "Medio", "Alto"
recomendaciones	Acciones sugeridas	TEXT	-	false	true	false	-	Ej: "Reforzar estructura"

cumple_minima	Indica si la puntuación cumple el mínimo FEMA	BOOLEAN	-	false	false	false	-	Valores: true/false
fuelle_tipo_suelo	Fuente para determinar el tipo de suelo	TEXT	-	false	true	false	-	Ej: "Estudio geotécnico 2023"
fuelle_peligros_geologicos	Fuente para peligros geológicos	TEXT	-	false	true	false	-	Ej: "Mapa IGEPN 2024"
contacto_persona	Contacto que proporcionó información	TEXT	-	false	true	false	-	Ej: "Arq. Luis Pérez"
peligro_golpeteo	Riesgo de golpeteo con edificios cercanos	BOOLEAN	-	false	true	false	-	Valores: true/false
peligro_caída_adyacentes	Riesgo de caída de edificios adyacentes	BOOLEAN	-	false	true	false	-	Valores: true/false
peligro_geologico	Peligro geológico o suelo tipo F	BOOLEAN	-	false	true	false	-	Valores: true/false
peligro_deterioro_estructural	Deterioro estructural significativo	BOOLEAN	-	false	true	false	-	Valores: true/false
accion_eval_estructural	Requiere evaluación estructural detallada	BOOLEAN	-	false	true	false	-	Valores: true/false
accion_eval_noestructural	Requiere evaluación no estructural	BOOLEAN	-	false	true	false	-	Valores: true/false
tipo_accion_requerida	Tipo de acción requerida según FEMA	VARCHAR	255	false	true	false	catalogos.id_catalogo	Ej: "Si, resultado menor al mínimo"
evaluacion_no_estructural	Evaluación de elementos no estructurales según FEMA	VARCHAR	255	false	true	false	catalogos.id_catalogo	Ej: "No se sabe"

Tabla 21: Inspecciones

2.3. Entity Name: `archivos_inspeccion`

Entity Description: Archivos asociados a una inspección, como fotos, planos y documentos.)

Column Name	Column Description	Data Type	Length	PK	Nullable	Unique	FK	Restricciones / Ejemplo
<code>id_archivo</code>	Identificador único	UUID	-	true	false	true	-	Autoincremental
<code>inspeccion_id</code>	Inspección asociada	INT	-	false	false	false	<code>inspecciones.id_inspeccion</code>	Debe existir en inspecciones
<code>tipo</code>	Tipo de archivo	VARCHAR	20	false	false	false	<code>catalogos.id_catalogo</code>	Ej: "foto_fachada", "plano"
<code>descripcion</code>	Descripción opcional	TEXT	-	false	true	false	-	Ej: "Vista principal desde el norte"
<code>archivo</code>	Archivo binario o ruta	BYTEA	-	false	true	false	-	Formatos: JPEG, PNG, PDF
<code>formato</code>	Formato del archivo	TEXT (URL de Supabase Storage)	10	false	true	false	-	Ej: "JPEG"

Tabla 22: `archivos_inspeccion`

2.4. Entity Name: `comentarios_inspeccion`

Entity Description: Observaciones textuales realizadas durante una inspección.

Column Name	Column Description	Data Type	Length	PK	Nullable	Unique	FK	Restricciones / Ejemplo
<code>id_comentario</code>	Identificador único	UUID	-	true	false	true	-	Autoincremental
<code>inspeccion_id</code>	Inspección asociada	INT	-	false	false	false	<code>inspecciones.id_inspeccion</code>	Debe existir en inspecciones
<code>comentario</code>	Texto con la observación	TEXT	-	false	false	false	-	Ej: "Suelo tipo D por falta de información"
<code>fecha_registro</code>	Fecha y hora registro	timestampz	-	false	false	false	-	Formato: YYYY-MM-DD HH:MM:SS
<code>inspector_id</code>	Inspector que ingresó	INT	-	false	false	false	<code>usuarios.id_usuario</code>	Debe existir en usuarios

Tabla 23: `comentarios_inspeccion`

2.5. Entity Name: `catalogo_inspeccion`

Entity Description: Lista unificada de catálogos y sus elementos para uso en campos controlados.

Column Name	Column Description	Data Type	Length	PK	Nullable	Unique	FK	Restricciones / Ejemplo
<u>id_catalogo</u>	<u>Identificador único</u>	UUID	=	<u>true</u>	<u>false</u>	<u>true</u>	=	<u>Autoincremental</u>
<u>tipo</u>	<u>Tipo de catálogo</u>	<u>VARCHAR</u>	<u>50</u>	<u>false</u>	<u>false</u>	<u>false</u>	=	<u>Ej: "tipo_suelo", "tipo_material", "tipo_archivo", "tipo_accion_requerida"</u>
<u>codigo</u>	<u>Código del elemento</u>	<u>VARCHAR</u>	<u>20</u>	<u>false</u>	<u>false</u>	<u>false</u>	=	<u>Ej: "SUELO_A"</u>
<u>valor</u>	<u>Valor descriptivo</u>	<u>VARCHAR</u>	<u>100</u>	<u>false</u>	<u>false</u>	<u>false</u>	=	<u>Ej: "Arcilla"</u>
<u>descripcion</u>	<u>Descripción del elemento</u>	<u>TEXT</u>	=	<u>false</u>	<u>true</u>	<u>false</u>	=	<u>Ej: "Suelo arcilloso de baja resistencia"</u>
<u>estado</u>	<u>Estado del elemento</u>	<u>CHAR</u>	<u>1</u>	<u>false</u>	<u>false</u>	<u>false</u>	=	<u>'A' (Activo) o 'I' (Inactivo)</u>

Tabla 24: catalogo_inspeccion

2.6. Entity Name: matriz_puntuacion

Entity Description: Contiene los valores base y modificadores de puntuación FEMA P-154 para cada tipo y característica de edificación, permitiendo calcular el puntaje final y evaluar si cumple con la mínima puntuación requerida.

Column Name	Column Description	Data Type	Length	PK	Nullable	Unique	FK	Restricciones / Ejemplo
<u>id_codigo_indicador</u>	<u>Identificador único del indicador</u>	UUID	=	true	false	true	=	<u>Autoincremental</u>
<u>tipo_edificacion</u>	<u>Tipo de edificación según catálogo FEMA</u>	<u>VARCHAR</u>	<u>10</u>	<u>false</u>	<u>false</u>	<u>false</u>	<u>catalogos.id_catalogo</u>	Ej: “W1”, “S1”, “C1”, “URM”
<u>caracteristicas_edificio</u>	<u>Código de característica específica del edificio</u>	<u>SMALLINT</u>	=	<u>false</u>	<u>false</u>	<u>false</u>	<u>catalogos.id_catalogo</u>	Ej: 1 = Irregularidad vertical severa, 4 = Pre-código sísmico
<u>resultado_base</u>	<u>Puntaje base conforme al tipo y características de la edificación</u>	<u>NUMERIC</u>	<u>8,2</u>	<u>false</u>	<u>false</u>	<u>false</u>	=	Ej: 3.50
<u>modificador_puntuacion</u>	<u>Valor que se suma o resta al puntaje base según la característica</u>	<u>NUMERIC</u>	<u>8,2</u>	<u>false</u>	<u>false</u>	<u>false</u>	=	Ej: -0.50 (reduce), +0.30 (aumenta)
<u>minima_puntuacion</u>	<u>Puntaje mínimo permitido para considerar aceptable la edificación</u>	<u>NUMERIC</u>	<u>8,2</u>	<u>false</u>	<u>false</u>	<u>false</u>	=	Ej: 2.50

Tabla 25: matriz_puntuacion

3. Resultados

Entity Name: resultados_inspeccion

Entity Description: Contiene los datos calculados de cada inspección FEMA P-154, incluyendo puntaje final, clasificación de riesgo, recomendaciones y vínculo con el reporte PDF.

Column Name	Column Description	Data Type	Length	PK	Nullable	Unique	FK	Restricciones / Ejemplo
id_resultado	Identificador único del resultado	UUID	-	true	false	true	-	Autoincremental
inspeccion_id	Inspección asociada	INT	-	false	false	false	inspecciones.id_inspeccion	Debe existir en inspecciones
puntaje_final	Puntaje final calculado según metodología FEMA P-154	NUMERIC	8,2	false	false	false	-	Ej: 3.25
clasificacion_riesgo	Clasificación final de riesgo	VARCHAR	20	false	false	false	catalogos.id_catalogo	Ej: "Bajo", "Medio", "Alto"
cumple_minima	Indica si cumple la puntuación mínima FEMA	BOOLEAN	-	false	false	false	-	Valores: true/false

factores_modificadores	Factores que aumentaron/disminuyeron el puntaje	TEXT	-	false	true	false	-	Ej: "Irregularidad vertical, Suelo tipo E"
peligros_detectados	Resumen de peligros adicionales observados	TEXT	-	false	true	false	-	Ej: "Golpeteo entre edificios, riesgo por suelos blandos"
recomendaciones_finales	Recomendaciones consolidadas según riesgo y peligros detectados	TEXT	-	false	true	false	-	Ej: "Refuerzo estructural urgente, inspección nivel 2"
fecha_generacion	Fecha y hora en que se generó el resultado	timestampz	-	false	false	false	-	DEFAULT now()
archivo_reporte_pdf	Ruta o archivo binario del reporte PDF generado	BYTEA	-	false	true	false	-	Formato binario PDF
formato_archivo	Formato del archivo de reporte	VARCHAR	10	false	true	false	-	Ej: "PDF"

Tabla 26: : resultados De la inspeccion

4. Historial y Auditoría

Entity Name: historial_auditoria

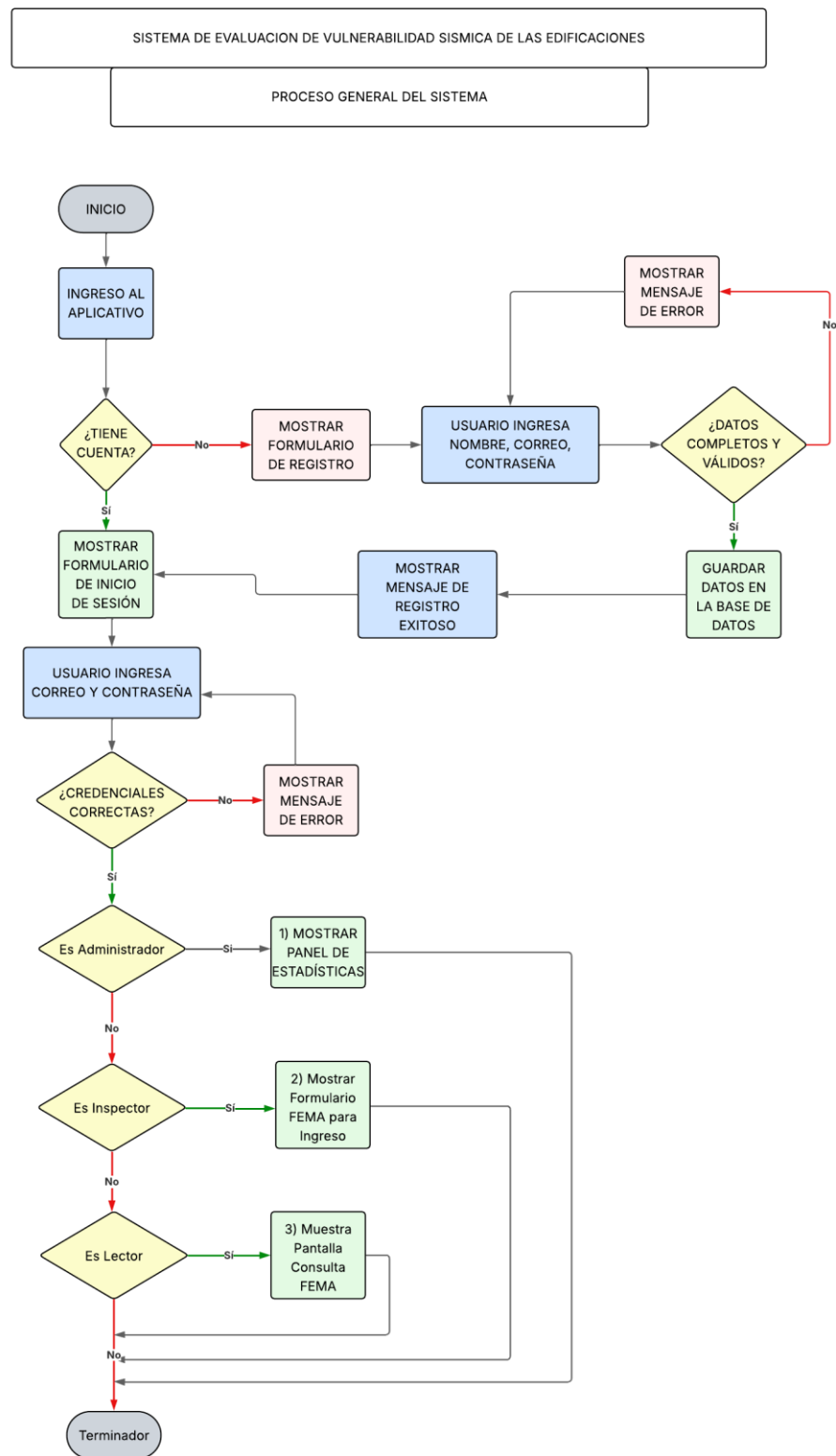
Entity Description: Registra cambios realizados en las entidades del sistema, incluyendo usuario que ejecutó la acción, fecha, entidad afectada y descripción del cambio.

Column Name	Column Description	Data Type	Length	PK	Nullable	Unique	FK	Restricciones / Ejemplo
id_auditoria	Identificador único del registro de auditoría	UUID	-	true	false	true	-	Autoincremental
usuario_id	Usuario que realizó la acción	INT	-	false	false	false	usuarios.id_usuario	Debe existir en usuarios
fecha_accion	Fecha y hora en que se realizó la acción	timestampz	-	false	false	false	-	DEFAULT now()
entidad_afectada	Nombre de la entidad (tabla) modificada	VARCHAR	100	false	false	false	-	Ej: "edificios", "inspecciones"
id_registro_afectado	Identificador del registro modificado	INT	-	false	false	false	-	Ej: 15
tipo_accion	Tipo de acción ejecutada	VARCHAR	20	false	false	false	catalogos.id_catalogo	Ej: "INSERT", "UPDATE", "DELETE"

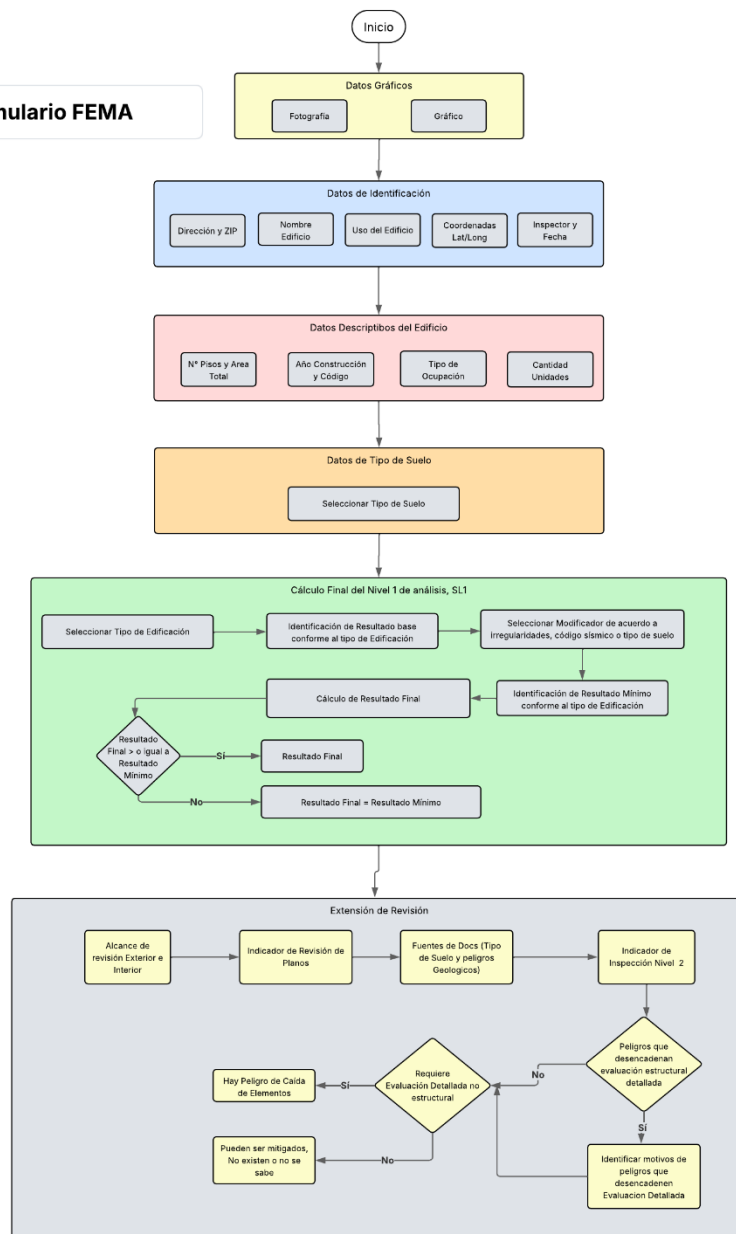
descripcion_cambio	Descripción detallada del cambio realizado	TEXT	-	false	true	false	-	Ej: "Se actualizó la dirección del edificio"
valores_anteriores	Valores previos de los campos modificados (formato JSON)	JSONB	-	false	true	false	-	Ej: {"direccion": "Av. Central 123", "telefono": "+593987654321"}
valores_nuevos	Valores actualizados de los campos modificados (formato JSON)	JSONB	-	false	true	false	-	Ej: {"direccion": "Av. Amazonas 456", "telefono": "+593912345678"}
ip_origen	Dirección IP desde la cual se realizó la acción	VARCHAR	45	false	true	false	-	Ej: "192.168.0.15"

Tabla 27: Historial_auditoria

11. Diagrama de flujos



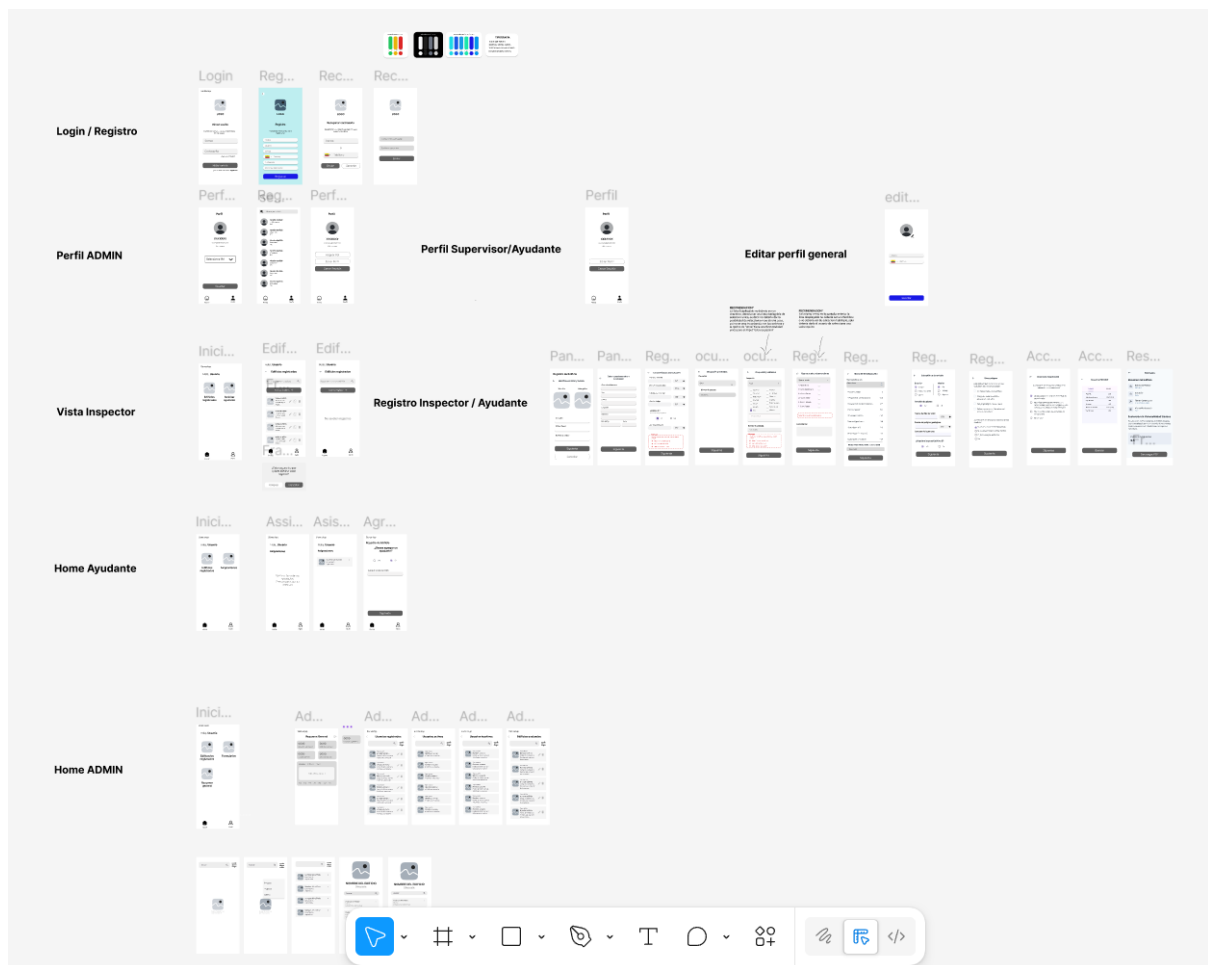
Flujo de Formulario FEMA



12. PROTOTIPOS DE PANTALLAS DEL APLIACTIVO

Enlace de figma donde se encuentra el prototipado:

<https://www.figma.com/design/JN0WS5ulUw3K6Lbw2ujmtD/Wirerfma-FEMA-P154?node-id=0-1&t=U4Ste6H0ATR87FyP-1>



Para mas información del acerca de cada pantalla visualizar el archivo adjunto.

1. Descripción de los prototipos de pantalla

https://1drv.ms/w/c/5533d696c998a3a2/Ed-vBOlvVZhFmFcK_bW_JC8BnDHTZf0XIm5LD8gsktTUIJg?e=BPBia2

13. Anexo de sugerencias y soluciones a problemas

Problemas de Sdk: Revisar la parte de variables de entorno.

Problemas con la compilación:

Abre el terminal del flutter y ejecuta el siguiente comando:

Flutter config --jdk-dir=*(ruta donde esta el jdk)*

instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. <https://aka.ms/PSWin>

```
C:\Users\User\AndroidStudioProjects\untitled1> flutter config --jdk-dir="C:\Program Files\Java\jdk-17"
```